



CUDA Programming Guide

Release 13.3

NVIDIA Corporation

May 27, 2026

Contents

1	Introduction to CUDA	3
1.1	Introduction	3
1.1.1	The Graphics Processing Unit	3
1.1.2	The Benefits of Using GPUs	3
1.1.3	Getting Started Quickly	4
1.2	Programming Model	4
1.2.1	Heterogeneous Systems	5
1.2.2	GPU Hardware Model	5
1.2.2.1	Thread Blocks and Grids	5
1.2.2.2	Warps and SIMT	7
1.2.2.3	Tile Programming in CUDA	11
1.2.3	GPU Memory	13
1.2.3.1	DRAM Memory in Heterogeneous Systems	13
1.2.3.2	On-Chip Memory in GPUs	14
1.2.3.3	Unified Memory	14
1.3	The CUDA platform	15
1.3.1	Compute Capability and Streaming Multiprocessor Versions	15
1.3.2	CUDA Toolkit and NVIDIA Driver	15
1.3.2.1	CUDA Runtime API and CUDA Driver API	15
1.3.3	Parallel Thread Execution (PTX)	16
1.3.4	Cubins and Fatbins	16
1.3.4.1	Binary Compatibility	16
1.3.4.2	PTX Compatibility	18
1.3.4.3	Just-in-Time Compilation	18
2	Programming GPUs in CUDA	19
2.1	Intro to CUDA C++	19
2.1.1	Compilation with NVCC	19
2.1.2	Kernels	19
2.1.2.1	Specifying Kernels	19
2.1.2.2	Launching Kernels	20
2.1.2.3	Thread and Grid Index Intrinsic	21
2.1.3	Memory in GPU Computing	23
2.1.3.1	Unified Memory	23
2.1.3.2	Explicit Memory Management	24
2.1.3.3	Memory Management and Application Performance	26
2.1.4	Synchronizing CPU and GPU	26
2.1.5	Putting it All Together	26
2.1.6	Runtime Initialization	32
2.1.7	Error Checking in CUDA	32
2.1.7.1	Error State	33
2.1.7.2	Asynchronous Errors	34
2.1.7.3	CUDA_LOG_FILE	34

2.1.8	Device and Host Functions	35
2.1.9	Variable Specifiers	35
2.1.9.1	Detecting Device Compilation	35
2.1.10	Thread Block Clusters	36
2.1.10.1	Launching with Clusters in Triple Chevron Notation	36
2.2	Intro to CUDA Python	37
2.2.1	CUDA Python Ecosystem	37
2.2.1.1	Using CUDA Libraries in Python	38
2.2.1.2	Scope of This Chapter	38
2.2.1.3	Getting Setup	38
2.2.1.4	Running CUDA Python Applications	38
2.2.2	SIMT Kernels in Python	39
2.2.2.1	Specifying Kernels	39
2.2.2.2	Launching Kernels	39
2.2.2.3	Thread and Grid Index Intrinsic	40
2.2.3	Memory in GPU computing	41
2.2.3.1	Instantiating arrays on the GPU	41
2.2.3.2	Copying Arrays between the Host and GPU memory	42
2.2.3.3	The ndarray object type	42
2.2.4	Synchronizing the CPU and the GPU	42
2.2.5	Putting it All Together	43
2.2.6	Error Checking in CUDA Python	44
2.3	Writing SIMT Kernels	45
2.3.1	Basics of SIMT	45
2.3.2	Thread Hierarchy	46
2.3.2.1	Thread Block Synchronization	46
2.3.3	GPU Device Memory Spaces	47
2.3.3.1	Global Memory	47
2.3.3.2	Shared Memory	48
2.3.3.3	Registers	51
2.3.3.4	Local Memory	51
2.3.3.5	Constant Memory	52
2.3.3.6	Caches	53
2.3.3.7	Texture and Surface Memory	53
2.3.3.8	Distributed Shared Memory	54
2.3.4	Memory Performance	56
2.3.4.1	Coalesced Global Memory Access	56
2.3.4.2	Shared Memory Access Patterns	60
2.3.5	Atomics	67
2.3.5.1	C++ std::atomic-like Atomics	68
2.3.5.2	Memory Atomics in Python	69
2.3.6	Cooperative Groups	70
2.3.7	Kernel Launch and Occupancy	70
2.4	Writing Tile Kernels	72
2.4.1	Kernel and Function Declarations	73
2.4.2	Launching Kernels	74
2.4.2.1	Grid-Sizing Pattern	74
2.4.3	Querying Block Position	75
2.4.4	Creating Tiles	76
2.4.5	Compile-Time Constants	77
2.4.5.1	Python Constant[T]	77
2.4.5.2	C++ integral_constant and _ic Literals	77
2.4.6	Loading and Storing Tiles	78
2.4.6.1	Tile-Space Loads and Stores	79

2.4.6.2	Gather and Scatter	83
2.4.7	Control Flow	85
2.4.7.1	Loops	85
2.4.7.2	Conditionals	87
2.4.8	Element-wise Arithmetic and Broadcasting	88
2.4.8.1	Broadcasting	88
2.4.8.2	Arithmetic Operators	88
2.4.9	Tile Primitives	90
2.4.9.1	Matrix Multiply	90
2.4.9.2	Reductions and Scans	92
2.4.9.3	Transpose and Permutation	92
2.4.9.4	Element-wise Selection	93
2.4.9.5	Mathematical Functions	93
2.4.10	Atomic Memory Operations	94
2.4.10.1	Cross-block Contention	94
2.4.10.2	Intra-block Contention	95
2.4.10.3	Supported Atomic Operations	96
2.4.11	Optimization Hints	97
2.4.11.1	C++ – the <code>cutile::hint</code> Attribute	97
2.4.11.2	Python – Decorator Arguments and Call-Site Keywords	98
2.4.11.3	Hint Kinds	99
2.4.12	C++ Performance Tips	100
2.4.12.1	Use <code>__restrict__</code> Pointers for Arrays in Memory	100
2.4.12.2	Mark Array Pointers as 16-Byte Aligned	101
2.4.12.3	Prefer <code>ct::partition_view</code> for Memory Access	101
2.4.12.4	Use <code>ct::irange</code> for Bounded Loops	101
2.5	Asynchronous Execution	101
2.5.1	What is Asynchronous Concurrent Execution?	101
2.5.2	CUDA Streams	103
2.5.2.1	Creating and Destroying CUDA Streams	103
2.5.2.2	Launching Kernels in CUDA Streams	103
2.5.2.3	Launching Memory Transfers in CUDA Streams	104
2.5.2.4	Stream Synchronization	104
2.5.3	CUDA Events	105
2.5.3.1	Creating and Destroying CUDA Events	105
2.5.3.2	Inserting Events into CUDA Streams	105
2.5.3.3	Timing Operations in CUDA Streams	106
2.5.3.4	Checking the Status of CUDA Events	107
2.5.4	Callback Functions from Streams	109
2.5.4.1	Using <code>cudaStreamAddCallback()</code>	110
2.5.4.2	Asynchronous Error Handling	110
2.5.5	CUDA Stream Ordering	111
2.5.6	Blocking and non-blocking streams and the default stream	112
2.5.6.1	Legacy Default Stream	112
2.5.6.2	Per-thread Default Stream	113
2.5.7	Explicit Synchronization	113
2.5.8	Implicit Synchronization	113
2.5.9	Miscellaneous and Advanced topics	113
2.5.9.1	Stream Prioritization	113
2.5.9.2	Introduction to CUDA Graphs with Stream Capture	114
2.5.10	Summary of Asynchronous Execution	115
2.6	Unified and System Memory	116
2.6.1	Unified Virtual Address Space	117
2.6.2	Unified Memory	117

2.6.2.1	Unified Memory Paradigms	117
2.6.2.2	Full Unified Memory Feature Support	120
2.6.2.3	Limited Unified Memory Support	121
2.6.2.4	Memory Advise and Prefetch	122
2.6.3	Page-Locked Host Memory	122
2.6.3.1	Mapped Memory	123
2.6.4	Summary	125
2.7	NVCC: The NVIDIA CUDA Compiler	126
2.7.1	CUDA Source Files and Headers	126
2.7.2	NVCC Compilation Workflow	126
2.7.3	NVCC Basic Usage	127
2.7.3.1	NVCC PTX and Cubin Generation	129
2.7.3.2	Host Code Compilation Notes	130
2.7.3.3	Separate Compilation of GPU Code	130
2.7.4	Common Compiler Options	131
2.7.4.1	Language Features	131
2.7.4.2	Debugging Options	131
2.7.4.3	Optimization Options	132
2.7.4.4	Link-Time Optimization (LTO)	132
2.7.4.5	Profiling Options	133
2.7.4.6	Fatbin Compression	133
2.7.4.7	Compiler Performance Controls	133
3	Advanced CUDA	135
3.1	Advanced CUDA APIs and Features	135
3.1.1	cudaLaunchKernelEx	135
3.1.2	Launching Clusters:	135
3.1.2.1	Launching with Clusters using cudaLaunchKernelEx	136
3.1.2.2	Blocks as Clusters	137
3.1.3	More on Streams and Events	138
3.1.3.1	Stream Priorities	139
3.1.3.2	Explicit Synchronization	139
3.1.3.3	Implicit Synchronization	139
3.1.4	Programmatic Dependent Kernel Launch	140
3.1.5	Batched Memory Transfers	142
3.1.6	Environment Variables	145
3.2	Advanced Kernel Programming	146
3.2.1	Using PTX	146
3.2.2	Hardware Implementation	147
3.2.2.1	SIMT Execution Model	147
3.2.2.2	Hardware Multithreading	148
3.2.2.3	Asynchronous Execution Features	149
3.2.3	Thread Scopes	150
3.2.4	Advanced Synchronization Primitives	151
3.2.4.1	Scoped Atomics	151
3.2.4.2	Asynchronous Barriers	154
3.2.4.3	Pipelines	159
3.2.5	Asynchronous Data Copies	159
3.2.6	Configuring L1/Shared Memory Balance	163
3.3	The CUDA Driver API	164
3.3.1	Context	167
3.3.2	Module	168
3.3.3	Kernel Execution	169
3.3.4	Interoperability between Runtime and Driver APIs	171

3.4	Programming Systems with Multiple GPUs	171
3.4.1	Multi-Device Context and Execution Management	172
3.4.1.1	Device Enumeration	173
3.4.1.2	Device Selection	173
3.4.1.3	Multi-Device Stream, Event, and Memory Copy Behavior	173
3.4.2	Multi-Device Peer-to-Peer Transfers and Memory Access	174
3.4.2.1	Peer-to-Peer Memory Transfers	174
3.4.2.2	Peer-to-Peer Memory Access	175
3.4.2.3	Peer-to-Peer Memory Consistency	175
3.4.2.4	Multi-Device Managed Memory	176
3.4.2.5	Host IOMMU Hardware, PCI Access Control Services, and VMs	176
3.5	A Tour of CUDA Features	176
3.5.1	Improving Kernel Performance	176
3.5.1.1	Asynchronous Barriers	176
3.5.1.2	Asynchronous Data Copies and the Tensor Memory Accelerator (TMA)	176
3.5.1.3	Pipelines	177
3.5.1.4	Work Stealing with Cluster Launch Control	177
3.5.2	Improving Latencies	177
3.5.2.1	Green Contexts	177
3.5.2.2	Stream-Ordered Memory Allocation	177
3.5.2.3	CUDA Graphs	178
3.5.2.4	Programmatic Dependent Launch	178
3.5.2.5	Lazy Loading	178
3.5.3	Functionality Features	178
3.5.3.1	Extended GPU Memory	178
3.5.3.2	Dynamic Parallelism	178
3.5.4	CUDA Interoperability	179
3.5.4.1	CUDA Interoperability with other APIs	179
3.5.4.2	Interprocess Communication	179
3.5.5	Fine-Grained Control	179
3.5.5.1	Virtual Memory Management	179
3.5.5.2	Driver Entry Point Access	179
3.5.5.3	Error Log Management	180
4	CUDA Features	181
4.1	Unified Memory	181
4.1.1	Unified Memory on Devices with Full CUDA Unified Memory Support	181
4.1.1.1	Unified Memory: In-Depth Examples	182
4.1.1.2	Performance Tuning	185
4.1.2	Unified Memory on Devices with only CUDA Managed Memory Support	195
4.1.3	Unified Memory on Windows, WSL, and Tegra	196
4.1.3.1	Multi-GPU	196
4.1.3.2	Coherency and Concurrency	197
4.1.3.3	Stream Associated Unified Memory	198
4.1.4	Performance Hints	201
4.1.4.1	Data Prefetching	202
4.1.4.2	Data Usage Hints	203
4.1.4.3	Memory Discarding	206
4.1.4.4	Querying Data Usage Attributes on Managed Memory	207
4.1.4.5	GPU Memory Oversubscription	208
4.2	CUDA Graphs	209
4.2.1	Graph Structure	209
4.2.1.1	Node Types	209
4.2.1.2	Edge Data	210

4.2.2	Building and Running Graphs	211
4.2.2.1	Graph Creation	211
4.2.2.2	Graph Instantiation	219
4.2.2.3	Graph Execution	219
4.2.3	Updating Instantiated Graphs	219
4.2.3.1	Whole Graph Update	220
4.2.3.2	Individual Node Update	222
4.2.3.3	Individual Node Enable	222
4.2.3.4	Graph Update Limitations	222
4.2.4	Conditional Graph Nodes	223
4.2.4.1	Conditional Handles	224
4.2.4.2	Conditional Node Body Graph Requirements	224
4.2.4.3	Conditional IF Nodes	224
4.2.4.4	Conditional WHILE Nodes	227
4.2.4.5	Conditional SWITCH Nodes	229
4.2.5	Graph Memory Nodes	231
4.2.5.1	Introduction	231
4.2.5.2	API Fundamentals	231
4.2.5.3	Optimized Memory Reuse	239
4.2.5.4	Performance Considerations	242
4.2.5.5	Physical Memory Footprint	244
4.2.5.6	Peer Access	244
4.2.6	Device Graph Launch	245
4.2.6.1	Device Graph Creation	246
4.2.6.2	Device Launch	247
4.2.7	Using Graph APIs	254
4.2.8	CUDA User Objects	254
4.3	Stream-Ordered Memory Allocator	256
4.3.1	Introduction	256
4.3.2	Memory Management	257
4.3.2.1	Allocating Memory	257
4.3.2.2	Freeing Memory	258
4.3.3	Memory Pools	258
4.3.3.1	Default/Implicit Pools	259
4.3.3.2	Explicit Pools	259
4.3.3.3	Device Accessibility for Multi-GPU Support	260
4.3.3.4	Enabling Memory Pools for IPC	260
4.3.4	Best Practices and Tuning	264
4.3.4.1	Query for Support	264
4.3.4.2	Physical Page Caching Behavior	264
4.3.4.3	Resource Usage Statistics	265
4.3.4.4	Memory Reuse Policies	266
4.3.4.5	Synchronization API Actions	268
4.3.5	Addendums	268
4.3.5.1	cudaMemcpyAsync Current Context/Device Sensitivity	268
4.3.5.2	cudaPointerGetAttributes Query	268
4.3.5.3	cudaGraphAddMemsetNode	268
4.3.5.4	Pointer Attributes	268
4.3.5.5	CPU Virtual Memory	268
4.4	Cooperative Groups	268
4.4.1	Introduction	269
4.4.2	Cooperative Group Handle & Member Functions	269
4.4.3	Default Behavior / Groupless Execution	269
4.4.3.1	Create Implicit Group Handles As Early As Possible	270

4.4.3.2	Only Pass Group Handles by Reference	270
4.4.4	Creating Cooperative Groups	270
4.4.4.1	Avoiding Group Creation Hazards	271
4.4.5	Synchronization	271
4.4.5.1	Sync	271
4.4.5.2	Barriers	271
4.4.6	Collective Operations	272
4.4.6.1	Reduce	272
4.4.6.2	Scans	273
4.4.6.3	Invoke One	274
4.4.7	Asynchronous Data Movement	274
4.4.7.1	Memcpy Async Alignment Requirements	275
4.4.8	Large Scale Groups	275
4.4.8.1	When to use <code>cudaLaunchCooperativeKernel</code>	275
4.5	Programmatic Dependent Launch and Synchronization	276
4.5.1	Background	276
4.5.2	API Description	277
4.5.3	Use in CUDA Graphs	278
4.6	Green Contexts	279
4.6.1	Motivation / When to Use	279
4.6.2	Green Contexts: Ease of use	281
4.6.3	Green Contexts: Device Resource and Resource Descriptor	283
4.6.4	Green Context Creation Example	285
4.6.4.1	Step 1: Get available GPU resources	285
4.6.4.2	Step 2: Partition SM resources	286
4.6.4.3	Step 2 (continued): Add workqueue resources	294
4.6.4.4	Step 3: Create a Resource Descriptor	294
4.6.4.5	Step 4: Create a Green Context	295
4.6.5	Green Contexts - Launching work	295
4.6.6	Additional Execution Contexts APIs	298
4.6.7	Green Contexts Example	299
4.7	Lazy Loading	300
4.7.1	Introduction	300
4.7.2	Change History	300
4.7.3	Requirements for Lazy Loading	300
4.7.3.1	CUDA Runtime Version Requirement	301
4.7.3.2	CUDA Driver Version Requirement	301
4.7.3.3	Compiler Requirements	301
4.7.3.4	Kernel Requirements	301
4.7.4	Usage	301
4.7.4.1	Enabling & Disabling	301
4.7.4.2	Checking if Lazy Loading is Enabled at Runtime	301
4.7.4.3	Forcing a Module to Load Eagerly at Runtime	302
4.7.5	Potential Hazards	302
4.7.5.1	Impact on Concurrent Kernel Execution	302
4.7.5.2	Large Memory Allocations	302
4.7.5.3	Impact on Performance Measurements	302
4.8	Error Log Management	302
4.8.1	Background	303
4.8.2	Activation	303
4.8.3	Output	303
4.8.4	API Description	303
4.8.5	Limitations and Known Issues	304
4.9	Asynchronous Barriers	304

4.9.1	Initialization	305
4.9.2	A Barrier's Phase: Arrival, Countdown, Completion, and Reset	306
4.9.2.1	Warp Entanglement	307
4.9.3	Explicit Phase Tracking	307
4.9.4	Early Exit	310
4.9.5	Completion Function	312
4.9.6	Tracking Asynchronous Memory Operations	315
4.9.7	Producer-Consumer Pattern Using Barriers	316
4.10	Pipelines	323
4.10.1	Initialization	323
4.10.2	Submitting Work	324
4.10.3	Consuming Work	324
4.10.4	Warp Entanglement	324
4.10.5	Early Exit	325
4.10.6	Tracking Asynchronous Memory Operations	326
4.10.7	Producer-Consumer Pattern using Pipelines	329
4.11	Asynchronous Data Copies	331
4.11.1	Using LDGSTS	331
4.11.1.1	Batching Loads in Conditional Code	332
4.11.1.2	Prefetching Data	337
4.11.1.3	Producer-Consumer Pattern Through Warp Specialization	343
4.11.2	Using the Tensor Memory Accelerator (TMA)	347
4.11.2.1	Using TMA to transfer one-dimensional arrays	348
4.11.2.2	Using TMA to transfer multi-dimensional arrays	355
4.11.3	Using STAS	372
4.12	Work Stealing with Cluster Launch Control	375
4.12.1	API Details	377
4.12.1.1	Thread Block Cancellation	377
4.12.1.2	Constraints on Thread Block Cancellation	378
4.12.2	Example: Vector-Scalar Multiplication	379
4.12.2.1	Use-case: Thread Blocks	379
4.12.2.2	Use-case: Thread Block Clusters	381
4.13	L2 Cache Control	383
4.13.1	L2 Cache Set-Aside for Persisting Accesses	383
4.13.2	L2 Policy for Persisting Accesses	384
4.13.3	L2 Access Properties	385
4.13.4	L2 Persistence Example	385
4.13.5	Reset L2 Access to Normal	387
4.13.6	Manage Utilization of L2 set-aside cache	387
4.13.7	Query L2 cache Properties	387
4.13.8	Control L2 Cache Set-Aside Size for Persisting Memory Access	387
4.14	Memory Synchronization Domains	388
4.14.1	Memory Fence Interference	388
4.14.2	Isolating Traffic with Domains	389
4.14.3	Using Domains in CUDA	389
4.15	Interprocess Communication	390
4.15.1	IPC using the Legacy Interprocess Communication API	391
4.15.2	IPC using the Virtual Memory Management API	392
4.16	Virtual Memory Management	392
4.16.1	Preliminaries	393
4.16.1.1	Definitions	393
4.16.1.2	Query for Support	394
4.16.2	API Overview	395
4.16.3	Unicast Memory Sharing	397

4.16.3.1	Allocate and Export	397
4.16.3.2	Share and Import	399
4.16.3.3	Reserve and Map	403
4.16.3.4	Access Rights	404
4.16.3.5	Releasing the Memory	404
4.16.4	Multicast Memory Sharing	405
4.16.4.1	Allocating Multicast Objects	405
4.16.4.2	Add Devices to Multicast Objects	406
4.16.4.3	Bind Memory to Multicast Objects	406
4.16.4.4	Use Multicast Mappings	406
4.16.5	Advanced Configuration	407
4.16.5.1	Memory Type	407
4.16.5.2	Compressible Memory	407
4.16.5.3	Virtual Aliasing Support	408
4.16.5.4	OS-Specific Handle Details for IPC	409
4.17	Extended GPU Memory	410
4.17.1	Preliminaries	410
4.17.1.1	EGM Platforms: System topology	410
4.17.1.2	Socket Identifiers: What are they? How to access them?	411
4.17.1.3	Allocators and EGM support	411
4.17.1.4	Memory management extensions to current APIs	411
4.17.2	Using the EGM Interface	412
4.17.2.1	Single-Node, Single-GPU	412
4.17.2.2	Single-Node, Multi-GPU	412
4.17.2.3	Multi-Node, Multi-GPU	414
4.18	CUDA Dynamic Parallelism	415
4.18.1	Introduction	415
4.18.1.1	Overview	415
4.18.2	Execution Environment	415
4.18.2.1	Parent and Child Grids	416
4.18.2.2	Scope of CUDA Primitives	416
4.18.2.3	Streams and Events	416
4.18.2.4	Ordering and Concurrency	417
4.18.3	Memory Coherence and Consistency	417
4.18.3.1	Global Memory	418
4.18.3.2	Mapped Memory	418
4.18.3.3	Shared and Local Memory	419
4.18.3.4	Local Memory	419
4.18.4	Programming Interface	420
4.18.4.1	Basics	420
4.18.4.2	C++ Language Interface for CDP	421
4.18.5	Programming Guidelines	422
4.18.5.1	Performance	422
4.18.5.2	Implementation Restrictions and Limitations	422
4.18.5.3	Compatibility and Interoperability	423
4.18.6	Device-side Launch from PTX	423
4.18.6.1	Kernel Launch APIs	424
4.18.6.2	Parameter Buffer Layout	425
4.19	CUDA Interoperability with APIs	425
4.19.1	Graphics Interoperability	425
4.19.1.1	OpenGL Interoperability	426
4.19.1.2	Direct3D Interoperability	430
4.19.1.3	Interoperability in a Scalable Link Interface (SLI) configuration	432
4.19.2	External resource interoperability	433

4.19.2.1	Vulkan interoperability	433
4.19.2.2	Direct3D Interoperability	446
4.19.2.3	NVIDIA Software Communication Interface Interoperability (NVSCI)	453
4.20	Driver Entry Point Access	458
4.20.1	Introduction	458
4.20.2	Driver Function Typedefs	459
4.20.3	Driver Function Retrieval	460
4.20.3.1	Using the Driver API	460
4.20.3.2	Using the Runtime API	461
4.20.3.3	Retrieve Per-thread Default Stream Versions	461
4.20.3.4	Access New CUDA features	462
4.20.4	Guidelines for cuGetProcAddress	463
4.20.4.1	Guidelines for Runtime API Usage	463
4.20.5	Determining cuGetProcAddress Failure Reasons	463
5	Technical Appendices	465
5.1	Compute Capabilities	465
5.1.1	Obtain the GPU Compute Capability	465
5.1.2	Feature Availability	466
5.1.2.1	Architecture-Specific Features	466
5.1.2.2	Family-Specific Features	466
5.1.2.3	Feature Set Compiler Targets	466
5.1.3	Features and Technical Specifications	467
5.2	CUDA Environment Variables	473
5.2.1	Device Enumeration and Properties	473
5.2.1.1	CUDA_VISIBLE_DEVICES	473
5.2.1.2	CUDA_DEVICE_ORDER	474
5.2.1.3	CUDA_MANAGED_FORCE_DEVICE_ALLOC	474
5.2.2	JIT Compilation	474
5.2.2.1	CUDA_CACHE_DISABLE	474
5.2.2.2	CUDA_CACHE_PATH	475
5.2.2.3	CUDA_CACHE_MAXSIZE	475
5.2.2.4	CUDA_FORCE_PTX_JIT and CUDA_FORCE_JIT	475
5.2.2.5	CUDA_DISABLE_PTX_JIT and CUDA_DISABLE_JIT	476
5.2.2.6	CUDA_FORCE_PRELOAD_LIBRARIES	476
5.2.3	Execution	476
5.2.3.1	CUDA_LAUNCH_BLOCKING	476
5.2.3.2	CUDA_DEVICE_MAX_CONNECTIONS	477
5.2.3.3	CUDA_DEVICE_MAX_COPY_CONNECTIONS	477
5.2.3.4	CUDA_SCALE_LAUNCH_QUEUES	477
5.2.3.5	CUDA_GRAPHS_USE_NODE_PRIORITY	478
5.2.3.6	CUDA_DEVICE_WAITS_ON_EXCEPTION	478
5.2.3.7	CUDA_DEVICE_DEFAULT_PERSISTING_L2_CACHE_PERCENTAGE_LIMIT	478
5.2.3.8	CUDA_DISABLE_PERF_BOOST	479
5.2.3.9	CUDA_AUTO_BOOST [[deprecated]]	479
5.2.4	Module Loading	479
5.2.4.1	CUDA_MODULE_LOADING	479
5.2.4.2	CUDA_MODULE_DATA_LOADING	480
5.2.4.3	CUDA_BINARY_LOADER_THREAD_COUNT	480
5.2.5	CUDA Error Log Management	480
5.2.5.1	CUDA_LOG_FILE	480
5.3	C++ Language Support	481
5.3.1	C++11 Language Features	481
5.3.2	C++14 Language Features	484

5.3.3	C++17 Language Features	484
5.3.4	C++20 Language Features	486
5.3.5	C++23 Language Features	488
5.3.6	CUDA C++ Standard Library	490
5.3.7	C Standard Library Functions	491
5.3.7.1	clock() and clock64()	491
5.3.7.2	printf()	491
5.3.7.3	memcpy() and memset()	495
5.3.7.4	malloc() and free()	495
5.3.7.5	alloca()	499
5.3.8	Lambda Expressions	499
5.3.8.1	Lambda Expressions and __global__ Function Parameters	500
5.3.8.2	Extended Lambdas	501
5.3.8.3	Extended Lambda Type Traits	502
5.3.8.4	Extended Lambda Restrictions	503
5.3.8.5	Host-Device Lambda Optimization Notes	513
5.3.8.6	*this Capture By-Value	513
5.3.8.7	Argument Dependent Lookup (ADL)	515
5.3.9	Polymorphic Function Wrappers	516
5.3.10	C/C++ Language Restrictions	519
5.3.10.1	Unsupported Features	519
5.3.10.2	Namespace Reservations	519
5.3.10.3	Pointers and Memory Addresses	520
5.3.10.4	Variables	520
5.3.10.5	Functions	526
5.3.10.6	Classes	528
5.3.10.7	Templates	532
5.3.10.8	Restrictions in Tile Code	533
5.3.11	C++11 Restrictions	533
5.3.11.1	inline Namespaces	533
5.3.11.2	inline Unnamed Namespaces	534
5.3.11.3	constexpr Functions	534
5.3.11.4	constexpr Variables	538
5.3.11.5	__global__ Variadic Template	539
5.3.11.6	Defaulted Functions = default	540
5.3.11.7	[cuda::]std::initializer_list	541
5.3.11.8	[cuda::]std::move, [cuda::]std::forward	541
5.3.12	C++14 Restrictions	541
5.3.12.1	Functions with Deduced Return Type	541
5.3.12.2	Variable Templates	543
5.3.13	C++17 Restrictions	543
5.3.13.1	inline Variables	543
5.3.13.2	Structured Binding	544
5.3.14	C++20 Restrictions	544
5.3.14.1	Three-way Comparison Operator	544
5.3.14.2	constexpr Functions	545
5.3.15	C++23 Restrictions	545
5.3.15.1	Equality Operator (P2468R2)	545
5.4	C/C++ Language Extensions	545
5.4.1	Function and Variable Annotations	545
5.4.1.1	Execution Space Specifiers	545
5.4.1.2	Memory Space Specifiers	546
5.4.1.3	Inlining Specifiers	551
5.4.1.4	__restrict__ Pointers	551

5.4.1.5	<code>__grid_constant__</code> Parameters	552
5.4.1.6	Annotation Summary	553
5.4.2	Built-in Types and Variables	554
5.4.2.1	Host Compiler Type Extensions	554
5.4.2.2	Built-in Variables	554
5.4.2.3	Built-in Types	555
5.4.3	Kernel Configuration	557
5.4.3.1	Thread Block Cluster	558
5.4.3.2	Launch Bounds	558
5.4.3.3	Maximum Number of Registers per Thread	560
5.4.4	Synchronization Primitives	561
5.4.4.1	Thread Block Synchronization Functions	561
5.4.4.2	Warp Synchronization Function	564
5.4.4.3	Memory Fence Functions	565
5.4.5	Atomic Functions	569
5.4.5.1	Legacy Atomic Functions	570
5.4.5.2	Built-in Atomic Functions	575
5.4.6	Warp Functions	581
5.4.6.1	Warp Active Mask	582
5.4.6.2	Warp Vote Functions	582
5.4.6.3	Warp Match Functions	583
5.4.6.4	Warp Reduce Functions	583
5.4.6.5	Warp Shuffle Functions	584
5.4.6.6	Warp <code>__sync</code> Intrinsic Constraints	589
5.4.7	CUDA-Specific Macros	591
5.4.7.1	<code>__CUDA_ARCH__</code>	591
5.4.7.2	<code>__CUDA_ARCH_SPECIFIC__</code> and <code>__CUDA_ARCH_FAMILY_SPECIFIC__</code>	594
5.4.7.3	CUDA Feature Testing Macros	594
5.4.7.4	<code>__nv_pure__</code> Attribute	595
5.4.8	CUDA-Specific Functions	595
5.4.8.1	Address Space Predicate Functions	595
5.4.8.2	Address Space Conversion Functions	595
5.4.8.3	Low-Level Load and Store Functions	597
5.4.8.4	<code>__trap()</code>	597
5.4.8.5	<code>__nanosleep()</code>	597
5.4.8.6	Dynamic Programming eXtension (DPX) Instructions	598
5.4.9	Compiler Optimization Hints	600
5.4.9.1	<code>#pragma unroll</code>	600
5.4.9.2	<code>__builtin_assume_aligned()</code>	601
5.4.9.3	<code>__builtin_assume()</code> and <code>__assume()</code>	602
5.4.9.4	<code>__builtin_expect()</code>	602
5.4.9.5	<code>__builtin_unreachable()</code>	602
5.4.9.6	Custom ABI Pragmas	603
5.4.9.7	MMA Throughput Pragma	604
5.4.10	Debugging and Diagnostics	605
5.4.10.1	Assertion	605
5.4.10.2	Breakpoint Function	606
5.4.10.3	Diagnostic Pragmas	606
5.4.11	Warp Matrix Functions	607
5.4.11.1	Description	607
5.4.11.2	Alternate Floating Point	609
5.4.11.3	Double Precision	609
5.4.11.4	Sub-byte Operations	610
5.4.11.5	Restrictions	611

5.4.11.6	Element Types and Matrix Sizes	611
5.4.11.7	Example	613
5.5	Floating-Point Computation	613
5.5.1	Floating-Point Introduction	613
5.5.1.1	Floating-Point Format	613
5.5.1.2	Normal and Subnormal Values	615
5.5.1.3	Special Values	615
5.5.1.4	Associativity	616
5.5.1.5	Fused Multiply-Add (FMA)	617
5.5.1.6	Dot Product Example	619
5.5.1.7	Rounding	619
5.5.1.8	Notes on Host/Device Computation Accuracy	620
5.5.2	Floating-Point Data Types	621
5.5.3	CUDA and IEEE-754 Compliance	623
5.5.4	CUDA and C/C++ Compliance	624
5.5.5	Floating-Point Functionality Exposure	625
5.5.6	Built-In Arithmetic Operators	628
5.5.7	CUDA C++ Mathematical Standard Library Functions	629
5.5.7.1	Basic Operations	629
5.5.7.2	Exponential Functions	630
5.5.7.3	Power Functions	631
5.5.7.4	Trigonometric Functions	632
5.5.7.5	Hyperbolic Functions	632
5.5.7.6	Error and Gamma Functions	633
5.5.7.7	Nearest Integer Floating-Point Operations	634
5.5.7.8	Floating-Point Manipulation Functions	634
5.5.7.9	Classification and Comparison	635
5.5.8	Non-Standard CUDA Mathematical Functions	636
5.5.9	Intrinsic Functions	638
5.5.9.1	Basic Intrinsic Functions	638
5.5.9.2	Single-Precision-Only Intrinsic Functions	639
5.5.9.3	--use_fast_math Effect	640
5.5.10	References	641
5.6	Device-Callable APIs and Intrinsics	641
5.6.1	Memory Barrier Primitives Interface	641
5.6.1.1	Data Types	642
5.6.1.2	Memory Barrier Primitives API	642
5.6.2	Pipeline Primitives Interface	643
5.6.2.1	memcpy_async Primitive	643
5.6.2.2	Commit Primitive	644
5.6.2.3	Wait Primitive	644
5.6.2.4	Arrive On Barrier Primitive	644
5.6.3	Cooperative Groups API	645
5.6.3.1	cooperative_groups.h	645
5.6.3.2	cooperative_groups/async.h	651
5.6.3.3	cooperative_groups/partition.h	653
5.6.3.4	cooperative_groups/reduce.h	655
5.6.3.5	cooperative_groups/scan.h	657
5.6.3.6	cooperative_groups/sync.h	660
5.6.4	CUDA Device Runtime	663
5.6.4.1	Including Device Runtime API in CUDA Code	663
5.6.4.2	Memory in the CUDA Device Runtime	663
5.6.4.3	SM Id and Warp Id	666
5.6.4.4	Launch Setup APIs	666

5.6.4.5	Device Management	667
5.6.4.6	API Reference	667
5.6.4.7	API Errors and Launch Failures	669
5.6.4.8	Device Runtime Streams	669
5.6.4.9	ECC Errors	671
5.7	CUDA C++ Memory Model	671
5.7.1	Thread Scopes	671
5.7.1.1	Scope Relationships	672
5.7.2	Synchronization primitives	672
5.7.3	Atomicity	672
5.7.4	Data Races	673
5.7.5	Example: Message Passing	674
5.8	CUDA C++ Execution model	675
5.8.1	Host threads	676
5.8.2	Device threads	676
5.8.3	CUDA APIs	678
5.8.3.1	Dependencies	679
6	Notices	681
6.1	Notice	681
6.2	OpenCL	682
6.3	Trademarks	682

CUDA and the CUDA Programming Guide

CUDA is a parallel computing platform and programming model developed by NVIDIA that enables dramatic increases in computing performance by harnessing the power of the GPU. It allows developers to accelerate compute-intensive applications and is widely used in fields such as deep learning, scientific computing, and high-performance computing (HPC).

This CUDA Programming Guide is the official, comprehensive resource on the CUDA programming model and how to write code that executes on the GPU using the CUDA platform. This guide covers everything from the CUDA programming model and the CUDA platform to the details of language extensions and covers how to make use of specific hardware and software features. This guide provides a pathway for developers to learn CUDA if they are new, and also provides an essential resource for developers as they build applications using CUDA.

Organization of This Guide

Even for developers who primarily use libraries, frameworks, or DSLs, an understanding of the CUDA programming model and how GPUs execute code is valuable in knowing what is happening behind the layers of abstraction. This guide starts with a chapter on the *CUDA programming model* outside of any specific programming language which is applicable to anyone interested in understanding how CUDA works, even non-developers.

The guide is broken down into five primary parts:

- ▶ Part 1: Introduction and Programming Model Abstract
 - ▶ A language agnostic overview of the CUDA programming model as well as a brief tour of the CUDA platform.
 - ▶ This section is meant to be read by anyone wanting to understand GPUs and the concepts of executing code on GPUs, even if they are not developers.
- ▶ Part 2: Programming GPUs in CUDA
 - ▶ The basics of programming GPUs in C++ and Python.
 - ▶ This section is meant to be read by anyone wanting to get started in GPU programming.
 - ▶ This section is meant to be instructional, not complete, and teaches the most important and common parts of CUDA programming, including some common performance considerations.
- ▶ Part 3: Advanced CUDA
 - ▶ Introduces some more advance features of CUDA that enable both fine-grained control and more opportunities to maximize performance, including the use of multiple GPUs in a single application.
 - ▶ This section concludes with a *tour of the features covered in part 4* with a brief introduction to the purpose and function of each, sorted by when and why a developer may find each feature useful.
- ▶ Part 4: CUDA Features
 - ▶ This section contains complete coverage of specific CUDA features such as CUDA graphs, dynamic parallelism, interoperability with graphics APIs, and unified memory.
 - ▶ This section should be consulted when knowing the complete picture of a specific CUDA feature is needed. Where possible, care has been taken to introduce and motivate the features covered in this section in earlier sections.
- ▶ Part 5: Technical Appendices

- ▶ The technical appendices provide some reference documentation on CUDA's C++ high-level language support, hardware-specific specifications, and other technical specifications.
- ▶ This section is meant as technical reference for specific description of syntax, semantics, and technical behavior of elements of CUDA.

Parts 1-3 provide a guided learning experience for developers new to CUDA, though they also provide insight and updated information useful for CUDA developers of any experience level.

Parts 4 and 5 provide a wealth of information about specific features and detailed topics, and are intended to provide a curated, well-organized reference for developers needing to know more details as they write CUDA applications.

Chapter 1. Introduction to CUDA

1.1. Introduction

1.1.1. The Graphics Processing Unit

Born as a special-purpose processor for 3D graphics, the *Graphics Processing Unit* (GPU) started out as fixed-function hardware to accelerate parallel operations in real-time 3D rendering. Over successive generations, GPUs became more programmable. By 2003, some stages of the graphics pipeline became fully programmable, running custom code in parallel for each component of a 3D scene or an image.

In 2006, NVIDIA introduced the *Compute Unified Device Architecture* (CUDA) to enable any computational workload to use the throughput capability of GPUs independent of graphics APIs.

Since then, CUDA and GPU computing have been used to accelerate computational workloads of nearly every type, from scientific simulations such as fluid dynamics or energy transport to business applications like databases and analytics. Moreover, the capability and programmability of GPUs has been foundational to the advancement of new algorithms and technologies ranging from image classification to generative artificial intelligence such as diffusion or large language models.

1.1.2. The Benefits of Using GPUs

A GPU provides much higher instruction throughput and memory bandwidth than a CPU within a similar price and power envelope. Many applications leverage these capabilities to run significantly faster on the GPU than on the CPU (see [GPU Applications](#)). Other computing devices, like FPGAs, are also very energy efficient, but offer much less programming flexibility than GPUs.

GPUs and CPUs are designed with different goals in mind. While a CPU is designed to excel at executing a serial sequence of operations (called a thread) as fast as possible and can execute a few tens of these threads in parallel, a GPU is designed to excel at executing thousands of threads in parallel, trading off lower single-thread performance to achieve much greater total throughput.

GPUs are specialized for highly parallel computations and devote more transistors to data processing units, while CPUs dedicate more transistors to data caching and flow control. [Figure 1](#) shows an example distribution of chip resources for a CPU versus a GPU.

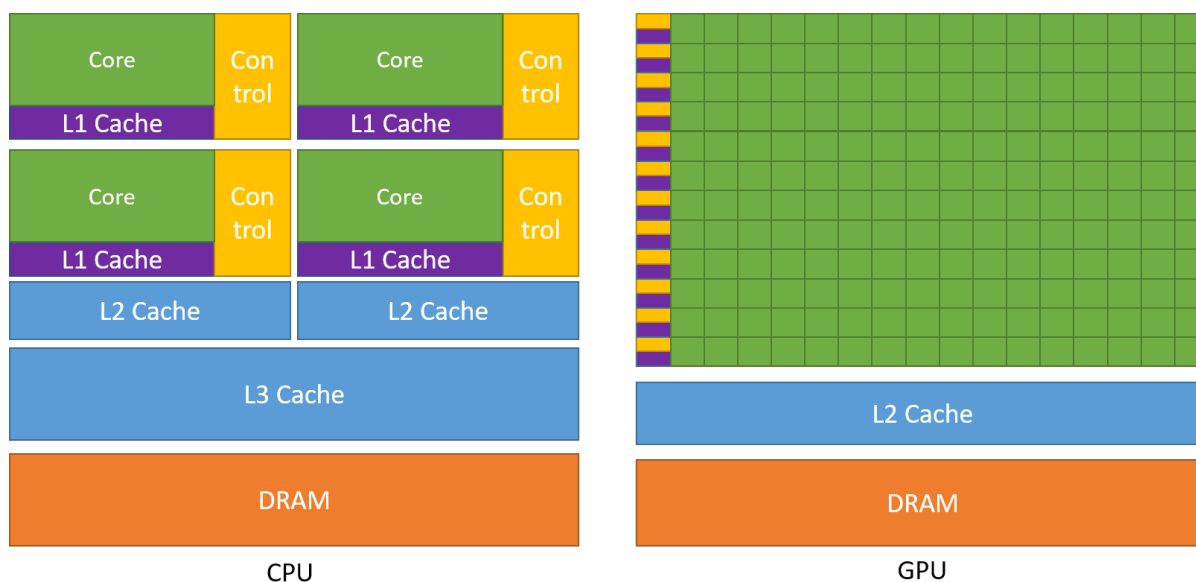


Figure 1: The GPU Devotes More Transistors to Data Processing

1.1.3. Getting Started Quickly

There are many ways to leverage the compute power provided by GPUs. This guide covers programming for the CUDA GPU platform in high-level languages such as C++. However, there are many ways to utilize GPUs in applications that do not require directly writing GPU code.

An ever-growing collection of algorithms and routines from a variety of domains is available through specialized libraries. When a library has already been implemented—especially those provided by NVIDIA—using it is often more productive and performant than reimplementing algorithms from scratch. Libraries like cuBLAS, cuFFT, cuDNN, and CUTLASS are just a few examples of libraries that help developers avoid reimplementing well-established algorithms. These libraries have the added benefit of being optimized for each GPU architecture, providing an ideal mix of productivity, performance, and portability.

There are also frameworks, particularly those used for artificial intelligence, that provide GPU-accelerated building blocks. Many of these frameworks achieve their acceleration by leveraging the GPU-accelerated libraries mentioned above.

Additionally, domain-specific languages (DSLs) such as NVIDIA's Warp or OpenAI's Triton compile to run directly on the CUDA platform. This provides an even higher-level method of programming GPUs than the high-level languages covered in this guide.

The [NVIDIA Accelerated Computing Hub](#) contains resources, examples, and tutorials to teach GPU and CUDA computing.

1.2. Programming Model

This chapter introduces the CUDA programming model at a high level and separate from any language. The terminology and concepts introduced here apply to CUDA in any supported programming language. Later chapters will illustrate these concepts in C++.

1.2.1. Heterogeneous Systems

The CUDA programming model assumes a heterogeneous computing system, which means a system that includes both GPUs and CPUs. The CPU and the memory directly connected to it are called the *host* and *host memory*, respectively. A GPU and the memory directly connected to it are referred to as the *device* and *device memory*, respectively. In some system-on-chip (SoC) systems, these may be part of a single package. In larger systems, there may be multiple CPUs or GPUs.

CUDA applications execute some part of their code on the GPU, but applications always start execution on the CPU. The host code, which is the code that runs on the CPU, can use CUDA APIs to copy data between the host memory and device memory, start code executing on the GPU, and wait for data copies or GPU code to complete. The CPU and GPU can both be executing code simultaneously, and best performance is usually found by maximizing utilization of both CPUs and GPUs.

The code an application executes on the GPU is referred to as *device code*, and a function that is invoked for execution on the GPU is, for historical reasons, called a *kernel*. The act of starting a kernel running is called *launching* the kernel. A kernel launch can be thought of as starting many threads executing the kernel code in parallel on the GPU. GPU threads operate similarly to threads on CPUs, though there are some differences important to both correctness and performance that will be covered in later sections (see [Section 3.2.2.1.1](#)).

1.2.2. GPU Hardware Model

Like any programming model, CUDA relies on a conceptual model of the underlying hardware. For the purposes of CUDA programming, the GPU can be considered to be a collection of *Streaming Multiprocessors* (SMs) which are organized into groups called *Graphics Processing Clusters* (GPCs). Each SM contains a local register file, a unified data cache, and a number of functional units that perform computations. The unified data cache provides the physical resources for *shared memory* and L1 cache. The allocation of the unified data cache to L1 and shared memory can be configured at runtime. The sizes of different types of memory and the number of functional units within an SM can vary across GPU architectures.

Note

The actual hardware layout of a GPU or the way it physically carries out the execution of the programming model may vary. These differences do not affect correctness of software written using the CUDA programming model.

1.2.2.1 Thread Blocks and Grids

When an application launches a kernel, it does so with many threads, often millions of threads. These threads are organized into blocks. A block of threads is referred to, perhaps unsurprisingly, as a *thread block*. Thread blocks are organized into a *grid*. All the thread blocks in a grid have the same size and dimensions. [Figure 3](#) shows an illustration of a grid of thread blocks.

Thread blocks and grids may be 1, 2, or 3 dimensional. These dimensions can simplify mapping of individual threads to units of work or data items.

When a kernel is launched, it is launched using a specific *execution configuration* which specifies the grid and thread block dimensions. The execution configuration may also include optional parameters such as cluster size, stream, and SM configuration settings, which will be introduced in later sections.

Using built-in variables, each thread executing the kernel can determine its location within its containing block and the location of its block within the containing grid. A thread can also use these

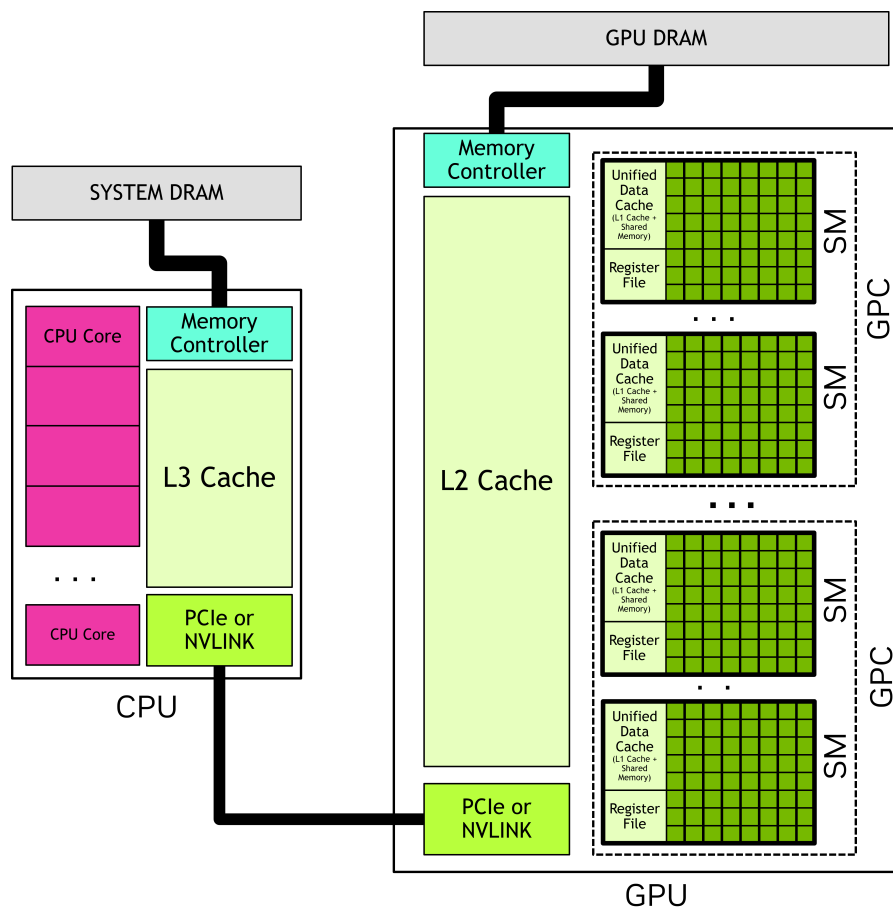


Figure 2: A GPU has many streaming multiprocessors (SMs), each of which contains many functional units. Graphics processing clusters (GPCs) are collections of SMs. A GPU is a set of GPCs connected to the GPU memory. A CPU typically has several cores and a memory controller which connects to the system memory. A CPU and a GPU are connected by an interconnect such as PCIe or NVLINK.

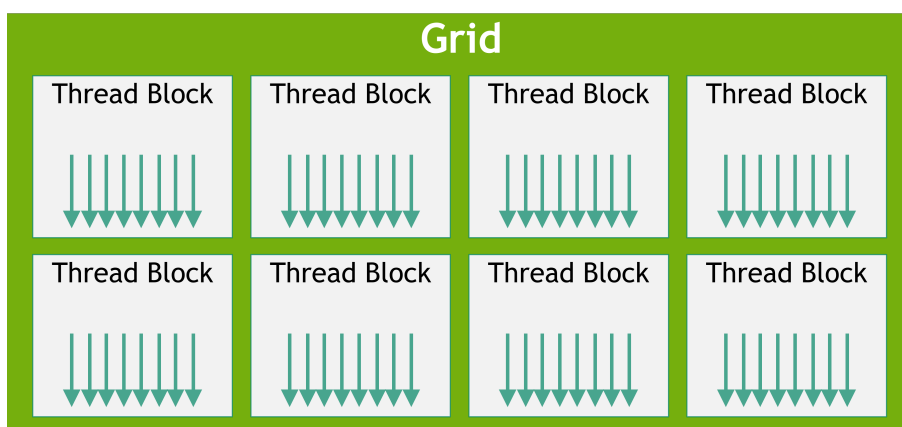


Figure 3: Grid of Thread Blocks. Each arrow represents a thread (the number of arrows is not representative of actual number of threads).

built-in variables to determine the dimensions of the thread blocks and the grid on which the kernel was launched. This gives each thread a unique identity among all the threads running the kernel. This identity is frequently used to determine what data or operations a thread is responsible for.

All threads of a thread block are executed in a single SM. This allows threads within a thread block to communicate and synchronize with each other efficiently. Threads within a thread block all have access to the on-chip shared memory, which can be used for exchanging information between threads of a thread block.

A grid may consist of millions of thread blocks, while the GPU executing the grid may have only tens or hundreds of SMs. All threads of a thread block are executed by a single SM and, in most cases¹, run to completion on that SM. There is no guarantee of scheduling between thread blocks, so a thread block cannot rely on results from other thread blocks, as they may not be able to be scheduled until that thread block has completed. Figure 4 shows an example of how thread blocks from a grid are assigned to an SM.

The CUDA programming model enables arbitrarily large grids to run on GPUs of any size, whether it has only one SM or thousands of SMs. To achieve this, the CUDA programming model, with some exceptions, requires that there be no data dependencies between threads in different thread blocks. That is, a thread should not depend on results from or synchronize with a thread in a different thread block of the same grid. All the threads within a thread block run on the same SM at the same time. Different thread blocks within the grid are scheduled among the available SMs and may be executed in any order. In short, the CUDA programming model requires that it be possible to execute thread blocks in any order, in parallel or in series.

1.2.2.1.1 Thread Block Clusters

In addition to thread blocks, GPUs with compute capability 9.0 and higher have an optional level of grouping called *clusters*. Clusters are a group of thread blocks which, like thread blocks and grids, can be laid out in 1, 2, or 3 dimensions. Figure 5 illustrates a grid of thread blocks that is also organized into clusters. Specifying clusters does not change the grid dimensions or the indices of a thread block within a grid.

Specifying clusters groups adjacent thread blocks into clusters and provides some additional opportunities for synchronization and communication at the cluster level. Specifically, all thread blocks in a cluster are executed in a single GPC. Figure 6 shows how thread blocks are scheduled to SMs in a GPC when clusters are specified. Because the thread blocks are scheduled simultaneously and within a single GPC, threads in different blocks but within the same cluster can communicate and synchronize with each other using software interfaces provided by *Cooperative Groups*. Threads in clusters can access the shared memory of all blocks in the cluster, which is referred to as *distributed shared memory*. The maximum size of a cluster is hardware dependent and varies between devices.

Figure 6 illustrates the how thread blocks within a cluster are scheduled simultaneously on SMs within a GPC. Thread blocks within a cluster are always adjacent to each other within the grid.

1.2.2.2 Warps and SIMT

Within a thread block, threads are organized into groups of 32 threads called *warps*. A warp executes the kernel code in a *Single-Instruction Multiple-Threads* (SIMT) paradigm. In SIMT, all threads in the warp are executing the same kernel code, but each thread may follow different branches through the code. That is, though all threads of the program execute the same code, threads do not need to follow the same execution path.

¹ In certain situations when using features such as *CUDA Dynamic Parallelism*, a thread block may be suspended to memory. This means the state of the SM is stored to a system-managed area of GPU memory and the SM is freed to execute other thread blocks. This is similar to context swapping on CPUs. This is not common.

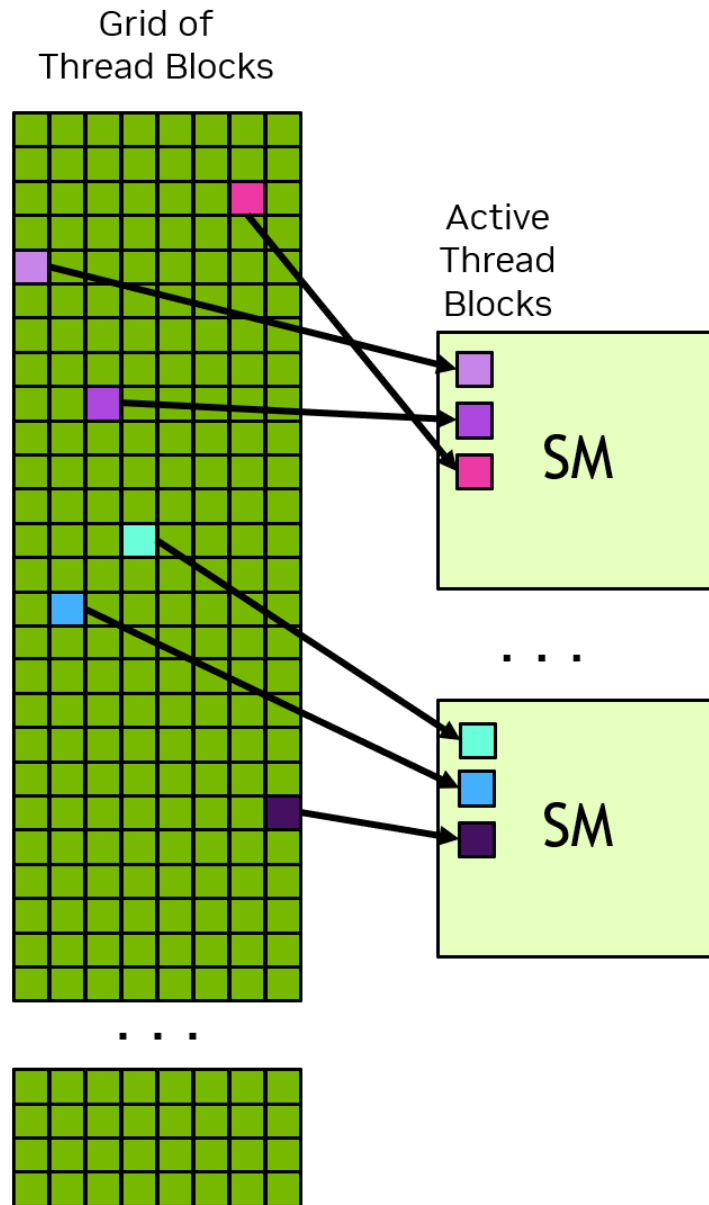


Figure 4: Each SM has one or more active thread blocks. In this example, each SM has three thread blocks scheduled simultaneously. There are no guarantees about the order in which thread blocks from a grid are assigned to SMs.

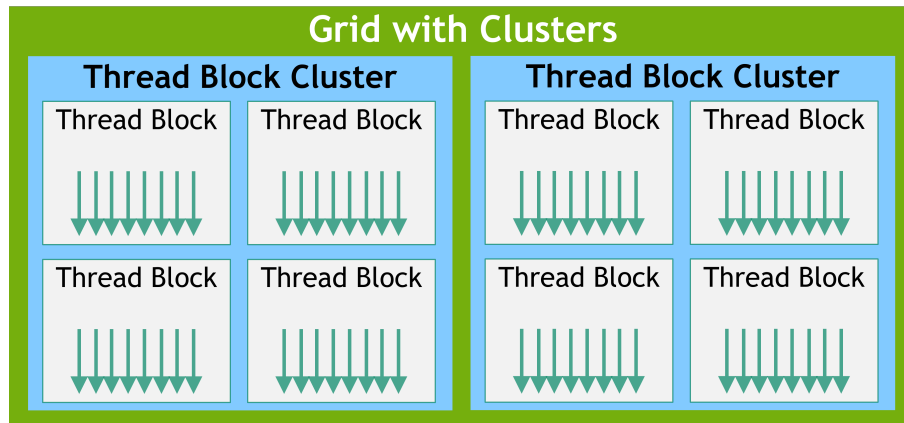


Figure 5: When clusters are specified, thread blocks are in the same location in the grid but also have a position within the containing cluster.

When threads are executed by a warp, they are assigned a warp lane. Warp lanes are numbered 0 to 31 and threads from a thread block are assigned to warps in a predictable fashion detailed in [Hardware Multithreading](#).

All threads in the warp execute the same instruction simultaneously. If some threads within a warp follow a control flow branch in execution while others do not, the threads which do not follow the branch will be masked off while the threads which follow the branch are executed. For example, if a conditional is only true for half the threads in a warp, the other half of the warp would be masked off while the active threads execute those instructions. This situation is illustrated in [Figure 7](#). When different threads in a warp follow different code paths, this is sometimes called warp divergence. It follows that utilization of the GPU is maximized when threads within a warp follow the same control flow path.

In the SIMT model, all threads in a warp progress through the kernel in lock step. Hardware execution may differ. See the sections on [Independent Thread Execution](#) for more information on where this distinction is important. Exploiting knowledge of how warp execution is actually mapped to real hardware is discouraged. The CUDA programming model and SIMT say that all threads in a warp progress through the code together. Hardware may optimize masked lanes in ways that are transparent to the program so long as the programming model is followed. If the program violates this model, this can result in undefined behavior that can be different in different GPU hardware.

While it is not necessary to consider warps when writing CUDA code, understanding the warp execution model is helpful in understanding concepts such as [global memory coalescing](#) and [shared memory bank access patterns](#). Some advanced programming techniques use specialization of warps within a thread block to limit thread divergence and maximize utilization. This and other optimizations make use of the knowledge that threads are grouped into warps when executing.

One implication of warp execution is that thread blocks are best specified to have a total number of threads which is a multiple of 32. It is legal to use any number of threads, but when the total is not a multiple of 32, the last warp of the thread block will have some lanes that are unused throughout execution. This will likely lead to suboptimal functional units utilization and memory access for that warp.

SIMT is often compared to Single Instruction Multiple Data (SIMD) parallelism, but there are some important differences. In SIMD, execution follows a single control flow path, while in SIMT, each thread is allowed to follow its own control flow path. Because of this, SIMT does not have a fixed data-width like SIMD. A more detailed discussion of SIMT can be found in [SIMT Execution Model](#).

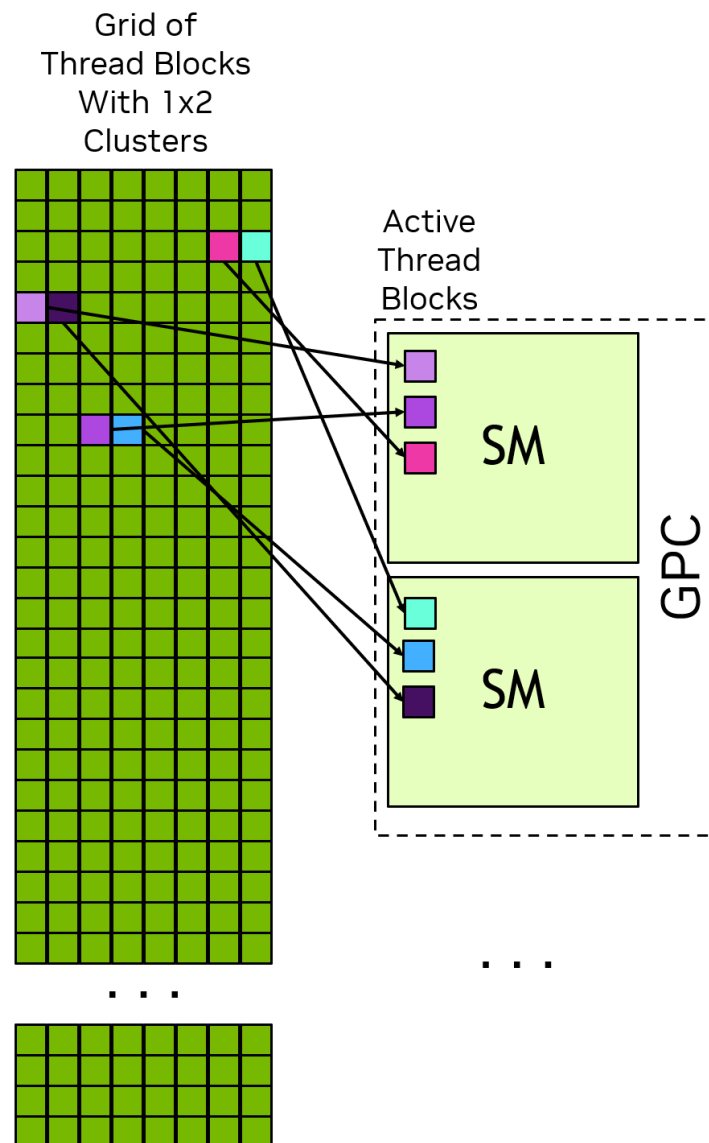


Figure 6: When clusters are specified, the thread blocks in a cluster are arranged in their cluster shape within the grid. The thread blocks of a cluster are scheduled simultaneously on the SMs of a single GPC.

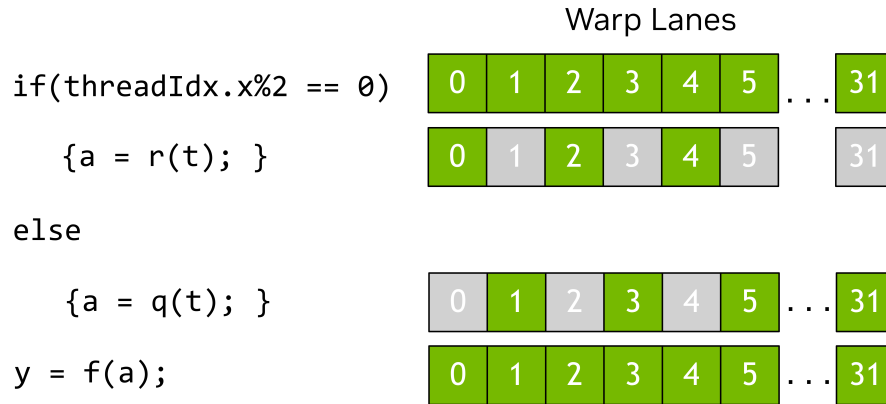


Figure 7: In this example, only threads with even thread index execute the body of the if statement, the others are masked off while the body is executed.

1.2.2.3 Tile Programming in CUDA

In addition to the SIMT model described in the preceding sections, CUDA supports a tile programming model. In tile programming, the programmer writes code at the level of an entire thread block, describing operations on multidimensional collections of data called **tiles**. The compiler maps these operations to the individual threads of the block.

Tile kernels are launched on a grid of blocks, as described in the [Thread Blocks and Grids](#) section. Each block executes the tile kernel and can query its position within the grid to determine which portion of the data it is responsible for. The programmer specifies only the grid dimensions; the number of threads per block is determined by the compiler based on the tile operations in the kernel ([Figure 8](#)).

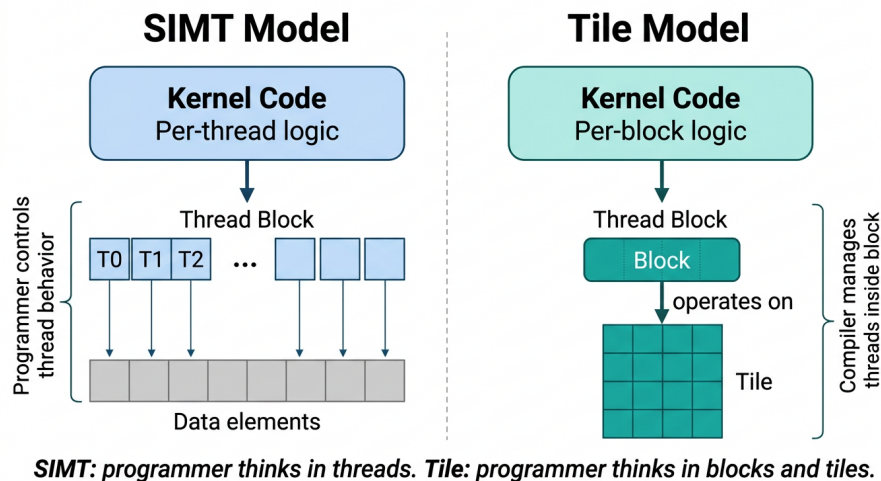


Figure 8: Programmer's view in the SIMT and tile programming models. In SIMT, the programmer writes per-thread code and controls how each thread accesses data. In tile programming, the programmer writes per-block code that operates on tiles; the compiler maps operations to the threads of the block.

Within a tile kernel, the block executes a single control flow. The programmer specifies operations on tiles, and the compiler distributes the work across the threads of the block. Standard control flow constructs such as conditionals and loops are supported, but because the block follows a single control flow, there is no concept of warp divergence. Scalar operations, such as computing an index or a loop

bound, are executed by a single thread of the block. Tile operations, such as adding two tiles element by element, are collectively executed in parallel by all threads of the block.

It is important not to confuse blocks—units of execution—with tiles—units of data. A single block may create and operate on many tiles of different shapes and data types.

1.2.2.3.1 Arrays and tiles

Tile kernels work with two types of data: **arrays** and **tiles**. An array (or global array) is a multidimensional container of elements stored in device memory. Arrays are mutable: their contents can be modified by store operations within a kernel. An array has a shape and a data type.

A tile is a multidimensional collection of values that exists only within tile code and is local to a single block. Tiles are immutable: every operation on a tile produces a new tile rather than modifying an existing one. Unlike an array, a tile does not necessarily have a representation in memory—the compiler decides how tile data is stored, and may use registers, shared memory, or other resources of the SM. Each dimension of a tile must be a power of two and must be known at compile time (that is, its value must be determinable before the kernel executes, rather than computed during execution). Tiles cannot be passed as kernel parameters; they are created and consumed entirely within tile code.

1.2.2.3.2 Tile space and data movement

Data moves between arrays and tiles through load and store operations. These operations use a concept called the **tile space**, which is the result of conceptually partitioning an array into equally sized, non-overlapping tiles. For example, consider a two-dimensional array of shape (M, N) . If a load operation specifies a tile shape of (t_m, t_n) , the array is conceptually divided into $\lceil M/t_m \rceil$ rows and $\lceil N/t_n \rceil$ columns of tiles. An index into this tile space, such as (i, j) , identifies which tile to load. The load returns a tile of shape (t_m, t_n) containing the corresponding elements from the array. When a tile extends beyond the boundaries of the array—for example, at the edges when the array dimensions are not exact multiples of the tile dimensions—the load specifies how out-of-bounds elements are handled, such as by filling them with zeros (Figure 9).

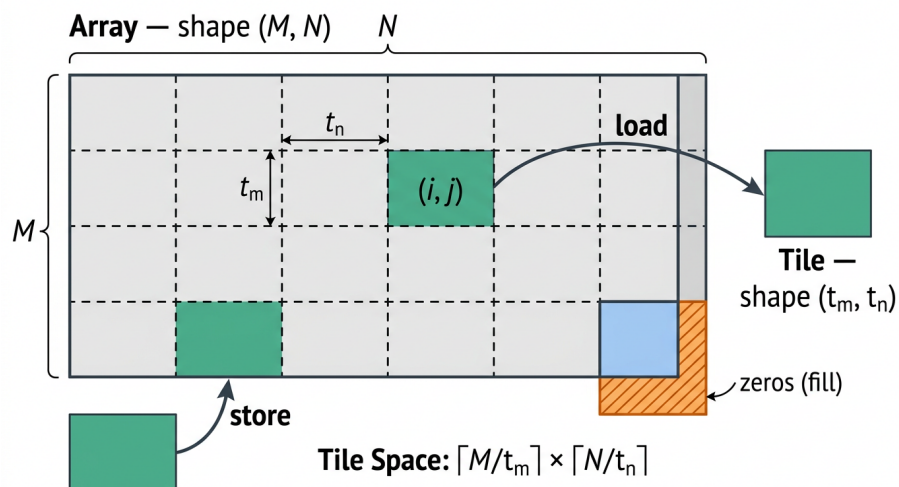


Figure 9: Tile space and data movement. A two-dimensional array of shape (M, N) is conceptually partitioned into a grid of tiles with shape (t_m, t_n) . A load at tile-space index (i, j) returns the corresponding tile. At the array boundary, elements that fall outside the array can be filled with zeros. A store writes a tile back to the array at a given tile-space index.

A store operation performs the inverse: given a tile and an index into the tile space, it writes the tile's

elements to the corresponding region of the array. Any writes that fall outside the array boundaries are silently discarded. Tile programs also support gather and scatter operations, which load from or store to arbitrary positions in an array.

1.2.2.3.3 Operations on tiles

Tile programs provide a set of built-in operations that act on tiles, including elementwise arithmetic, matrix multiplication, reductions (such as sum and maximum) along one or more axes, shape manipulation (such as reshape and transpose), and type conversion. When two tiles of different shapes are combined in an operation, the smaller tile is automatically expanded to match the larger one before the operation is applied.

1.2.2.3.4 Relationship to SIMT programming

Tile programming and SIMT programming coexist within CUDA. An application may contain both SIMT and tile kernels, and both types of kernels can operate on the same data in device memory. The choice of programming model is a per-kernel decision. Tile programming does not replace SIMT programming. SIMT provides fine-grained control over individual threads, which remains necessary for some algorithms and optimization techniques. Tile programming provides a higher-level abstraction that can simplify kernel development. Because thread-level decisions are left to the compiler, the same tile kernel can run on different GPU architectures without requiring changes to the source. Both models are built on the same underlying hardware—SMs, thread blocks, and grids—described in the preceding sections. Both models also use the same device memory spaces, which are introduced in the following section.

1.2.3. GPU Memory

In modern computing systems, efficiently utilizing memory is just as important as maximizing the use of functional units performing computations. Heterogeneous systems have multiple memory spaces, and GPUs contain various types of programmable on-chip memory in addition to caches. The following sections introduce these memory spaces in more details.

1.2.3.1 DRAM Memory in Heterogeneous Systems

GPUs and CPUs both have directly attached DRAM chips. In systems with more than one GPU, each GPU has its own memory. From the perspective of device code, the DRAM attached to the GPU is called *global memory*, because it is accessible to all SMs in the GPU. This terminology does not mean it is necessarily accessible everywhere within the system. The DRAM attached to the CPU(s) is called *system memory* or *host memory*.

Like CPUs, GPUs use virtual memory addressing. On all currently-supported systems, the CPU and GPU use a single unified virtual memory space. This means that the virtual memory address range for each GPU in the system is unique and distinct from the CPU and every other GPU in the system. For a given virtual memory address, it is possible to determine whether that address is in GPU memory or system memory and, on systems with multiple GPUs, which GPU memory contains that address.

There are CUDA APIs to allocate GPU memory, CPU memory, and to copy between allocations on the CPU and GPU, within a GPU, or between GPUs in multi-GPU systems. The locality of data can be explicitly controlled when desired. *Unified Memory*, discussed below, allows the placement of memory to be handled automatically by the CUDA runtime or system hardware.

1.2.3.2 On-Chip Memory in GPUs

In addition to the global memory, each GPU has some on-chip memory. Each SM has its own register file and shared memory. These memories are part of the SM and can be accessed extremely quickly from threads executing within the SM.

The register file stores thread local variables which are usually allocated by the compiler. The shared memory is accessible by all threads within a thread block or cluster. Shared memory can be used for exchanging data between threads of a thread block or cluster.

The register file and unified data cache in an SM have finite sizes. The size of an SM's register file, unified data cache, and how the unified data cache can be configured for L1 and shared memory balance can be found in [Memory Information per Compute Capability](#). The register file, shared memory space, and L1 cache are shared among all threads in a thread block.

To schedule a thread block to an SM, the total number of registers needed for each thread multiplied by the number of threads in the thread block must be less than or equal to the available registers in the SM. If the number of registers required for a thread block exceeds the size of the register file, the kernel is not launchable and the number of threads in the thread block must be decreased to make the thread block launchable.

Shared memory allocations are done at the thread block level. That is, unlike register allocations which are per thread, allocations of shared memory are common to the entire thread block.

1.2.3.2.1 Caches

In addition to programmable memories, GPUs have both L1 and L2 caches. Each SM has an L1 cache which is part of the unified data cache. A larger L2 cache is shared by all SMs within a GPU. This can be seen in the GPU block diagram in [Figure 2](#). Each SM also has a separate [constant cache](#), which is used to cache values in global memory that have been declared to be constant over the life of a kernel. The compiler may place kernel parameters into constant memory as well. This can improve kernel performance by allowing kernel parameters to be cached in the SM separately from the L1 data cache.

1.2.3.3 Unified Memory

When an application allocates memory explicitly on the GPU or CPU, that memory is only accessible to code running on that device. That is, CPU memory can only be accessed from CPU code, and GPU memory can only be accessed from kernels running on the GPU². CUDA APIs for copying memory between the CPU and GPU are used to explicitly copy data to the correct memory at the right time.

A CUDA feature called *unified memory* allows applications to make memory allocations which can be accessed from CPU or GPU. The CUDA runtime or underlying hardware enables access or relocates the data to the correct place when needed. Even with unified memory, optimal performance is attained by keeping the migration of memory to a minimum and accessing data from the processor directly attached to the memory where it resides as much as possible.

The hardware features of the system determine how access and exchange of data between memory spaces is achieved. Section [Unified Memory](#) introduces the different categories of unified memory systems. Section [Unified Memory](#) contains many more details about use and behavior of unified memory in all situations.

² An exception to this is [mapped memory](#), which is CPU memory allocated with properties that enable it to be directly accessed from the GPU. However, mapped access occurs over the PCIe or NVLINK connection. The GPU is unable to hide the higher latency and lower bandwidth behind parallelism, so mapped memory is not a performant replacement to unified memory or placing data in the appropriate memory space.

1.3. The CUDA platform

The NVIDIA CUDA platform consists of many pieces of software and hardware and many important technologies developed to enable computing on heterogeneous systems. This chapter serves to introduce some of the fundamental concepts and components of the CUDA platform that are important for application developers to understand. This chapter, like *Programming Model*, is not specific to any programming language, but applies to everything that uses the CUDA platform.

1.3.1. Compute Capability and Streaming Multiprocessor Versions

Every NVIDIA GPU has a *Compute Capability* (CC) number, which indicates what features are supported by that GPU and specifies some hardware parameters for that GPU. These specifications are documented in the [Section 5.1](#) appendix. A list of all NVIDIA GPUs and their compute capabilities is maintained on the [CUDA GPU Compute Capability page](#).

Compute capability is denoted as a major and minor version number in the format X.Y where X is the major version number and Y is the minor version number. For example, CC 12.0 has a major version of 12 and a minor version of 0. The compute capability directly corresponds to the version number of the SM. For example, the SMs within a GPU of CC 12.0 have SM version *sm_120*. This version is used to label binaries.

[Section 5.1.1](#) shows how to query and determine the compute capability of the GPU(s) in a system.

1.3.2. CUDA Toolkit and NVIDIA Driver

The *NVIDIA Driver* can be thought of as the operating system of the GPU. The NVIDIA Driver is a software component which must be installed on the host system's operating system and is necessary for all GPU uses, including display and graphical functionality. The NVIDIA Driver is foundational to the CUDA platform. In addition to CUDA, the NVIDIA Driver provides all other methods of using the GPU, for example Vulkan and Direct3D. The NVIDIA Driver has version numbers such as r580.

The *CUDA Toolkit* is a set of libraries, headers, and tools for writing, building, and analyzing software which utilizes GPU computing. The CUDA Toolkit is a separate software product from the NVIDIA driver.

The *CUDA runtime* is a special case of one of the libraries provided by the CUDA Toolkit. The CUDA runtime provides both an API and some language extensions to handle common tasks such as allocating memory, copying data between GPUs and other GPUs or CPUs, and launching kernels. The API components of the CUDA runtime are referred to as the CUDA runtime API.

The [CUDA Compatibility](#) document provides full details of compatibility between different GPUs, NVIDIA Drivers, and CUDA Toolkit versions.

1.3.2.1 CUDA Runtime API and CUDA Driver API

The CUDA runtime API is implemented on top of a lower-level API called the *CUDA driver API*, which is an API exposed by the NVIDIA Driver. This guide focuses on the APIs exposed by the CUDA runtime API. All the same functionality can be achieved using only the driver API if desired. Some features are only available using the driver API. Applications may use either API or both interoperably. [Section The CUDA Driver API](#) covers interoperation between the runtime and driver APIs.

The full API reference for the CUDA runtime API functions can be found in the [CUDA Runtime API Documentation](#).

The full API reference for the CUDA driver API can be found in the [CUDA Driver API Documentation](#).

1.3.3. Parallel Thread Execution (PTX)

A fundamental but sometimes invisible layer of the CUDA platform is the *Parallel Thread Execution* (PTX) virtual instruction set architecture (ISA). PTX is a high-level assembly language for NVIDIA GPUs. PTX provides an abstraction layer over the physical ISA of real GPU hardware. Like other platforms, applications can be written directly in this assembly language, though doing so can add unnecessary complexity and difficulty to software development.

Domain-specific languages and compilers for high-level languages can generate PTX code as an intermediate representation (IR) and then use NVIDIA's offline or just-in-time (JIT) compilation tools to produce executable binary GPU code. This enables the CUDA platform to be programmable from languages other than just those supported by NVIDIA-provided tools such as [NVCC: The NVIDIA CUDA Compiler](#).

Since GPU capabilities change and grow over time, the PTX virtual ISA specification is versioned. PTX versions, like SM versions, correspond to a compute capability. For example, PTX which supports all the features of compute capability 8.0 is called *compute_80*.

Full documentation on PTX can be found in the [PTX ISA](#).

1.3.4. Cubins and Fatbins

CUDA applications and libraries are usually written in a higher-level language like C++. That higher-level language is compiled to PTX, and then the PTX is compiled into real binary for a physical GPU, called a *CUDA binary*, or *cubin* for short. A cubin has a specific binary format for a specific SM version, such as *sm_120*.

Executables and library binaries that use GPU computing contain both CPU and GPU code. The GPU code is stored within a container called a *fatbin*. Fatbins can contain cubins and PTX for multiple different targets. For example, an application could be built with binaries for multiple different GPU architectures, that is, different SM versions. When an application is run, its GPU code is loaded onto a specific GPU and the best binary for that GPU from the fatbin is used.

Fatbins can also contain one or more PTX versions of GPU code, the use for which is described in [PTX Compatibility](#). [Figure 10](#) shows an example of an application or library binary which contains multiple cubin versions of GPU code as well as one version of PTX code.

1.3.4.1 Binary Compatibility

NVIDIA GPUs guarantee binary compatibility in certain circumstances. Specifically, within a major version of compute capability, GPUs with minor compute capability greater than or equal to the targeted version of cubin can load and execute that cubin. For example, if an application contains a cubin with code compiled for compute capability 8.6, that cubin can be loaded and executed on GPUs with compute capability 8.6 or 8.9. It cannot, however, be loaded on GPUs with compute capability 8.0, because the GPU's CC minor version, 0, is lower than the code's minor version, 6.

NVIDIA GPUs are not binary compatible between major compute capability versions. That is, cubin code compiled for compute capability 8.6 will not load on GPUs of compute capability 9.0.

When discussing binary code, the binary code is often referred to as having a version such as *sm_86* in the above example. This is the same as saying the binary was built for compute capability 8.6. This shorthand is often used because it is how a developer specifies this binary build target to the NVIDIA CUDA compiler, [nvcc](#).

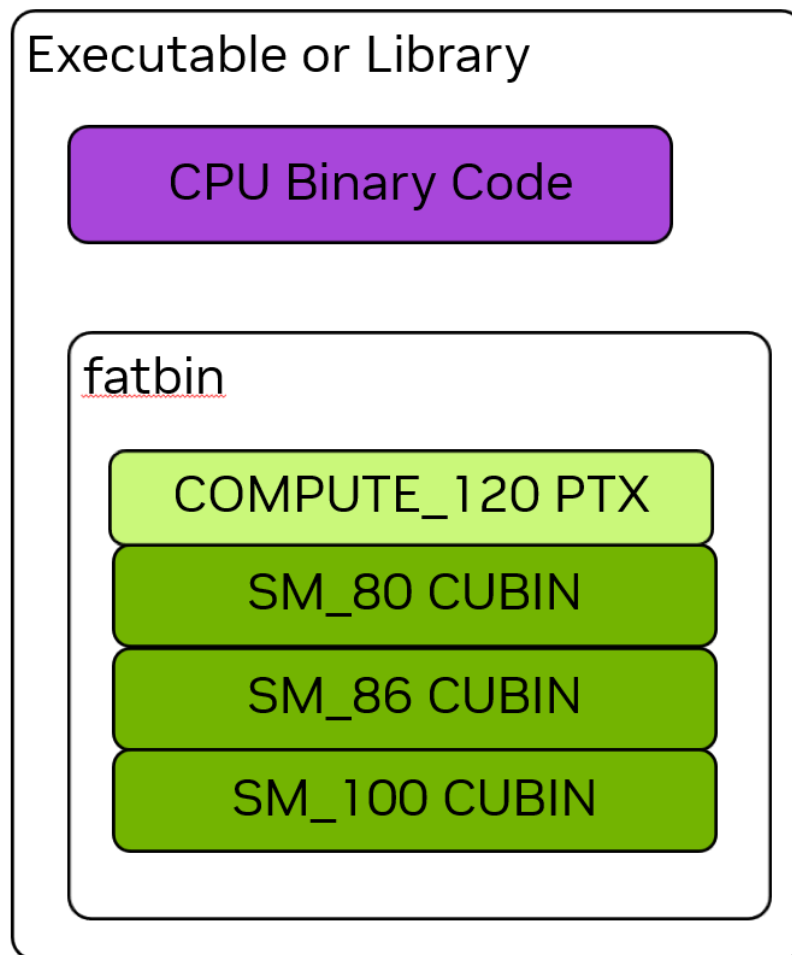


Figure 10: The binary for an executable or library contains both CPU binary code and a fatbin container for GPU code. A fatbin can contain both cubin GPU binary code and PTX virtual ISA code. PTX code can be JIT compiled for future targets.

Note

Binary compatibility is promised only for binaries created by NVIDIA tools such as `nvcc`. Manual editing or generating binary code for NVIDIA GPUs is not supported. Compatibility promises are invalidated if binaries are modified in any way.

1.3.4.2 PTX Compatibility

GPU code can be stored in executables in binary or PTX form, which is covered in [Cubins and Fatbins](#). When an application stores the PTX version of GPU code, that PTX can be JIT compiled at application runtime for any compute capability equal or higher to the compute capability of the PTX code. For example, if an application contains PTX for `compute_80`, that PTX code can be JIT compiled to later SM versions, such as `sm_120` at application runtime. This enables forward compatibility with future GPUs without the need to rebuild applications or libraries.

1.3.4.3 Just-in-Time Compilation

PTX code loaded by an application at runtime is compiled to binary code by the device driver. This is called just-in-time (JIT) compilation. Just-in-time compilation increases application load time, but allows the application to benefit from any new compiler improvements coming with each new device driver. It also enables applications to run on devices that did not exist at the time the application was compiled.

When the device driver just-in-time compiles PTX code for an application, it automatically caches a copy of the generated binary code in order to avoid repeating the compilation in subsequent invocations of the application. The cache - called the compute cache - is automatically invalidated when the device driver is upgraded, so that applications can benefit from the improvements in the new just-in-time compiler built into the device driver.

How and when PTX is JIT compiled at runtime has been relaxed since the earliest versions of CUDA, allowing more flexibility for when and if to JIT compile some or all kernels. The section [Lazy Loading](#) describes the available options and how to control JIT behavior. There are also a few environment variables which control just-in-time compilation behavior, as described in [CUDA Environment Variables](#).

As an alternative to using `nvcc` to compile CUDA C++ device code, NVRTC can be used to compile CUDA C++ device code to PTX at runtime. NVRTC is a runtime compilation library for CUDA C++; more information can be found in the NVRTC User guide.

Chapter 2. Programming GPUs in CUDA

2.1. Intro to CUDA C++

This chapter introduces some of the basic concepts of the CUDA programming model by illustrating how they are exposed in C++.

This programming guide focuses on the CUDA runtime API. The CUDA runtime API is the most commonly used way of using CUDA in C++ and is built on top of the lower level CUDA driver API.

CUDA Runtime API and CUDA Driver API discusses the difference between the APIs and *CUDA driver API* discusses writing code that mixes the APIs.

This guide assumes the CUDA Toolkit and NVIDIA Driver are installed and that a supported NVIDIA GPU is present. See *The CUDA Quickstart Guide* for instructions on installing the necessary CUDA components.

2.1.1. Compilation with NVCC

GPU code written in C++ is compiled using the NVIDIA Cuda Compiler, `nvcc`. `nvcc` is a compiler driver that simplifies the process of compiling C++ or PTX code: It provides simple and familiar command line options and executes them by invoking the collection of tools that implement the different compilation stages.

This guide will show `nvcc` command lines which can be used on any Linux system with the CUDA Toolkit installed, at a Windows command line or power shell, or on Windows Subsystem for Linux with the CUDA Toolkit. The *nvcc chapter* of this guide covers common use cases of `nvcc`, and complete documentation is provided by *the nvcc user manual*.

2.1.2. Kernels

As mentioned in the introduction to the *CUDA Programming Model*, functions which execute on the GPU which can be invoked from the host are called kernels. Kernels are written to be run by many parallel threads simultaneously.

2.1.2.1 Specifying Kernels

The code for a kernel is specified using the `__global__` declaration specifier. This indicates to the compiler that this function will be compiled for the GPU in a way that allows it to be invoked from a kernel launch. A kernel launch is an operation which starts a kernel running, usually from the CPU. Kernels are functions with a `void` return type.

```
// Kernel definition
__global__ void vecAdd(float* A, float* B, float* C)
{
}
}
```

2.1.2.2 Launching Kernels

The number of threads that will execute the kernel in parallel is specified as part of the kernel launch. This is called the execution configuration. Different invocations of the same kernel may use different execution configurations, such as a different number of threads or thread blocks.

There are two ways of launching kernels from CPU code, *triple chevron notation* and `cudaLaunchKernelEx`. Triple chevron notation, the most common way of launching kernels, is introduced here. An example of launching a kernel using `cudaLaunchKernelEx` is shown and discussed in detail in section [Section 3.1.1](#).

2.1.2.2.1 Triple Chevron Notation

Triple chevron notation is a *CUDA C++ Language Extension* which is used to launch kernels. It is called triple chevron because it uses three chevron characters to encapsulate the execution configuration for the kernel launch, i.e. `<<< >>>`. Execution configuration parameters are specified as a comma separated list inside the chevrons, similar to parameters to a function call. The syntax for a kernel launch of the `vecAdd` kernel is shown below.

```
__global__ void vecAdd(float* A, float* B, float* C)
{
}

int main()
{
    ...
    // Kernel invocation
    vecAdd<<<1, 256>>>(A, B, C);
    ...
}
```

The first two parameters to the triple chevron notation are the grid dimensions and the thread block dimensions, respectively. When using 1-dimensional thread blocks or grids, integers can be used to specify dimensions.

The above code launches a single thread block containing 256 threads. Each thread will execute the exact same kernel code. In *Thread and Grid Index Intrinsics*, we'll show how each thread can use its index within the thread block and grid to change the data it operates on.

There is a limit to the number of threads per block, since all threads of a block reside on the same streaming multiprocessor(SM) and must share the resources of the SM. On current GPUs, a thread block may contain up to 1024 threads. If resources allow, more than one thread block can be scheduled on an SM simultaneously.

Kernel launches are asynchronous with respect to the host thread. That is, the kernel will be setup for execution on the GPU, but the host code will not wait for the kernel to complete (or even start) executing on the GPU before proceeding. Some form of synchronization between the GPU and CPU

must be used to determine that the kernel has completed. The most basic version, completely synchronizing the entire GPU, is shown in [Synchronizing CPU and GPU](#). More sophisticated methods of synchronization are covered in [Asynchronous Execution](#).

When using 2 or 3-dimensional grids or thread blocks, the CUDA type `dim3` is used as the grid and thread block dimension parameters. The code fragment below shows a kernel launch of a `MatAdd` kernel using 16 by 16 grid of thread blocks, each thread block is 8 by 8.

```
int main()
{
    ...
    dim3 grid(16,16);
    dim3 block(8,8);
    MatAdd<<<grid, block>>>(A, B, C);
    ...
}
```

2.1.2.3 Thread and Grid Index Intrinsics

Within kernel code, CUDA provides intrinsics to access parameters of the execution configuration and the index of a thread or block.

- ▶ `threadIdx` gives the index of a thread within its thread block. Each thread in a thread block will have a different index.
- ▶ `blockDim` gives the dimensions of the thread block, which was specified in the execution configuration of the kernel launch.
- ▶ `blockIdx` gives the index of a thread block within the grid. Each thread block will have a different index.
- ▶ `gridDim` gives the dimensions of the grid, which was specified in the execution configuration when the kernel was launched.

Each of these intrinsics is a 3-component vector with a `.x`, `.y`, and `.z` member. Dimensions not specified by a launch configuration will default to 1. `threadIdx` and `blockIdx` are zero indexed. That is, `threadIdx.x` will take on values from 0 up to and including `blockDim.x-1`. `.y` and `.z` operate the same in their respective dimensions.

Similarly, `blockIdx.x` will have values from 0 up to and including `gridDim.x-1`, and the same for `.y` and `.z` dimensions, respectively.

These allow an individual thread to identify what work it should carry out. Returning to the `vecAdd` kernel, the kernel takes three parameters, each is a vector of floats. The kernel performs an element-wise addition of A and B and stores the result in C. The kernel is parallelized such that each thread will perform one addition. Which element it computes is determined by its thread and grid index.

```
__global__ void vecAdd(float* A, float* B, float* C)
{
    // calculate which element this thread is responsible for computing
    int workIndex = threadIdx.x + blockDim.x * blockIdx.x

    // Perform computation
    C[workIndex] = A[workIndex] + B[workIndex];
}

int main()
```

(continues on next page)

(continued from previous page)

```

{
    ...
    // A, B, and C are vectors of 1024 elements
    vecAdd<<<4, 256>>>(A, B, C);
    ...
}

```

In this example, 4 thread blocks of 256 threads are used to add a vector of 1024 elements. In the first thread block, `blockIdx.x` will be zero, and so each thread's `workIndex` will simply be its `threadIdx.x`. In the second thread block, `blockIdx.x` will be 1, so `blockDim.x * blockIdx.x` will be the same as `blockDim.x`, which is 256 in this case. The `workIndex` for each thread in the second thread block will be its `threadIdx.x + 256`. In the third thread block `workIndex` will be `threadIdx.x + 512`.

This computation of `workIndex` is very common for 1-dimensional parallelizations. Expanding to two or three dimensions often follows the same pattern in each of those dimensions.

2.1.2.3.1 Bounds Checking

The example given above assumes that the length of the vector is a multiple of the thread block size, 256 threads in this case. To make the kernel handle any vector length, we can add checks that the memory access is not exceeding the bounds of the arrays as shown below, and then launch one thread block which will have some inactive threads.

```

__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    // calculate which element this thread is responsible for computing
    int workIndex = threadIdx.x + blockDim.x * blockIdx.x

    if(workIndex < vectorLength)
    {
        // Perform computation
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}

```

With the above kernel code, more threads than needed can be launched without causing out-of-bounds accesses to the arrays. When `workIndex` exceeds `vectorLength`, threads exit and do not do any work. Launching extra threads in a block that do no work does not incur a large overhead cost, however launching thread blocks in which no threads do work should be avoided. This kernel can now handle vector lengths which are not a multiple of the block size.

The number of thread blocks which are needed can be calculated as the ceiling of the number of threads needed, the vector length in this case, divided by the number of threads per block. That is, the integer division of the number of threads needed by the number of threads per block, rounded up. A common way of expressing this as a single integer division is given below. By adding `threads - 1` before the integer division, this behaves like a ceiling function, adding another thread block only if the vector length is not divisible by the number of threads per block.

```

// vectorLength is an integer storing number of elements in the vector
int threads = 256;
int blocks = (vectorLength + threads-1)/threads;
vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);

```

The [CUDA Core Compute Library \(CCCL\)](#) provides a convenient utility, `cuda::ceil_div`, for doing this

ceiling divide to calculate the number of blocks needed for a kernel launch. This utility is available by including the header `<cuda/cmath>`.

```
// vectorLength is an integer storing number of elements in the vector
int threads = 256;
int blocks = cuda::ceil_div(vectorLength, threads);
vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
```

The choice of 256 threads per block here is arbitrary, but this is quite often a good value to start with.

2.1.3. Memory in GPU Computing

In order to use the `vecAdd` kernel shown above, the arrays A, B, and C must be in memory accessible to the GPU. There are several different ways to do this, two of which will be illustrated here. Other methods will be covered in later sections on *unified memory*. The memory spaces available to code running on the GPU were introduced in *GPU Memory* and are covered in more detail in *GPU Device Memory Spaces*.

2.1.3.1 Unified Memory

Unified memory is a feature of the CUDA runtime which lets the NVIDIA Driver manage movement of data between host and device(s). Memory is allocated using the `cudaMallocManaged` API or by declaring a variable with the `__managed__` specifier. The NVIDIA Driver will make sure that the memory is accessible to the GPU or CPU whenever either tries to access it.

The code below shows a complete function to launch the `vecAdd` kernel which uses unified memory for the input and output vectors that will be used on the GPU. `cudaMallocManaged` allocates buffers which can be accessed from either the CPU or the GPU. These buffers are released using `cudaFree`.

```
void unifiedMemExample(int vectorLength)
{
    // Pointers to memory vectors
    float* A = nullptr;
    float* B = nullptr;
    float* C = nullptr;
    float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

    // Use unified memory to allocate buffers
    cudaMallocManaged(&A, vectorLength*sizeof(float));
    cudaMallocManaged(&B, vectorLength*sizeof(float));
    cudaMallocManaged(&C, vectorLength*sizeof(float));

    // Initialize vectors on the host
    initArray(A, vectorLength);
    initArray(B, vectorLength);

    // Launch the kernel. Unified memory will make sure A, B, and C are
    // accessible to the GPU
    int threads = 256;
    int blocks = cuda::ceil_div(vectorLength, threads);
    vecAdd<<<blocks, threads>>>(A, B, C, vectorLength);
    // Wait for the kernel to complete execution
    cudaDeviceSynchronize();
}
```

(continues on next page)

(continued from previous page)

```

// Perform computation serially on CPU for comparison
serialVecAdd(A, B, comparisonResult, vectorLength);

// Confirm that CPU and GPU got the same answer
if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
{
    printf("Unified Memory: CPU and GPU answers match\n");
}
else
{
    printf("Unified Memory: Error - CPU and GPU answers do not match\n");
}

// Clean Up
cudaFree(A);
cudaFree(B);
cudaFree(C);
free(comparisonResult);
}

```

Unified memory is supported on all operating systems and GPUs supported by CUDA, though the underlying mechanism and performance may differ based on system architecture. *Unified Memory* provides more details. On some Linux systems, (e.g. those with *address translation services* or *heterogeneous memory management*) all system memory is automatically unified memory, and there is no need to use `cudaMallocManaged` or the `__managed__` specifier.

2.1.3.2 Explicit Memory Management

Explicitly managing memory allocation and data migration between memory spaces can help improve application performance, though it does make for more verbose code. The code below explicitly allocates memory on the GPU using `cudaMalloc`. Memory on the GPU is freed using the same `cudaFree` API as was used for unified memory in the previous example.

```

void explicitMemExample(int vectorLength)
{
    // Pointers for host memory
    float* A = nullptr;
    float* B = nullptr;
    float* C = nullptr;
    float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

    // Pointers for device memory
    float* devA = nullptr;
    float* devB = nullptr;
    float* devC = nullptr;

    //Allocate Host Memory using cudaMallocHost API. This is best practice
    // when buffers will be used for copies between CPU and GPU memory
    cudaMallocHost(&A, vectorLength*sizeof(float));
    cudaMallocHost(&B, vectorLength*sizeof(float));
}

```

(continues on next page)

(continued from previous page)

```

cudaMallocHost(&C, vectorLength*sizeof(float));

// Initialize vectors on the host
initArray(A, vectorLength);
initArray(B, vectorLength);

// start-allocate-and-copy
// Allocate memory on the GPU
cudaMalloc(&devA, vectorLength*sizeof(float));
cudaMalloc(&devB, vectorLength*sizeof(float));
cudaMalloc(&devC, vectorLength*sizeof(float));

// Copy data to the GPU
cudaMemcpy(devA, A, vectorLength*sizeof(float), cudaMemcpyDefault);
cudaMemcpy(devB, B, vectorLength*sizeof(float), cudaMemcpyDefault);
cudaMemset(devC, 0, vectorLength*sizeof(float));
// end-allocate-and-copy

// Launch the kernel
int threads = 256;
int blocks = cuda::ceil_div(vectorLength, threads);
vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
// wait for kernel execution to complete
cudaDeviceSynchronize();

// Copy results back to host
cudaMemcpy(C, devC, vectorLength*sizeof(float), cudaMemcpyDefault);

// Perform computation serially on CPU for comparison
serialVecAdd(A, B, comparisonResult, vectorLength);

// Confirm that CPU and GPU got the same answer
if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
{
    printf("Explicit Memory: CPU and GPU answers match\n");
}
else
{
    printf("Explicit Memory: Error - CPU and GPU answers to not match\n");
}

// clean up
cudaFree(devA);
cudaFree(devB);
cudaFree(devC);
cudaFreeHost(A);
cudaFreeHost(B);
cudaFreeHost(C);
free(comparisonResult);
}

```

The CUDA API `cudaMemcpy` is used to copy data from a buffer residing on the CPU to a buffer residing on the GPU. Along with the destination pointer, source pointer, and size in bytes, the final parameter

of `cudaMemcpy` is a `cudaMemcpyKind_t`. This can have values such as:

- ▶ `cudaMemcpyHostToDevice` for copies from the CPU to a GPU
- ▶ `cudaMemcpyDeviceToHost` for copies from the GPU to the CPU
- ▶ `cudaMemcpyDeviceToDevice` for copies within a GPU or between GPUs

In this example, `cudaMemcpyDefault` is passed as the last argument to `cudaMemcpy`. This causes CUDA to use the value of the source and destination pointers to determine the type of copy to perform.

The `cudaMemcpy` API is synchronous. That is, it does not return until the copy has completed. Asynchronous copies are introduced in [Launching Memory Transfers in CUDA Streams](#).

The code uses `cudaMallocHost` to allocate memory on the CPU. This allocates *page-locked memory* on the host, which can improve copy performance and is necessary for *asynchronous* memory transfers. In general, it is good practice to use page-locked memory for CPU buffers that will be used in data transfers to and from GPUs. Performance can degrade on some systems if too much host memory is page-locked. Best practice is to page-lock only buffers which will be used for sending or receiving data from the GPU.

2.1.3.3 Memory Management and Application Performance

As can be seen in the above example, explicit memory management is more verbose, requiring the programmer to specify copies between the host and device. This is the advantage and disadvantage of explicit memory management: it affords more control of when data is copied between host and devices, where memory is resident, and exactly what memory is allocated where. Explicit memory management can provide performance opportunities controlling memory transfers and overlapping them with other computations.

When using unified memory, there are CUDA APIs (which will be covered in [Memory Advise and Prefetch](#)), which provide hints to the NVIDIA driver managing the memory, which can enable some of the performance benefits of using explicit memory management when using unified memory.

2.1.4. Synchronizing CPU and GPU

As mentioned in [Launching Kernels](#), kernel launches are asynchronous with respect to the CPU thread which called them. This means the control flow of the CPU thread will continue executing before the kernel has completed, and possibly even before it has launched. In order to guarantee that a kernel has completed execution before proceeding in host code, some synchronization mechanism is necessary.

The simplest way to synchronize the GPU and a host thread is with the use of `cudaDeviceSynchronize`, which blocks the host thread until all previously issued work on the GPU has completed. In the examples of this chapter this is sufficient because only single operations are being executed on the GPU. In larger applications, there may be multiple *streams* executing work on the GPU and `cudaDeviceSynchronize` will wait for work in all streams to complete. In these applications, using [Stream Synchronization](#) APIs to synchronize only with a specific stream or [CUDA Events](#) is recommended. These will be covered in detail in the [Asynchronous Execution](#) chapter.

2.1.5. Putting it All Together

The following listings show the entire code for the simple vector addition kernel introduced in this chapter along with all host code and utility functions for checking to verify that the answer obtained is correct. These examples default to using a vector length of 1024, but accept a different vector length as a command line argument to the executable.

Unified Memory

```

#include <cuda_runtime_api.h>
#include <memory.h>
#include <cstdlib>
#include <ctime>
#include <stdio.h>
#include <cuda/cmath>

__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}

void initArray(float* A, int length)
{
    std::srand(std::time({}));
    for(int i=0; i<length; i++)
    {
        A[i] = rand() / (float)RAND_MAX;
    }
}

void serialVecAdd(float* A, float* B, float* C, int length)
{
    for(int i=0; i<length; i++)
    {
        C[i] = A[i] + B[i];
    }
}

bool vectorApproximatelyEqual(float* A, float* B, int length, float epsilon=0.
→00001)
{
    for(int i=0; i<length; i++)
    {
        if(fabs(A[i] -B[i]) > epsilon)
        {
            printf("Index %d mismatch: %f != %f", i, A[i], B[i]);
            return false;
        }
    }
    return true;
}

//unified-memory-begin
void unifiedMemExample(int vectorLength)
{
    // Pointers to memory vectors

```

(continues on next page)

(continued from previous page)

```

float* A = nullptr;
float* B = nullptr;
float* C = nullptr;
float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

// Use unified memory to allocate buffers
cudaMallocManaged(&A, vectorLength*sizeof(float));
cudaMallocManaged(&B, vectorLength*sizeof(float));
cudaMallocManaged(&C, vectorLength*sizeof(float));

// Initialize vectors on the host
initArray(A, vectorLength);
initArray(B, vectorLength);

// Launch the kernel. Unified memory will make sure A, B, and C are
// accessible to the GPU
int threads = 256;
int blocks = cuda::ceil_div(vectorLength, threads);
vecAdd<<<blocks, threads>>>(A, B, C, vectorLength);
// Wait for the kernel to complete execution
cudaDeviceSynchronize();

// Perform computation serially on CPU for comparison
serialVecAdd(A, B, comparisonResult, vectorLength);

// Confirm that CPU and GPU got the same answer
if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
{
    printf("Unified Memory: CPU and GPU answers match\n");
}
else
{
    printf("Unified Memory: Error - CPU and GPU answers do not match\n");
}

// Clean Up
cudaFree(A);
cudaFree(B);
cudaFree(C);
free(comparisonResult);
}
//unified-memory-end

int main(int argc, char** argv)
{
    int vectorLength = 1024;
    if(argc >= 2)
    {
        vectorLength = std::atoi(argv[1]);
    }
}

```

(continues on next page)

(continued from previous page)

```

    unifiedMemExample(vectorLength);
    return 0;
}

```

Explicit Memory Management

```

#include <cuda_runtime_api.h>
#include <memory.h>
#include <cstdlib>
#include <ctime>
#include <stdio.h>
#include <cuda/cmath>

__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}

void initArray(float* A, int length)
{
    std::srand(std::time({}));
    for(int i=0; i<length; i++)
    {
        A[i] = rand() / (float)RAND_MAX;
    }
}

void serialVecAdd(float* A, float* B, float* C, int length)
{
    for(int i=0; i<length; i++)
    {
        C[i] = A[i] + B[i];
    }
}

bool vectorApproximatelyEqual(float* A, float* B, int length, float epsilon=0.
↪00001)
{
    for(int i=0; i<length; i++)
    {
        if(fabs(A[i] -B[i]) > epsilon)
        {
            printf("Index %d mismatch: %f != %f", i, A[i], B[i]);
            return false;
        }
    }
    return true;
}

```

(continues on next page)

(continued from previous page)

```

}

//explicit-memory-begin
void explicitMemExample(int vectorLength)
{
    // Pointers for host memory
    float* A = nullptr;
    float* B = nullptr;
    float* C = nullptr;
    float* comparisonResult = (float*)malloc(vectorLength*sizeof(float));

    // Pointers for device memory
    float* devA = nullptr;
    float* devB = nullptr;
    float* devC = nullptr;

    //Allocate Host Memory using cudaMallocHost API. This is best practice
    // when buffers will be used for copies between CPU and GPU memory
    cudaMallocHost(&A, vectorLength*sizeof(float));
    cudaMallocHost(&B, vectorLength*sizeof(float));
    cudaMallocHost(&C, vectorLength*sizeof(float));

    // Initialize vectors on the host
    initArray(A, vectorLength);
    initArray(B, vectorLength);

    // start-allocate-and-copy
    // Allocate memory on the GPU
    cudaMalloc(&devA, vectorLength*sizeof(float));
    cudaMalloc(&devB, vectorLength*sizeof(float));
    cudaMalloc(&devC, vectorLength*sizeof(float));

    // Copy data to the GPU
    cudaMemcpy(devA, A, vectorLength*sizeof(float), cudaMemcpyDefault);
    cudaMemcpy(devB, B, vectorLength*sizeof(float), cudaMemcpyDefault);
    cudaMemset(devC, 0, vectorLength*sizeof(float));
    // end-allocate-and-copy

    // Launch the kernel
    int threads = 256;
    int blocks = cuda::ceil_div(vectorLength, threads);
    vecAdd<<<blocks, threads>>>(devA, devB, devC, vectorLength);
    // wait for kernel execution to complete
    cudaDeviceSynchronize();

    // Copy results back to host
    cudaMemcpy(C, devC, vectorLength*sizeof(float), cudaMemcpyDefault);

    // Perform computation serially on CPU for comparison
    serialVecAdd(A, B, comparisonResult, vectorLength);

    // Confirm that CPU and GPU got the same answer

```

(continues on next page)

(continued from previous page)

```

    if(vectorApproximatelyEqual(C, comparisonResult, vectorLength))
    {
        printf("Explicit Memory: CPU and GPU answers match\n");
    }
    else
    {
        printf("Explicit Memory: Error - CPU and GPU answers to not match\n");
    }

    // clean up
    cudaFree(devA);
    cudaFree(devB);
    cudaFree(devC);
    cudaFreeHost(A);
    cudaFreeHost(B);
    cudaFreeHost(C);
    free(comparisonResult);
}
//explicit-memory-end

int main(int argc, char** argv)
{
    int vectorLength = 1024;
    if(argc >= 2)
    {
        vectorLength = std::atoi(argv[1]);
    }
    explicitMemExample(vectorLength);
    return 0;
}

```

These can be built and run using `nvcc` as follows:

```

$ nvcc vecAdd_unifiedMemory.cu -o vecAdd_unifiedMemory
$ ./vecAdd_unifiedMemory
Unified Memory: CPU and GPU answers match
$ ./vecAdd_unifiedMemory 4096
Unified Memory: CPU and GPU answers match

```

```

$ nvcc vecAdd_explicitMemory.cu -o vecAdd_explicitMemory
$ ./vecAdd_explicitMemory
Explicit Memory: CPU and GPU answers match
$ ./vecAdd_explicitMemory 4096
Explicit Memory: CPU and GPU answers match

```

In these examples, all threads are doing independent work and do not need to coordinate or synchronize with each other. Frequently, threads will need to cooperate and communicate with other threads to carry out their work. Threads within a block can share data through *shared memory* and synchronize to coordinate memory accesses.

The most basic mechanism for synchronization at the block level is the `__syncthreads()` intrinsic, which acts as a barrier at which all threads in the block must wait before any threads are allowed to

proceed. *Shared Memory* gives an example of using shared memory.

For efficient cooperation, shared memory is expected to be a low-latency memory near each processor core (much like an L1 cache) and `__syncthreads()` is expected to be lightweight. `__syncthreads()` only synchronizes the threads within a single thread block.

Synchronization between blocks is only supported in certain circumstances. For example, *thread block clusters* allow blocks within a cluster to synchronize, and *the cooperative Groups APIs* provide mechanisms to create cross-block synchronization domains.

Best performance is usually achieved when synchronization is kept within a thread block. Thread blocks can still work on common results using *atomic memory functions*, which will be covered in coming sections.

Section [Section 3.2.4](#) covers CUDA synchronization primitives that provide very fine-grained control for maximizing performance and resource utilization.

2.1.6. Runtime Initialization

The CUDA runtime creates a *CUDA context* for each device in the system. This context is the primary context for this device and is initialized at the first runtime function which requires an active context on this device. The context is shared among all the host threads of the application. As part of context creation, the device code is *just-in-time compiled* if necessary and loaded into device memory. This all happens transparently. The primary context created by the CUDA runtime can be accessed from the driver API for interoperability as described in *Interoperability between Runtime and Driver APIs*.

As of CUDA 12.0, the `cudaInitDevice` and `cudaSetDevice` calls initialize the runtime and the primary *context* associated with the specified device. The runtime will implicitly use device 0 and self-initialize as needed to process runtime API requests if they occur before these calls. This is important when timing runtime function calls and when interpreting the error code from the first call into the runtime. Prior to CUDA 12.0, `cudaSetDevice` would not initialize the runtime.

`cudaDeviceReset` destroys the primary context of the current device. If CUDA runtime APIs are called after the primary context has been destroyed, a new primary context for that device will be created.

Note

The CUDA interfaces use global state that is initialized during host program initiation and destroyed during host program termination. Using any of these interfaces (implicitly or explicitly) during program initiation or termination after main will result in undefined behavior.

As of CUDA 12.0, `cudaSetDevice` explicitly initializes the runtime, if it has not already been initialized, after changing the current device for the host thread. Previous versions of CUDA delayed runtime initialization on the new device until the first runtime call was made after `cudaSetDevice`. Because of this, it is very important to check the return value of `cudaSetDevice` for initialization errors.

The runtime functions from the error handling and version management sections of the reference manual do not initialize the runtime.

2.1.7. Error Checking in CUDA

Every CUDA API returns a value of an enumerated type, `cudaError_t`. In example code these errors are often not checked. In production applications, it is best practice to always check and manage the

return value of every CUDA API call. When there are no errors, the value returned is `cudaSuccess`. Many applications choose to implement a utility macro such as the one shown below

```
#define CUDA_CHECK(expr_to_check) do { \
    cudaError_t result = expr_to_check; \
    if(result != cudaSuccess) \
    { \
        fprintf(stderr, \
            "CUDA Runtime Error: %s:%i:%d = %s\n", \
            __FILE__, \
            __LINE__, \
            result, \
            cudaGetErrorString(result)); \
    } \
} while(0)
```

This macro uses the `cudaGetErrorString` API, which returns a human readable string describing the meaning of a specific `cudaError_t` value. Using the above macro, an application would call CUDA runtime API calls within a `CUDA_CHECK(expression)` macro, as shown below:

```
CUDA_CHECK(cudaMalloc(&devA, vectorLength*sizeof(float)));
CUDA_CHECK(cudaMalloc(&devB, vectorLength*sizeof(float)));
CUDA_CHECK(cudaMalloc(&devC, vectorLength*sizeof(float)));
```

If any of these calls detect an error, it will be printed to `stderr` using this macro. This macro is common for smaller projects, but can be adapted to a logging system or other error handling mechanism in larger applications.

Note

It is important to note that the error state returned from any CUDA API call can also indicate an error from a previously issued asynchronous operation. Section [Asynchronous Error Handling](#) covers this in more detail.

2.1.7.1 Error State

The CUDA runtime maintains a `cudaError_t` state for each host thread. The value defaults to `cudaSuccess` and is overwritten whenever an error occurs. `cudaGetLastError` returns current error state and then resets it to `cudaSuccess`. Alternatively, `cudaPeekAtLastError` returns error state without resetting it.

Kernel launches using *triple chevron notation* do not return a `cudaError_t`. It is good practice to check the error state immediately after kernel launches to detect immediate errors in the kernel launch or *asynchronous errors* prior to the kernel launch. A value of `cudaSuccess` when checking the error state immediately after a kernel launch does not mean the kernel has executed successfully or even started execution. It only verifies that the kernel launch parameters and execution configuration passed to the runtime did not trigger any errors and that the error state is not a previous or asynchronous error before the kernel started.

2.1.7.2 Asynchronous Errors

CUDA kernel launches and many runtime APIs are asynchronous. Asynchronous CUDA runtime APIs will be discussed in detail in [Asynchronous Execution](#). The CUDA error state is set and overwritten whenever an error occurs. This means that errors which occur during the execution of asynchronous operations will only be reported when the error state is examined next. As noted, this may be a call to `cudaGetLastError`, `cudaPeekAtLastError`, or it could be any CUDA API which returns `cudaError_t`.

When errors are returned by CUDA runtime API functions, the error state is not cleared. This means that error code from an asynchronous error, such as an invalid memory access by a kernel, will be returned by every CUDA runtime API until the error state has been cleared by calling `cudaGetLastError`.

```
vecAdd<<<blocks, threads>>>(devA, devB, devC);
// check error state after kernel launch
CUDA_CHECK(cudaGetLastError());
// wait for kernel execution to complete
// The CUDA_CHECK will report errors that occurred during execution of the
→kernel
CUDA_CHECK(cudaDeviceSynchronize());
```

Note

The `cudaError_t` value `cudaErrorNotReady`, which may be returned by `cudaStreamQuery` and `cudaEventQuery`, is not considered an error and is not reported by `cudaPeekAtLastError` or `cudaGetLastError`.

2.1.7.3 CUDA_LOG_FILE

Another good way to identify CUDA errors is with the `CUDA_LOG_FILE` environment variable. When this environment variable is set, the CUDA driver will write error messages encountered out to a file whose path is specified in the environment variable. For example, take the following incorrect CUDA code, which attempts to launch a thread block which is larger than the maximum supported by any architecture.

```
__global__ void k()
{ }

int main()
{
    k<<<8192, 4096>>>(); // Invalid block size
    CUDA_CHECK(cudaGetLastError());
    return 0;
}
```

Building and running this, the check after the kernel launch detects and reports the error using the macros illustrated in [Section 2.1.7](#).

```
$ nvcc errorLogIllustration.cu -o errlog
$ ./errlog
CUDA Runtime Error: /home/cuda/intro-cpp/errorLogIllustration.cu:24:1 =
→invalid argument
```

However, when the application is run with `CUDA_LOG_FILE` set to a text file, that file contains a bit more information about the error.

```
$ env CUDA_LOG_FILE=cudaLog.txt ./errlog
CUDA Runtime Error: /home/cuda/intro-cpp/errorLogIllustration.cu:24:1 =
→invalid argument
$ cat cudaLog.txt
[12:46:23.854][137216133754880][CUDA][E] One or more of block dimensions of
→(4096,1,1) exceeds corresponding maximum value of (1024,1024,64)
[12:46:23.854][137216133754880][CUDA][E] Returning 1 (CUDA_ERROR_INVALID_
→VALUE) from cuLaunchKernel
```

Setting `CUDA_LOG_FILE` to `stdout` or `stderr` will print to standard out and standard error, respectively. Using the `CUDA_LOG_FILE` environment variable, it is possible to capture and identify CUDA errors, even if the application does not implement proper error checking on CUDA return values. This approach can be extremely powerful for debugging, but the environment variable alone does not allow an application to handle and recover from CUDA errors at runtime. The *error log management* feature of CUDA also allows a callback function to be registered with the driver which will be called whenever an error is detected. This can be used to capture and handle errors at runtime, and also to integrate CUDA error logging seamlessly into an application's existing logging system.

Section 4.8 shows more examples of the error log management feature of CUDA. Error log management and `CUDA_LOG_FILE` are available with NVIDIA Driver version r570 and later.

2.1.8. Device and Host Functions

The `__global__` specifier is used to indicate the entry point for a kernel. That is, a function which will be invoked for parallel execution on the GPU. Most often, kernels are launched from the host, however it is possible to launch a kernel from within another kernel using *dynamic parallelism*.

The specifier `__device__` indicates that a function should be compiled for the GPU and be callable from other `__device__` or `__global__` functions. A function, including class member functions, functors, and lambdas, can be specified as both `__device__` and `__host__` as in the example below.

2.1.9. Variable Specifiers

CUDA specifiers can be used on static variable declarations to control placement.

- ▶ `__device__` specifies that a variable is stored in *Global Memory*
- ▶ `__constant__` specifies that a variable is stored in *Constant Memory*
- ▶ `__managed__` specifies that a variable is stored as *Unified Memory*
- ▶ `__shared__` specifies that a variable is store in *Shared Memory*

When a variable is declared with no specifier inside a `__device__` or `__global__` function, it is allocated to registers when possible, and *local memory* when necessary. Any variable declared with no specifier outside a `__device__` or `__global__` function will be allocated in system memory.

2.1.9.1 Detecting Device Compilation

When a function is specified with `__host__` `__device__`, the compiler is instructed to generate both a GPU and a CPU code for this function. In such functions, it may be desirable to use the preprocessor to specify code only for the GPU or the CPU copy of the function. Checking whether `__CUDA_ARCH__` is defined is the most common way of doing this, as illustrated in the example below.

2.1.10. Thread Block Clusters

From compute capability 9.0 onward, the CUDA programming model includes an optional level of hierarchy called thread block clusters that are made up of thread blocks. Similar to how threads in a thread block are guaranteed to be co-scheduled on a streaming multiprocessor, thread blocks in a cluster are also guaranteed to be co-scheduled on a GPU Processing Cluster (GPC) in the GPU.

Similar to thread blocks, clusters are also organized into a one-dimension, two-dimension, or three-dimension grid of thread block clusters as illustrated by [Figure 5](#).

The number of thread blocks in a cluster can be user-defined, and a maximum of 8 thread blocks in a cluster is supported as a portable cluster size in CUDA. Note that on GPU hardware or MIG configurations which are too small to support 8 multiprocessors the maximum cluster size will be reduced accordingly. Identification of these smaller configurations, as well as of larger configurations supporting a thread block cluster size beyond 8, is architecture-specific and can be queried using the `cudaOccupancyMaxPotentialClusterSize` API.

All the thread blocks in the cluster are guaranteed to be co-scheduled to execute simultaneously on a single GPU Processing Cluster (GPC) and allow thread blocks in the cluster to perform hardware-supported synchronization using the [cooperative groups](#) API `cluster.sync()`. Cluster group also provides member functions to query cluster group size in terms of number of threads or number of blocks using `num_threads()` and `num_blocks()` API respectively. The rank of a thread or block in the cluster group can be queried using `dim_threads()` and `dim_blocks()` API respectively.

Thread blocks that belong to a cluster have access to the *distributed shared memory*, which is the combined shared memory of all thread blocks in the cluster. Thread blocks in a cluster have the ability to read, write, and perform atomics to any address in the distributed shared memory. [Distributed Shared Memory](#) gives an example of performing histograms in distributed shared memory.

Note

In a kernel launched using cluster support, the `gridDim` variable still denotes the size in terms of number of thread blocks, for compatibility purposes. The rank of a block in a cluster can be found using the [Cooperative Groups](#) API.

2.1.10.1 Launching with Clusters in Triple Chevron Notation

A thread block cluster can be enabled in a kernel either using a compile-time kernel attribute using `__cluster_dims__(X, Y, Z)` or using the CUDA kernel launch API `cudaLaunchKernelEx`. The example below shows how to launch a cluster using a compile-time kernel attribute. The cluster size using kernel attribute is fixed at compile time and then the kernel can be launched using the classical `<<<, >>>`. If a kernel uses compile-time cluster size, the cluster size cannot be modified when launching the kernel.

```
// Kernel definition
// Compile time cluster size 2 in X-dimension and 1 in Y and Z dimension
__global__ void __cluster_dims__(2, 1, 1) cluster_kernel(float *input, float*
→output)
{
}

int main()
{
```

(continues on next page)

(continued from previous page)

```

float *input, *output;
// Kernel invocation with compile time cluster size
dim3 threadsPerBlock(16, 16);
dim3 numBlocks(N / threadsPerBlock.x, N / threadsPerBlock.y);

// The grid dimension is not affected by cluster launch, and is still
→enumerated
// using number of blocks.
// The grid dimension must be a multiple of cluster size.
cluster_kernel<<<numBlocks, threadsPerBlock>>>(input, output);
}

```

2.2. Intro to CUDA Python

This chapter introduces CUDA kernel programming in Python. The CUDA Python ecosystem encapsulates a large and actively evolving ecosystem of tools and libraries. This chapter will start by introducing some of these components and then use a few of them to illustrate methods for writing and executing GPU code in Python.

There are a large number of ways to leverage GPU computing in Python, many of which do not require writing GPU kernels explicitly. Some components of the [CUDA Python Ecosystem](#) provide functions which do their operations on the GPU without the developer needing to do any specific GPU control or coding. The [NVIDIA Accelerated Computing Hub](#) has an [Accelerated Python User's Guide](#), which introduces and discusses many of the different libraries and tools which enable GPU-accelerated computing in Python. This resource is a good starting point for users looking to use GPUs in Python as quickly and easily as possible without necessarily writing GPU code directly.

This chapter, on the other hand, focuses on direct control of the GPU and writing kernels in Python that execute on the GPU. This chapter focuses on [CUDA Single Instruction Multiple Thread \(SIMT\)](#) programming in Python.

2.2.1. CUDA Python Ecosystem

CUDA Python is an ecosystem of tools and libraries enabling GPU computing in Python. The following list introduces the main pieces of CUDA Python, not all of which are required for the content covered here. This list is adapted from the complete list found at the [CUDA Python github repository](#).

Main Components - For GPU control and running library-provided GPU code

- ▶ `cuda.core` - Pythonic interface for CUDA controls such as memory and device management. Provides for Python what the CUDA Runtime provides for CUDA C++.
- ▶ `cuda.compute` - A Python module which provides GPU-accelerated functions provided by the [CUDA Core Compute Library \(CCCL\)](#).
- ▶ CuPy - A Python library which provides GPU-accelerated versions of Numpy routines, and also a GPU-accelerated version of the `ndarray` data container.

Kernel Authoring Components

- ▶ `cuda.lang` - A Python Domain-specific language (DSL) for writing CUDA kernels and device functions in the SIMT programming model using a subset the of Python language.

- ▶ `cuda.coop` - A Python module providing device-callable (using `cuda.lang`) primitives of the [CUDA Core Compute Library \(CCCL\)](#).
- ▶ `cuda.tile` - A Python Domain-specific language (DSL) for writing CUDA kernels and device functions in the Tile programming model.

Other Components

- ▶ `cuda.pathfinder` - A utility for locating CUDA components installed in the Python environment
- ▶ `cuda.bindings` - Low level Python bindings for CUDA libraries and utilities including the CUDA Driver API, CUDA Runtime API, NVRTC, NVVM, and others. `cuda.bindings` provides the same functionality available in `cuda.core` via the CUDA Driver and CUDA Runtime components. However, `cuda.bindings` provides these as Python wrappers on the C-language APIs, not native Pythonic interfaces.

2.2.1.1 Using CUDA Libraries in Python

CUDA C++ has a rich ecosystem of libraries which allow for GPU acceleration without the need to write kernel or GPU code directly. When CUDA C++ was introduced in 2006, there were few libraries and developers largely had to write GPU kernel code themselves. Since then, [a large number of libraries](#) have been developed which allow developers to take advantage of GPU computing in C++ without writing much, if any, GPU code.

The CUDA Python ecosystem has evolved from the other direction: Python libraries such as CuPy provided GPU-accelerated implementations of computations and algorithms to Python developers before the ability to write custom kernels directly using Python syntax and semantics became available. Many of these libraries provided Python bindings to GPU code implemented in CUDA C++.

In the modern era of CUDA, it is almost always advisable to use GPU-accelerated libraries if they provide the necessary expressiveness for your needs. Many of these libraries provide implementations tuned by GPU computing experts. When libraries are not available or sufficient, writing GPU kernels and device functions directly is available in Python as in C++.

2.2.1.2 Scope of This Chapter

While developers should prefer to use libraries when possible, the rest of this chapter introduces how to proceed when developers need to write custom GPU code in Python. This chapter covers writing and running custom Python code on the GPU in the same fashion as [Section 2.1](#) does for C++, starting with how to specify a GPU kernel and then how to use the GPU-accelerated `ndarray` provided by CuPy to allocate memory on the GPU and provide communication between the CPU and GPU.

2.2.1.3 Getting Setup

In general, most of the CUDA Python ecosystem components are available on PyPi and can be installed using `pip` commands or any popular package manager for Python. All packages require that an up-to-date NVIDIA Driver is installed on the system. The CUDA Toolkit is generally not required to write or run applications in CUDA Python.

See [CUDA Python on the NVIDIA Developer Zone](#) for information about installing and configuring CUDA Python for different platforms.

2.2.1.4 Running CUDA Python Applications

CUDA Python applications, whether using CUDA-accelerated libraries or having user-written GPU code, are run in the same way as conventional Python applications. In this section, examples will always be run from a command line by invoking `python3` as shown below to execute a program called `cuda-python-app.py`.

```
$ python3 cuda-python-app.py
```

2.2.2. SIMT Kernels in Python

As mentioned in the introduction to the *CUDA Programming Model*, functions which execute on the GPU which can be invoked from the host are called kernels. CUDA provides two different models: *Single Instruction Multiple Thread (SIMT)*, and *CUDA tile*. SIMT kernels are written to be run by many parallel threads simultaneously. This notion is the same in CUDA Python and CUDA C++. This chapter introduces CUDA Python using SIMT kernels.

2.2.2.1 Specifying Kernels

Before specifying kernels in CUDA Python, the package `numba.cuda` must be imported. This is commonly done as shown below.

```
from numba import cuda
```

This imports the `numba.cuda` package and allows us to use the components of the `cuda` namespace provided by that package.

To specify a function as a kernel in CUDA Python, place the decorator `@cuda.jit` on the line above the function definition as shown below.

```
from numba import cuda

@cuda.jit
def function(input_array, output_array):
    ...
```

This will cause the kernel to be JIT compiled for the active GPU the first time it is launched. The default CUDA device is used when no other GPU is specified, as is the case for the examples shown in this section.

2.2.2.2 Launching Kernels

The number of threads that will execute a kernel are specified as part of the kernel launch. This is called the execution configuration. Each invocation of a kernel can have a unique execution configuration, such as a different block size or number of thread blocks.

2.2.2.2.1 Kernel Launch

To launch a kernel, the execution configuration is placed in square braces `[ ]` after the kernel name but before the function arguments. The order of arguments is the same as in the triple chevron notation in C++ introduced in [Section 2.1.2.2.1](#), specifically:

```
kernel_name[number_of_thread_blocks, threads_per_block](arguments, ...)
```

The code snippet below shows how a kernel is defined and then invoked in a Python source file.

```
from numba import cuda

@cuda.jit
```

(continues on next page)

(continued from previous page)

```
def my_kernel(input, output):
    ...

## launch the kernel
my_kernel[num_thread_blocks, threads_per_block](in_array, out_array)
```

There is a limit to the number of threads per block, since all threads of a block reside on the same streaming multiprocessor (SM) and must share the resources of the SM. On current GPUs, a thread block may contain up to 1024 threads. If resources allow, more than one thread block can be scheduled on an SM simultaneously.

2.2.2.2 Multi-Dimension Grids and Thread Blocks

Thread blocks and the grid of thread blocks can be 1, 2, or 3-dimensional in CUDA. When the grid or thread block is one dimensional, an integer can be used when specifying the execution configuration of a kernel launch. When the thread block or grid is 2 or 3-dimensional, a two or three dimensional tuple is used as shown below for a 2D grid and thread block launch, where `gridX` and `gridY` are the x and y dimensions of the grid, and `blockX` and `blockY` are the x and y dimensions of each thread block within the grid.

```
from numba import cuda

@cuda.jit
def function(input, output):
    ...

## launch the kernel
function[(gridX, gridY), (blockX, blockY)](in_array, out_array)
```

2.2.2.3 Thread and Grid Index Intrinsics

[Section 1.2.2.1](#) introduced threads and grids, and [Section 2.2.2.2](#) showed how to specify the grid and thread block size for a kernel launch. Within a kernel, each thread can access the parameters of the execution configuration as well as the thread's index and thread block index within the grid.

The following variables can be accessed from within a kernel function to determine a thread's identity:

- ▶ `cuda.threadIdx.[xyz]` gives the index of a thread within its thread block. Each thread in a thread block will have a different index.
- ▶ `cuda.blockDim.[xyz]` gives the dimensions of the thread block, which was specified in the execution configuration of the kernel launch.
- ▶ `cuda.blockIdx.[xyz]` gives the index of a thread block within the grid. Each thread block will have a different index.
- ▶ `cuda.gridDim.[xyz]` gives the dimensions of the grid, which was specified in the execution configuration when the kernel was launched.

Each of these variables is a 3-component vector with a `.x`, `.y`, and `.z` member. If a dimension is not specified in the execution configuration at kernel launch, it defaults to a value of 1 for dimensions and 0 for indices.

`cuda.threadIdx` and `cuda.blockIdx` are zero indexed. That is, `cuda.threadIdx.x` will take on values from 0 up to and including `cuda.blockDim.x - 1`. `.y` and `.z` operate the same in their respective dimensions.

The code for a simple vector addition kernel, which adds two vectors together element-wise, is shown below. This function takes three arrays, A, B, and C to implement an element-wise vector addition $C = A + B$

```
# C = A + B vector addition
@cuda.jit
def vecadd(A, B, C):
    idx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    C[idx] = A[idx] + B[idx]
```

The kernel begins by calculating the unique index of the thread in the grid. This kernel assumes it is being launched with 1-dimensional thread blocks in a 1-dimensional grid. The `idx` variable is a unique index from 0 to $N-1$, where N is the total number of threads in the grid, that is, $N = \text{cuda.gridDim.x} * \text{cuda.blockDim.x}$.

The pattern for computing the thread index as shown in the code block above is so common, Numba provides a shorthand syntax for this operation: `cuda.grid(n)`, where n is the number of dimensions. In the example above, the line

```
idx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
```

can be replaced by the much simpler

```
idx = cuda.grid(1)
```

One notable aspect of this kernel is that it does not check for out-of-bounds access to A, B, or C. In this chapter, we assume that these are `ndarray`s created by `cuPy`, which will be introduced shortly in [Section 2.2.3.3](#). When using `cuPy ndarray`s, bounds checking is implicitly implemented by the array type.

2.2.3. Memory in GPU computing

Note

Python packages like `cuPy` and others perform GPU memory management by directly using CUDA C++ APIs, such as those covered in [Section 2.1.3.2](#). Multiple Python packages provide wrappers and utilities for controlling GPU memory allocation. In this guide, only `cuPy` will be covered. The concepts are similar for most packages, and most behave in similar fashion to their C++ counterparts except where noted.

As introduced in [Section 1.2.3](#), the GPU has an attached DRAM. Data arrays which will be used in kernels generally need to be in the GPU's DRAM before they are accessed from the kernel. In Python, controlling the location of data in memory, i.e. moving data between the CPU and GPU, is the responsibility of the programmer. This is the same situation as explicit memory management in C++, as introduced in [Section 2.1.3.2](#).

2.2.3.1 Instantiating arrays on the GPU

`CuPy` provides functions to create `ndarray` objects of a specified type and dimension on the GPU and also to copy data between the CPU and GPU. Many of the functions in `cuPy` have similar function signatures to functions for creating `ndarray`s in `Numpy`. The following shows a few examples of how to create and fill arrays in GPU memory using `CuPy`.

```
import cupy as cp
import numpy as np

## create a matrix of zeros on the GPU
## when a datatype is not specified, float32 is used by default
A_device = cp.zeros((1024, 1024))

## create an array of 2^20 random doubles on the GPU
B_device = cp.rand.random((2**20), dtype=np.double)

## create an array of zeroes with the same shape and datatype as an existing
→array
C_device = cp.zeros_like(A)
```

2.2.3.2 Copying Arrays between the Host and GPU memory

CuPy can also be used to copy data from Numpy ndarrays which reside in CPU memory to CuPy arrays which reside in GPU memory.

```
import cupy as cp
import numpy as np

## Create an array in host memory
A_host = np.zeros((1024, 1024))
## Copy the array to the GPU
A_device = cp.array(A_host)

## Create an array in GPU memory
B_device = cp.rand.random((1024, 1024))
## copy the array to host memory
B_host = cp.asnumpy(B_device)
```

2.2.3.3 The ndarray object type

The ndarray objects shown in the previous section exist either in the host memory or in the GPU memory, but not both. Passing arrays which reside on the host as arguments to kernels will result in an error. Passing arrays which reside in GPU memory to normal Python functions, i.e. not kernels, will also result in an error. CuPy does not implicitly perform the copies between CPU and GPU because they can be expensive and excessive data copying can hurt performance. As a result, CuPy requires the programmer to be intentional about when data is copied between CPU and GPU.

One advantage of using the ndarray type in GPU kernels is that the array carries with it the extents of its dimensions. As shown in [Section 2.2.2.3](#), boundary checking is done automatically and the kernel code does not need to check for out-of-bounds access when the total number of threads needed is slightly less than the total size of the execution block or grid.

2.2.4. Synchronizing the CPU and the GPU

Like C++, kernel launches in CUDA Python are asynchronous with respect to the host thread. That is, the host code proceeds executing on the CPU after a kernel launch without any guarantee that the kernel has completed or even started executing. In order to guarantee that GPU kernels have completed executing, the host thread must perform some form of synchronization with the GPU.

The simplest form of synchronization is to synchronize the entire GPU. This device-wide synchronization is an operation provided by the CUDA driver and is exposed to Python by both cuPy and numba.cuda as the method `synchronize()`.

```
import cupy as cp
from numba import cuda

..

## Wait on host thread for all pending GPU work to complete
## this uses the interface provided by cupy
cp.synchronize()

## Wait on host thread for all pending GPU work to complete
## this uses the interface provided by numba.cuda
cuda.synchronize()
```

Device-wide synchronization waits on the host thread until all previously issued work on the GPU has completed. Finer-grain synchronization is available using CUDA streams as described in [Section 2.5](#). In Python, when using streams, recommended practice is to use `cuda.core` to create CUDA streams and perform synchronization only with specific streams as needed.

2.2.5. Putting it All Together

The ubiquitous first GPU kernel to perform parallel vector addition is shown in its Python form in the source listing below.

```
import numpy as np
from numba import cuda
import cupy as cp

## Defines a CUDA kernel to perform C = A + B vector addition
@cuda.jit
def vecadd(A, B, C):
    work_index = cuda.grid(1)
    C[work_index] = A[work_index] + B[work_index]

# note that vector size is not a power of 2 nor a multiple of the block_size
→defined below
vector_size = 2**24 + 11

device = cp.cuda.Device()
## Create device arrays of uniform random float32 values as input, and an array
→of zeros
## as the result vector
a = cp.random.uniform(-1, 1, vector_size)
b = cp.random.uniform(-1, 1, vector_size)
c = cp.zeros_like(a)

block_size = 256
grid_size = int(np.ceil(vector_size/block_size))
```

(continues on next page)

(continued from previous page)

```

vecadd[grid_size, block_size](a, b, c)

## synchronize the CPU thread and the GPU to ensure that the kernel has
→completed
## this is included to illustrate good practices, even though the copy below
→would implicitly wait for
## the kernel to complete
device.synchronize()

## Copy all 3 arrays to the CPU as ndarrays
a_np = cp.asnumpy(a)
b_np = cp.asnumpy(b)
c_np = cp.asnumpy(c)

## Perform the copy on the CPU to verify the answer
expected = a_np + b_np

## Test that the answer is correct, within floating point epsilon
np.testing.assert_array_almost_equal(c_np, expected)

## The assert will print diagnostics and abort
## so this only prints if the assertion passes
print("Test succeeded")

```

In this example, the A and B input arrays are created and initialized to random values on the GPU by CuPy. They are copied to the CPU at the end of the code only so that the CPU can perform the vector addition as well and verify that the CPU and GPU answers match.

2.2.6. Error Checking in CUDA Python

Any operation that affects the GPU, from memory allocation and copies to kernel launches can potentially cause an error condition to arise. As illustrated in [Section 2.1.7](#) for C++, making sure that errors have not occurred in the course of interacting with the GPU is best practice.

In Python, CUDA errors throw exceptions which will terminate the program if they are not caught. The exceptions can be caught using normal Python syntax. The example below shows the same vector add as above, but with an error intentionally added: the number of threads per block, 2048, is larger than any current GPU can run. This will cause the kernel to fail to launch, and throw an exception, which this code will catch.

```

import numpy as np
from numba import cuda
import cupy as cp

## Defines a CUDA kernel to perform C = A + B vector addition
@cuda.jit
def vecadd(A, B, C):
    work_index = cuda.grid(1)
    C[work_index] = A[work_index] + B[work_index]

```

(continues on next page)

(continued from previous page)

```

try:
    vector_size = 2**24 + 11

    device = cp.cuda.Device()
    a = cp.random.uniform(-1, 1, vector_size)
    b = cp.random.uniform(-1, 1, vector_size)
    c = cp.zeros_like(a)

    ## this block size is too large for any current GPUs
    block_size = 2048
    grid_size = int(np.ceil(vector_size/block_size))
    # Error: launching kernel with invalid block size
    vecadd[grid_size, block_size](a, b, c)

    device.synchronize()
    print("Test did not encounter any errors")

except Exception as e:
    print(f"Exception occurred: {e}")

```

Running this code causes the error to be caught and displayed as shown below:

```

$ python3 vecadd_error.py
Exception occurred: CUDA_ERROR_INVALID_VALUE: This indicates that one or more
↳ of the parameters passed to the API call is not within an acceptable range
↳ of values.

```

The program exits normally having caught the exception. If this code was run without the `try:` and `except:`, it would exit abnormally and dump a traceback to the console which should show the same error.

2.3. Writing SIMT Kernels

CUDA kernels can largely be written in the same way that traditional CPU code would be written for a given problem. However, there are some unique features of the GPU that can be used to improve performance. Additionally, some understanding of how threads on the GPU are scheduled, how they access memory, and how their execution proceeds can help developers write kernels that maximize utilization of the available computing resources.

This chapter covers some more details of writing kernels using the *SIMT programming model* in both C++ and Python.

2.3.1. Basics of SIMT

From the developer's perspective, the CUDA thread is the fundamental unit of parallelism. [Section 1.2.2.2](#) describes the basic SIMT model of GPU execution and *SIMT Execution Model* provides additional details of the SIMT model. The SIMT model allows each thread to maintain its own state and control flow. From a functional perspective, each thread can execute a separate code path. However, substantial performance improvements can be realized by taking care that kernel code minimizes the situations where threads in the same warp take divergent code paths.

2.3.2. Thread Hierarchy

Threads are organized into thread blocks, which are then organized into a grid. Grids may be 1, 2, or 3 dimensional and the size of the grid can be queried inside a kernel with the `gridDim` built-in variable. Thread blocks may also be 1, 2, or 3 dimensional. The size of the thread block can be queried inside a kernel with the `blockDim` built-in variable. The index of the thread block can be queried with the `blockIdx` built-in variable. Within a thread block, the index of the thread is obtained using the `threadIdx` built-in variable. These built-in variables are used to compute a unique global thread index for each thread, thereby enabling each thread to load/store specific data from global memory and execute a unique code path as needed.

C++

- ▶ `gridDim.[x|y|z]`: Size of the grid in the x, y and z dimension respectively. These values are the same for all threads and are part of the execution configuration set at kernel launch.
- ▶ `blockDim.[x|y|z]`: Size of the block in the x, y and z dimension respectively. These values are the same for all threads and are part of the execution configuration set at kernel launch.
- ▶ `blockIdx.[x|y|z]`: Index of the block in the x, y and z dimension respectively. These values will differ among threads, to indicate which thread block is executing.
- ▶ `threadIdx.[x|y|z]`: Index of the thread in the x, y and z dimension respectively. These values will differ among threads, to indicate which thread is executing.

Python

- ▶ `cuda.threadIdx.[xyz]`: Size of the grid in the x, y and z dimension respectively. These values are the same for all threads and are part of the execution configuration set at kernel launch.
- ▶ `cuda.blockDim.[xyz]`: Size of the block in the x, y and z dimension respectively. These values are the same for all threads and are part of the execution configuration set at kernel launch.
- ▶ `cuda.blockIdx.[xyz]`: Index of the block in the x, y and z dimension respectively. These values will differ among threads, to indicate which thread block is executing.
- ▶ `cuda.gridDim.[xyz]`: Index of the thread in the x, y and z dimension respectively. These values will differ among threads, to indicate which thread is executing.

The use of multi-dimensional thread blocks and grids is for convenience only and does not affect performance. The threads of a block are linearized predictably: the first index x moves the fastest, followed by y and then z. This means that in the linearization of a thread indices, consecutive values of `threadIdx.x` indicate consecutive threads, `threadIdx.y` has a stride of `blockDim.x`, and `threadIdx.z` has a stride of `blockDim.x * blockDim.y`. This affects how threads are assigned to warps, as detailed in [Hardware Multithreading](#).

Figure 11 shows a simple example of a 2D grid, with 1D thread blocks.

2.3.2.1 Thread Block Synchronization

In examples shown previously, there was not a need to synchronize threads within a thread block. When threads within a thread block cooperate or access the same memory addresses, especially in [shared memory](#), described below, synchronization is necessary to avoid race conditions and memory hazards.

The most basic form of synchronization within a block is called `syncthreads`

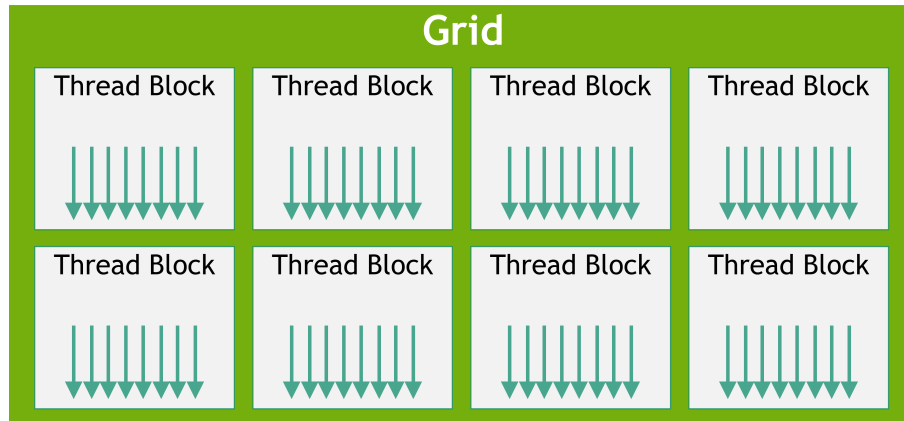


Figure 11: Grid of Thread Blocks

2.3.3. GPU Device Memory Spaces

CUDA devices have several memory spaces that can be accessed by CUDA threads within kernels. [Table 1](#) shows a summary of the common memory types, their thread scopes, and their lifetimes. The following sections explain each of these memory types in more detail.

Table 1: Memory Types, Scopes and Lifetimes

Memory Type	Scope	Lifetime	Location
Global	Grid	Application	Device
Constant	Grid	Application	Device
Shared	Block	Kernel	SM
Local	Thread	Kernel	Device
Register	Thread	Kernel	SM

2.3.3.1 Global Memory

Global memory (also called device memory) is the primary memory space for storing data that is accessible by all threads in a kernel. It is similar to RAM in a CPU system. Kernels running on the GPU have direct access to global memory in the same way code running on the CPU has access to system memory.

Global memory is persistent. That is, an allocation made in global memory and the data stored in it persist until the allocation is freed or until the application is terminated. `cudaDeviceReset` also frees all allocations.

Global memory is allocated with CUDA API calls such as `cudaMalloc` and `cudaMallocManaged`. Data can be copied into global memory from CPU memory using CUDA runtime API calls such as `cudaMemcpy`. Global memory allocations made with CUDA APIs are freed using `cudaFree`.

Prior to a kernel launch, global memory is allocated and initialized by CUDA API calls. During kernel execution, data from global memory can be read by the CUDA threads, and the result from operations carried out by CUDA threads can be written back to global memory. Once a kernel has completed execution, the results it wrote to global memory can be copied back to the host or used by other

kernels on the GPU.

Because global memory is accessible by all threads in a grid, care must be taken to avoid data races between threads. Since CUDA kernels launched from the host have the return type `void`, the only way for numerical results computed by a kernel to be returned to the host is by writing those results to global memory.

A simple example illustrating the use of global memory is the kernel shown below, where the three arrays A, B, and C are in global memory and are being accessed by this vector add kernel.

C++

```
__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}
```

Python

```
@cuda.jit
def vecadd(A, B, C):
    work_index = cuda.grid(1)
    C[work_index] = A[work_index] + B[work_index]
```

2.3.3.2 Shared Memory

Shared memory is a memory space that is accessible by all threads in a thread block. It is physically located on each SM and uses the same physical resource as the L1 cache, the unified data cache. The data in shared memory persists throughout the kernel execution. Shared memory can be considered a user-managed scratchpad for use during kernel execution. While small in size compared to global memory, because shared memory is located on each SM, the bandwidth is higher and the latency is lower than accessing global memory.

Since shared memory is accessible by all threads in a thread block, care must be taken to avoid data races between threads in the same thread block. Synchronization between threads in the same thread block can be achieved using the `__syncthreads()` function in C++. In Python, this same thread block synchronization is achieved using `cuda.syncthreads()`. This function blocks all threads in the thread block until all threads have reached the call to `__syncthreads()` or `cuda.syncthreads()`.

C++

```
// assuming blockDim.x is 128
__global__ void example_syncthreads(int* input_data, int* output_data)
{
    __shared__ int shared_data[128];
    shared_data[threadIdx.x] = input_data[blockDim.x*blockIdx.x + threadIdx.
    ↪x];
```

(continues on next page)

(continued from previous page)

```

    // All threads synchronize, guaranteeing all writes to 'shared_data' are
    →ordered
    // before any thread is unblocked from '__syncthreads()':
    __syncthreads();

    // A single thread safely reads 'shared_data':
    if (threadIdx.x == 0) {
        float sum = 0;
        for (int i = 0; i < blockDim.x; ++i) {
            sum += shared_data[i];
        }
        output_data[blockIdx.x] = sum;
    }
}

```

Python

```

import numpy as np
from numba import cuda
import cupy as cp

```

```

@cuda.jit
def example_syncthreads(input_data, output_data):
    shared_data = cuda.shared.array(shape=128, dtype=np.int32)

    shared_data[cuda.threadIdx.x] = input_data[cuda.blockIdx.x*cuda.blockDim.
    →x + cuda.threadIdx.x]
    cuda.syncthreads()

    if cuda.threadIdx.x == 0:
        sum = 0.0
        for x in shared_data:
            sum = sum + x
        output_data[cuda.blockIdx.x] = sum

```

The size of shared memory varies depending on the GPU architecture being used. Because shared memory and L1 cache share the same physical space, using shared memory reduces the size of the usable L1 cache for a kernel. Additionally, if no shared memory is used by the kernel, the entire physical space will be utilized by L1 cache. The CUDA runtime API provides functions to query the shared memory size on a per SM basis and a per thread block basis, using the `cudaGetDeviceProperties` function and investigating the `cudaDeviceProp.sharedMemPerMultiprocessor` and `cudaDeviceProp.sharedMemPerBlock` device properties.

The CUDA runtime API provides a function `cudaFuncSetCacheConfig` to tell the runtime whether to allocate more space to shared memory, or more space to L1 cache. This function specifies a preference to the runtime, but is not guaranteed to be honored. The runtime is free to make decisions based on the available resources and the needs of the kernel.

Shared memory can be allocated both statically and dynamically.

2.3.3.2.1 Static Allocation of Shared Memory

To allocate shared memory statically, the programmer must declare a variable inside the kernel using the `__shared__` specifier in C++ or the `cuda.shared.array()` in Python. The array will be allocated in shared memory and will persist for the duration of the kernel execution. The size of the shared memory declared in this way must be specified at compile time. For example, the following code snippet, located in the body of the kernel, declares a shared memory array of type `float` with 1024 elements.

C++

```
__shared__ float sharedArray[1024];
```

Python

```
from numba import cuda
import numpy as np

shared_array = cuda.shared.array(shape=1024, dtype=np.float32)
```

After this declaration, all the threads in the thread block will have access to this shared memory array.

2.3.3.2.2 Dynamic Allocation of Shared Memory

To allocate shared memory dynamically in C++, the programmer can specify the desired amount of shared memory per thread block in bytes as the third (and optional) argument to the kernel launch in the triple chevron notation like this `functionName<<<grid, block, sharedMemoryBytes>>>()`. When this parameter is not specified, it defaults to 0.

In Python, `cuda.core.launch()` must be used to launch the kernel. The `LaunchConfig` parameter takes a `cuda.core.LaunchConfig` object, which has a field called `shmem_size` which serves the same function as the 3rd argument in the triple chevron notation in C++.

Inside the kernel, the programmer can use the extern `__shared__` specifier with empty `[]` in C++ to declare a variable that will be allocated dynamically at kernel launch. In Python, the same `cuda.shared.array` method used to statically allocate shared memory is used with the `shape` argument set to 0.

C++

```
extern __shared__ float sharedArray[];
```

Python

```
from numba import cuda
import numpy as np

## setting shape=0 indicates that this array will be dynamically sized by the
→kernel launch's execution configuration
shared_array = cuda.shared.array(shape=0, type=np.float32)
```

One caveat is that a kernel can have only one dynamically allocated shared array. If one wants multiple dynamically allocated shared memory arrays, a single dynamically allocated shared memory array which is large enough to contain all the desired arrays must be allocated and manually partitioned. In C++, for example, if one wants the equivalent of the following,

C++

```
short array0[128];
float array1[64];
int array2[256];
```

in dynamically allocated shared memory, one could declare and initialize the arrays in the following way:

C++

```
extern __shared__ float array[];

short* array0 = (short*)array;
float* array1 = (float*)&array0[128];
int* array2 = (int*)&array1[64];
```

Note that pointers need to be aligned to the type they point to, so the following code, for example, does not work since `array1` is not aligned to 4 bytes.

C++

```
extern __shared__ float array[];
short* array0 = (short*)array;
float* array1 = (float*)&array0[127];
```

In Python, since there are no pointers, this form of type punning is not available.

2.3.3.3 Registers

Registers are located on the SM and have thread local scope. Register usage is managed by the compiler and registers are used for thread local storage during the execution of a kernel. The number of registers per SM and the number of registers per thread block can be queried using the `regsPerMultiprocessor` and `regsPerBlock` device properties of the GPU.

When compiling C++ code, NVCC allows the developer to **specify a maximum number of registers** to be used by a kernel via the `-maxrregcount` option. Using this option to reduce the number of registers a kernel can use may result in more thread blocks being scheduled on the SM concurrently, but may also result in more register spilling. Register spilling means that values currently stored on-chip in registers must be written out to global memory and then read back later to make space for other values.

2.3.3.4 Local Memory

Local memory is thread local storage similar to registers and managed by NVCC, but the physical location of local memory is in the global memory space. The 'local' label refers to its logical scope, not its physical location. Local memory is used for thread local storage during the execution of a kernel. Automatic variables that the compiler is likely to place in local memory are:

- ▶ Arrays for which it cannot determine that they are indexed with constant quantities,
- ▶ Large structures or arrays that would consume too much register space,
- ▶ Any variable if the kernel uses more registers than available, that is register spilling.

Because the local memory space resides in device memory, local memory accesses have the same latency and bandwidth as global memory accesses and are subject to the same requirements for memory coalescing as described in [Coalesced Global Memory Access](#). However, local memory is organized

such that consecutive 32-bit words are accessed by consecutive thread IDs. Accesses are therefore fully coalesced as long as all threads in a warp access the same relative address, such as the same index in an array variable or the same member in a structure variable.

2.3.3.5 Constant Memory

Constant memory has grid scope and is accessible for the lifetime of the application. The constant memory resides on the device and is read-only to the kernel.

In C++, variables or arrays are declared with the `__constant__` specifier outside any kernel or function.

In Python, the method `const_array = numba.cuda.const.array_like(ary)` is specified **within the kernel code** to create a constant memory array containing the data stored in the host array `ary`

Constant memory means that a variable:

- ▶ Resides in constant memory space,
- ▶ Has a distinct object per device,
- ▶ Is accessible from all the threads within the grid and from the host in C++ through the runtime library (`cudaGetSymbolAddress()` / `cudaGetSymbolSize()` / `cudaMemcpyToSymbol()` / `cudaMemcpyFromSymbol()`).

In C++, constant memory has the lifetime of the context in which it was created. In Python, constant memory has the lifetime of the kernel in which it is declared.

The total amount of constant memory can be queried with the `totalConstMem` device property element.

Constant memory is useful for small amounts of data that each thread will use in a read-only fashion. Constant memory is small relative to other memories, typically 64KB per device.

An example snippet of declaring and using constant memory follows.

C++

```
// In your .cu file
__constant__ float coeffs[4];

__global__ void compute(float *out) {
    int idx = threadIdx.x;
    out[idx] = coeffs[0] * idx + coeffs[1];
}

// In your host code
float h_coeffs[4] = {1.0f, 2.0f, 3.0f, 4.0f};
cudaMemcpyToSymbol(coeffs, h_coeffs, sizeof(h_coeffs));
compute<<<1, 10>>>(device_out);
```

Python

```
from numba import cuda
import numpy as np

host_array = np.zeros(128, dtype=np.float32)
## fill host_array with other data
```

(continues on next page)

(continued from previous page)

```
@cuda.jit
def kernel(args):
    ...

    const_array = cuda.const.array_like(a)

    # this access now goes through constant memory
    a = const_array[cuda.threadIdx.x]
```

2.3.3.6 Caches

GPU devices have a multi-level cache structure which includes L2 and L1 caches.

The L2 cache is located on the device and is shared among all the SMs. The size of the L2 cache can be queried with the `l2CacheSize` device property element from the function `cudaGetDeviceProperties`.

As described above in [Shared Memory](#), L1 cache is physically located on each SM and is the same physical space used by shared memory. If no shared memory is utilized by a kernel, the entire physical space will be utilized by the L1 cache.

The L2 and L1 caches can be controlled via functions that allow the developer to specify various caching behaviors. The details of these functions are found in [Configuring L1/Shared Memory Balance](#), [L2 Cache Control](#), and [Low-Level Load and Store Functions](#).

If these hints are not used, the compiler and runtime will do their best to utilize the caches efficiently.

2.3.3.7 Texture and Surface Memory

Note

Some older CUDA code may use texture memory because, in older NVIDIA GPUs, doing so provided performance benefits in some scenarios. On all currently supported GPUs, these scenarios may be handled using direct load and store instructions, and use of texture and surface memory instructions no longer provides any performance benefit for non-texture loads.

A GPU may have specialized instructions for loading data from an image to be used as textures in 3D rendering. CUDA exposes these instructions and the machinery to use them in the [texture object API](#) and the [surface object API](#).

Texture and surface memory do not provide any performance advantage for non-graphics applications in CUDA on any currently supported NVIDIA GPU. These APIs are still useful when reading texture or surface data for rendering, such as writing hit shaders for [NVIDIA OptiX](#), which uses CUDA as its shader language.

For developers working on existing code bases which still use these APIs for non-texture loads, explanations of these APIs can still be found in the legacy [CUDA C++ Programming Guide](#).

2.3.3.8 Distributed Shared Memory

Note

Distributed shared memory is a feature available when using *thread block clusters*. Thread block clusters make use of the *cooperative_groups APIs*, which are currently only available in C++.

Thread Block Clusters, introduced in compute capability 9.0 and facilitated by *Cooperative Groups*, provide the ability for threads in a thread block cluster to access shared memory of all the participating thread blocks in that cluster. This partitioned shared memory is called *Distributed Shared Memory*, and the corresponding address space is called Distributed Shared Memory address space. Threads that belong to a thread block cluster can read, write or perform atomics in the distributed address space, regardless of whether the address belongs to the local thread block or a remote thread block. Whether a kernel uses distributed shared memory or not, the shared memory size specifications, static or dynamic is still per thread block. The size of distributed shared memory is just the number of thread blocks per cluster multiplied by the size of shared memory per thread block.

Accessing data in distributed shared memory requires all the thread blocks to exist. A user can guarantee that all thread blocks have started executing using `cluster.sync()` from *class cluster_group*. The user also needs to ensure that all distributed shared memory operations happen before the exit of a thread block, e.g., if a remote thread block is trying to read a given thread block's shared memory, the program needs to ensure that the shared memory read by the remote thread block is completed before it can exit.

Let's look at a simple histogram computation and how to optimize it on the GPU using thread block clusters. A standard way of computing histograms is to perform the computation in the shared memory of each thread block and then perform global memory atomics. A limitation of this approach is the shared memory capacity. Once the histogram bins no longer fit in the shared memory, a user needs to directly compute histograms and hence the atomics in the global memory. With distributed shared memory, CUDA provides an intermediate step, where depending on the histogram bins size, the histogram can be computed in shared memory, distributed shared memory or global memory directly.

The CUDA kernel example below shows how to compute histograms in shared memory or distributed shared memory, depending on the number of histogram bins.

C++

```
#include <cooperative_groups.h>

// Distributed Shared memory histogram kernel
__global__ void clusterHist_kernel(int *bins, const int nbins, const int bins_
    ↳per_block, const int *__restrict__ input,
                                size_t array_size)
{
    extern __shared__ int smem[];
    namespace cg = cooperative_groups;
    int tid = cg::this_grid().thread_rank();

    // Cluster initialization, size and calculating local bin offsets.
    cg::cluster_group cluster = cg::this_cluster();
    unsigned int clusterBlockRank = cluster.block_rank();
    int cluster_size = cluster.dim_blocks().x;
```

(continues on next page)

(continued from previous page)

```

for (int i = threadIdx.x; i < bins_per_block; i += blockDim.x)
{
    smem[i] = 0; //Initialize shared memory histogram to zeros
}

// cluster synchronization ensures that shared memory is initialized to
→zero in
// all thread blocks in the cluster. It also ensures that all thread blocks
// have started executing and they exist concurrently.
cluster.sync();

for (int i = tid; i < array_size; i += blockDim.x * gridDim.x)
{
    int ldata = input[i];

    //Find the right histogram bin.
    int binid = ldata;
    if (ldata < 0)
        binid = 0;
    else if (ldata >= nbins)
        binid = nbins - 1;

    //Find destination block rank and offset for computing
    //distributed shared memory histogram
    int dst_block_rank = (int)(binid / bins_per_block);
    int dst_offset = binid % bins_per_block;

    //Pointer to target block shared memory
    int *dst_smem = cluster.map_shared_rank(smem, dst_block_rank);

    //Perform atomic update of the histogram bin
    atomicAdd(dst_smem + dst_offset, 1);
}

// cluster synchronization is required to ensure all distributed shared
// memory operations are completed and no thread block exits while
// other thread blocks are still accessing distributed shared memory
cluster.sync();

// Perform global memory histogram, using the local distributed memory
→histogram
int *lbins = bins + cluster.block_rank() * bins_per_block;
for (int i = threadIdx.x; i < bins_per_block; i += blockDim.x)
{
    atomicAdd(&lbins[i], smem[i]);
}
}

```

The above kernel can be launched at runtime with a cluster size depending on the amount of distributed shared memory required. If the histogram is small enough to fit in shared memory of just one block, the user can launch the kernel with cluster size 1. The code snippet below shows how to launch a cluster kernel dynamically based on shared memory requirements.

C++

```
// Launch via extensible launch
{
    cudaLaunchConfig_t config = {0};
    config.gridDim = array_size / threads_per_block;
    config.blockDim = threads_per_block;

    // cluster_size depends on the histogram size.
    // ( cluster_size == 1 ) implies no distributed shared memory, just thread
    ↪block local shared memory
    int cluster_size = 2; // size 2 is an example here
    int nbins_per_block = nbins / cluster_size;

    //dynamic shared memory size is per block.
    //Distributed shared memory size = cluster_size * nbins_per_block *
    ↪sizeof(int)
    config.dynamicSmemBytes = nbins_per_block * sizeof(int);

    CUDA_CHECK(cudaFuncSetAttribute((void *)clusterHist_kernel,
    ↪cudaFuncAttributeMaxDynamicSharedMemorySize, config.dynamicSmemBytes));

    cudaLaunchAttribute attribute[1];
    attribute[0].id = cudaLaunchAttributeClusterDimension;
    attribute[0].val.clusterDim.x = cluster_size;
    attribute[0].val.clusterDim.y = 1;
    attribute[0].val.clusterDim.z = 1;

    config.numAttrs = 1;
    config.attrs = attribute;

    cudaLaunchKernelEx(&config, clusterHist_kernel, bins, nbins, nbins_per_
    ↪block, input, array_size);
}
```

2.3.4. Memory Performance

Ensuring proper memory usage is key to achieving high performance in CUDA kernels. This section discusses some general principles and examples for achieving high memory throughput from global memory and shared memory in CUDA kernels. Global memory performance is a top performance consideration for most kernels. Shared memory is commonly used when threads operate on data they did not explicitly load or create, and so understanding performance-related characteristics of shared memory is also important.

The following subsections will illustrate the important aspects of memory access by progressively improving access to both global and shared memory in a sample matrix transpose kernel.

2.3.4.1 Coalesced Global Memory Access

Global memory is accessed via 32-byte memory transactions. When a CUDA thread requests a word of data from global memory, the relevant warp coalesces the memory requests from all the threads in that warp into the number of memory transactions necessary to satisfy the request, depending on the size of the word accessed by each thread and the distribution of the memory addresses across

the threads. For example, if a thread requests a 4-byte word, the actual memory transaction the warp will generate to global memory will be 32 bytes in total. To use the memory system most efficiently, the warp should use all the memory that is fetched in a single memory transaction. That is, if a thread is requesting a 4-byte word from global memory, and the transaction size is 32 bytes, if other threads in that warp can use other 4-byte words of data from that 32-byte request, this will result in the most efficient use of the memory system.

As a simple example, if consecutive threads in the warp request consecutive 4-byte words in memory, then the warp will request 128 bytes of memory total, and this 128 bytes required will be fetched in four 32-byte memory transactions. This results in 100% utilization of the memory transactions by threads in the warp. [Figure 12](#) illustrates this example of perfectly coalesced memory access.

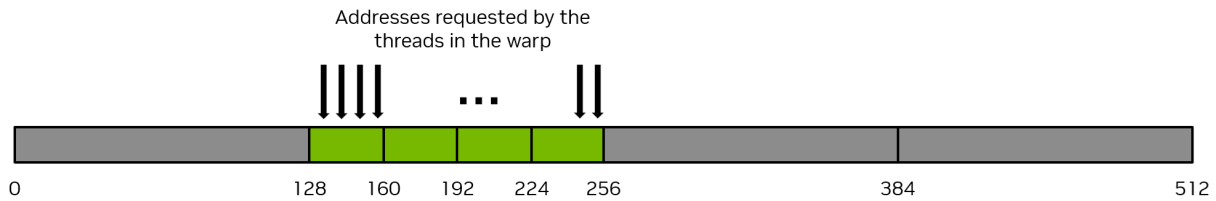


Figure 12: Coalesced memory access

Conversely, the pathologically worst case scenario is when consecutive threads access data elements that are 32 bytes or more apart from each other in memory. In this case, the warp will be forced to issue a 32-byte memory transaction for each thread, and the total number of bytes of memory traffic will be 32 bytes times 32 threads/warp = 1024 bytes. However, the amount of memory used will be 128 bytes only (4 bytes for each thread in the warp), so the memory utilization will only be $128 / 1024 = 12.5\%$. This is a very inefficient use of the memory system. [Figure 13](#) illustrates this example of uncoalesced memory access.

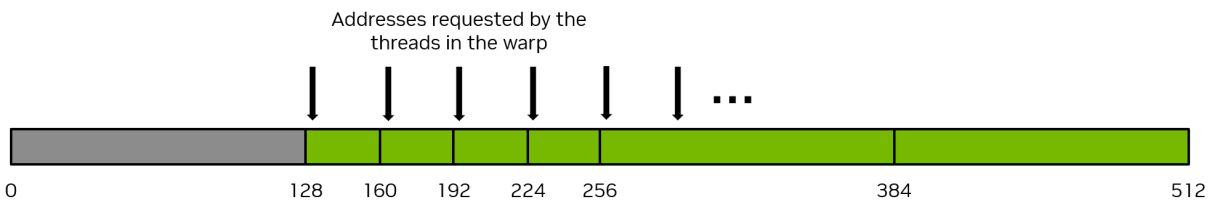


Figure 13: Uncoalesced memory access

The most straightforward way to achieve coalesced memory access is for consecutive threads to access consecutive elements in memory. For example, for a kernel launched with 1d thread blocks, the previously shown vector add kernels will achieve coalesced memory access. Notice how a thread accesses the three arrays: and consecutive threads (indicated by consecutive values of `workIndex`) access consecutive elements in the arrays.

C++

```
__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
```

(continues on next page)

(continued from previous page)

```

    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}

```

Python

```

## Defines a CUDA kernel to perform C = A + B vector addition
@cuda.jit
def vecadd(A, B, C):
    work_index = cuda.grid(1)
    C[work_index] = A[work_index] + B[work_index]

```

There is no requirement that consecutive threads access consecutive elements of memory to achieve coalesced memory access, it is merely the typical way coalescing is achieved. Coalesced memory access occurs provided different threads in the warp access elements from the same 32-byte segments of memory in some linear or permuted way. Stated another way, the best way to achieve coalesced memory access is to maximize the ratio of bytes used to bytes transferred.

An equivalent way to conceptualize global memory coalescing is to consider how many global memory transactions are required to satisfy the 32 addresses requested by a single load instruction from a single warp. In the best possible case, a single global memory transaction will be required to satisfy all loads. For perfectly coalesced access to 4-byte data elements, 4 global memory transactions will be required. In the worst case, 32 global memory transactions may be required to satisfy the addresses requested by a single load instruction from a single warp. In general, performance will be best when the number of global memory transactions required to satisfy a load is as small as possible.

Note

Ensuring proper coalescing of global memory accesses is one of the most important performance considerations for writing performant CUDA kernels. It is imperative that applications use the memory system as efficiently as possible.

2.3.4.1.1 Matrix Transpose Example Using Global Memory

As a simple example, consider an out-of-place matrix transpose kernel that transposes a 32 bit float square matrix of size $N \times N$, from matrix *a* to matrix *c*. This example uses a 2d grid, and assumes a launch of 2d thread blocks of size 32×32 threads, that is, `blockDim.x = 32` and `blockDim.y = 32`, so each 2d thread block will operate on a 32×32 tile of the matrix. Each thread operates on a unique element of the matrix, so no explicit synchronization of threads is necessary. [Figure 14](#) illustrates this matrix transpose operation. The kernel source code follows the figure.

C++

```

/* macro to index a 1D memory array with 2D indices in row-major order */
/* ld is the leading dimension, i.e. the number of columns in the matrix */

#define INDX( row, col, ld ) ( ( (row) * (ld) ) + (col) )

```

(continues on next page)

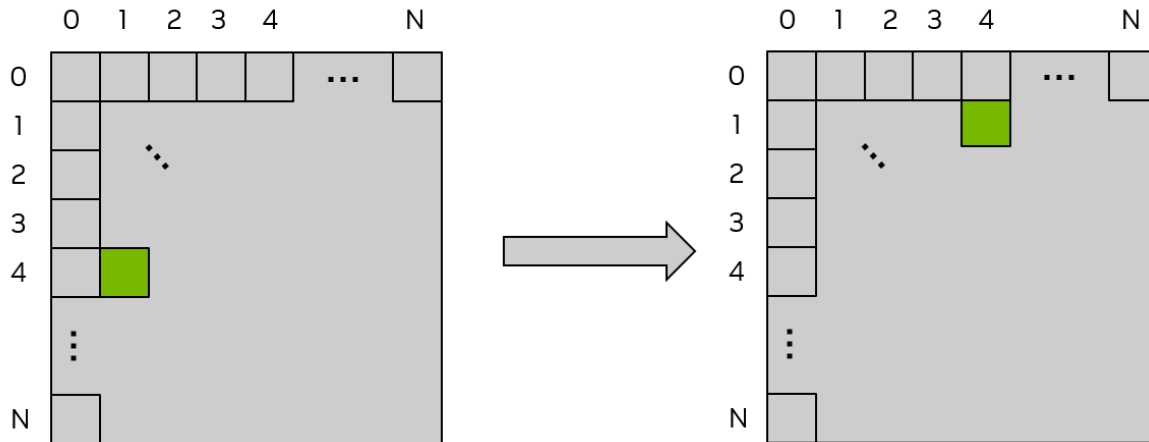


Figure 14: Matrix Transpose using Global memory

The labels on the top and left of each matrix are the 2d thread block indices and also can be considered the tile indices, where each small square indicates a tile of the matrix that will be operated on by a 2d thread block. In this example, the tile size is 32 x 32 elements, so each of the small squares represents a 32 x 32 tile of the matrix. The green shaded square shows the location of an example tile before and after the transpose operation.

(continued from previous page)

```
/* CUDA kernel for matrix transpose */
__global__ void cuda_transpose(int m, float *a, float *c )
{
    int myCol = blockDim.x * blockIdx.x + threadIdx.x;
    int myRow = blockDim.y * blockIdx.y + threadIdx.y;

    if( myRow < m && myCol < m )
    {
        c[INDX( myCol, myRow, m )] = a[INDX( myRow, myCol, m )];
    } /* end if */
    return;
} /* end cuda_transpose */
```

Python

```
import numpy as np
from numba import cuda
import cupy as cp

## Matrix transpose kernel, one thread per matrix element launched with
## 2D thread block on 2D grid to match matrix size
@cuda.jit
def transpose(a, c):
    col = cuda.blockDim.x * cuda.blockIdx.x + cuda.threadIdx.x
```

(continues on next page)

(continued from previous page)

```
row = cuda.blockDim.y * cuda.blockIdx.y + cuda.threadIdx.y
c[(col,row)] = a[(row,col)]
```

To determine whether this kernel is achieving coalesced memory access one needs to determine whether consecutive threads are accessing consecutive elements of memory. In a 2d thread block, the x index moves the fastest, so consecutive values of `threadIdx.x` should be accessing consecutive elements of memory. `threadIdx.x` appears in `myCol`, and one can observe that when `myCol` is the second argument to the `INDX` macro, consecutive threads are reading consecutive values of `a`, so the read of `a` is perfectly coalesced.

However, the writing of `c` is not coalesced, because consecutive values of `threadIdx.x` (again examine `myCol`) are writing elements to `c` that are `ld` (leading dimension) elements apart from each other. This is observed because now `myCol` is the first argument to the `INDX` macro, and as the first argument to `INDX` increments by 1, the memory location changes by `ld`. When `ld` is larger than 32 (which occurs whenever the matrix sizes are larger than 32), this is equivalent to the pathological case shown in [Figure 13](#).

To alleviate these uncoalesced writes, the use of shared memory can be employed, which will be described in the next section.

2.3.4.2 Shared Memory Access Patterns

Shared memory has 32 banks that are organized such that successive 32-bit words map to successive banks. Each bank has a bandwidth of 32 bits per clock cycle.

When multiple threads in the same warp attempt to access different elements in the same bank, a bank conflict occurs. In this case, the access to the data in that bank will be serialized until the data in that bank has been obtained by all the threads that have requested it. This serialization of access results in a performance penalty.

The two exceptions to this scenario happen when multiple threads in the same warp are accessing (either reading or writing) the same shared memory location. For read accesses, the word is broadcast to the requesting threads. For write accesses, each shared memory address is written by only one of the threads (which thread performs the write is undefined).

[Figure 15](#) shows some examples of strided access. The red box inside the bank indicates a unique location in shared memory.

[Figure 16](#) shows some examples of memory read accesses that involve the broadcast mechanism. The red box inside the bank indicates a unique location in shared memory. If multiple arrows point to the same location, the data is broadcast to all threads that requested it.

Note

Avoiding bank conflicts is an important performance consideration for writing performant CUDA kernels that use shared memory.

2.3.4.2.1 Matrix Transpose Example Using Shared Memory

In the previous example [Matrix Transpose Example Using Global Memory](#), an implementation of matrix transpose was illustrated that was functionally correct, but not optimized for efficient use of global memory because the write of the `c` matrix was not coalesced properly. In this example, shared memory

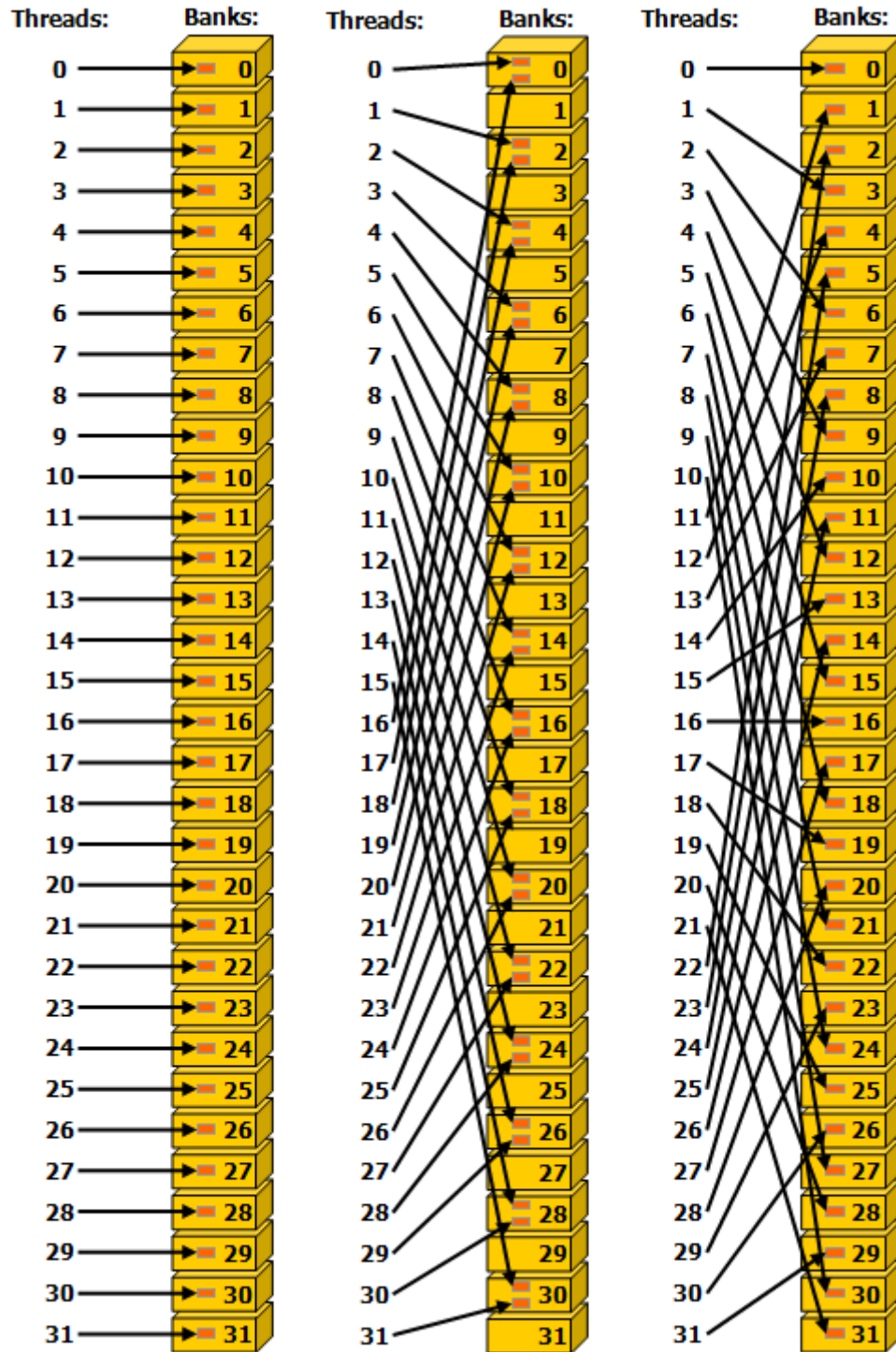


Figure 15: Strided Shared Memory Accesses in 32 bit bank size mode.

Left

Linear addressing with a stride of one 32-bit word (no bank conflict).

Middle

Linear addressing with a stride of two 32-bit words (two-way bank conflict).

Right

Linear addressing with a stride of three 32-bit words (no bank conflict).

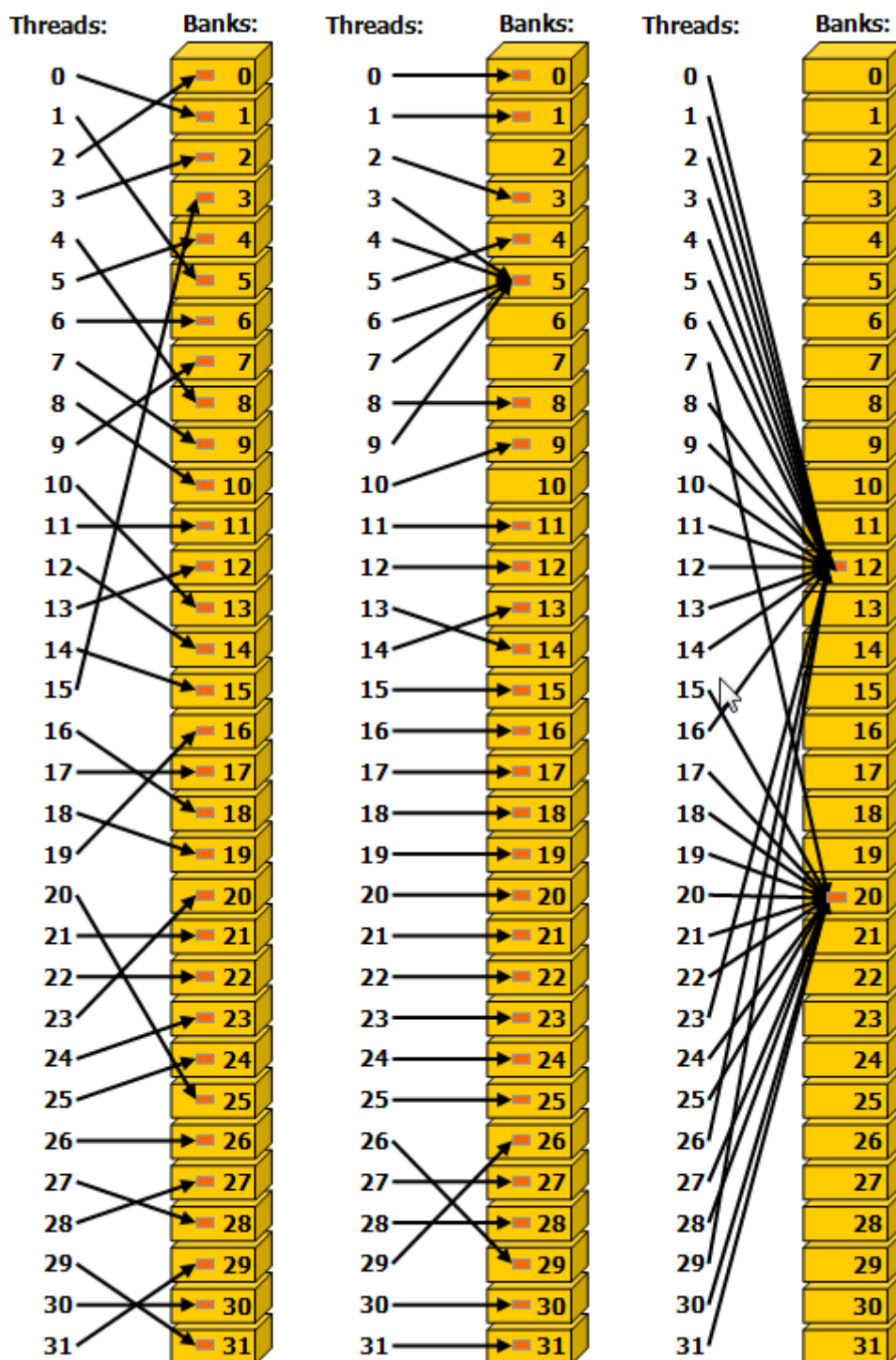


Figure 16: Irregular Shared Memory Accesses.

Left

Conflict-free access via random permutation.

Middle

Conflict-free access since threads 3, 4, 6, 7, and 9 access the same word within bank 5.

Right

Conflict-free broadcast access (threads access the same word within a bank).

will be treated as a user-managed cache to stage loads and stores from global memory, resulting in coalesced global memory access of both reads and writes.

C++

```
#define THREADS_PER_BLOCK_X 32
#define THREADS_PER_BLOCK_Y 32

/* macro to index a 1D memory array with 2D indices in column-major order */
/* ld is the leading dimension, i.e. the number of rows in the matrix */

#define INDX( row, col, ld ) ( ( (col) * (ld) ) + (row) )

/* CUDA kernel for shared memory matrix transpose */
__global__ void smem_transpose(int m,
                              float *a,
                              float *c )
{
    /* declare a statically allocated shared memory array */
    __shared__ float smemArray[THREADS_PER_BLOCK_X][THREADS_PER_BLOCK_Y];

    /* determine my row and column indices for the error checking code */

    const int myRow = blockDim.x * blockIdx.x + threadIdx.x;
    const int myCol = blockDim.y * blockIdx.y + threadIdx.y;

    /* determine my row tile and column tile index */

    const int tileX = blockDim.x * blockIdx.x;
    const int tileY = blockDim.y * blockIdx.y;

    if( myRow < m && myCol < m )
    {
        /* read from global memory into shared memory array */
        smemArray[threadIdx.x][threadIdx.y] = a[INDX( tileX + threadIdx.x,
→tileY + threadIdx.y, m )];
    } /* end if */

    /* synchronize the threads in the thread block */
    __syncthreads();

    if( myRow < m && myCol < m )
    {
        /* write the result from shared memory to global memory */
        c[INDX( tileY + threadIdx.x, tileX + threadIdx.y, m )] =
→smemArray[threadIdx.y][threadIdx.x];
    } /* end if */
    return;
}
```

Python

```

import numpy as np
from numba import cuda
import cupy as cp

## Matrix transpose kernel, one thread per matrix element launched with
## 2D thread block on 2D grid to match matrix size
## Stages input to shared memory
@cuda.jit
def smem_transpose(a, c):
    smemArray = cuda.shared.array(shape=(32, 32), dtype=np.float32)

    tile_col = cuda.blockDim.x * cuda.blockIdx.x
    tile_row = cuda.blockDim.y * cuda.blockIdx.y

    smemArray[(cuda.threadIdx.x, cuda.threadIdx.y)] = a[(tile_row + cuda.
→threadIdx.y, tile_col + cuda.threadIdx.x)]

    cuda.syncthreads()

    c[(tile_col + cuda.threadIdx.y, tile_row + cuda.threadIdx.x)] =
→smemArray[(cuda.threadIdx.y, cuda.threadIdx.x)]

```

The fundamental performance optimization illustrated in this example is to ensure that when accessing global memory, the memory accesses are coalesced properly. Prior to the execution of the copy, each thread computes its `tileRow` and `tileCol` indices. These are the indices for the specific tile that will be operated on, and these tile indices are based on which thread block is executing. Each thread in the same thread block has the same `tileRow` and `tileCol` values, so it can be thought of as the starting position of the tile that this specific thread block will operate on.

The kernel then proceeds with each thread block copying a 32 x 32 tile of the matrix from global memory to shared memory with the following statement. Since the size of a warp is 32 threads, this copy operation will be executed by 32 warps, with no guaranteed order between the warps.

C++

```

smemArray[threadIdx.x][threadIdx.y] = a[INDX( tileRow + threadIdx.y, tileCol
→+ threadIdx.x, m )];

```

Python

```

smemArray[(cuda.threadIdx.x, cuda.threadIdx.y)] = a[(tile_row + cuda.
→threadIdx.y, tile_col + cuda.threadIdx.x)]

```

Note that because `threadIdx.x` appears in the second argument to `INDX`, consecutive threads are accessing consecutive elements in memory, and the read of `a` is perfectly coalesced. In Python, this using `cuda.threadIdx.x` as the last index in the tuple has the same effect: access to `a` is perfectly coalesced.

The next step in the kernel is the call to `__syncthreads()` / `cuda.syncthreads()`. This ensures

that all threads in the thread block have completed their execution of the previous code before proceeding and therefore that the write of `a` into shared memory is completed before the next step. This is critically important because the next step will involve threads reading from shared memory. Without the `__syncthreads()`/`cuda.syncthreads()` call, the read of `a` into shared memory is not guaranteed to be completed by all warps in the thread block before some warps advance to reading from the array in shared memory. When a thread processes or stores data it did not load, synchronization is necessary to ensure that the load operation for that element has completed before accessing it.

At this point in the kernel, for each thread block, the shared memory array has a 32 x 32 tile of the matrix, arranged in the same order as the original matrix. To ensure that the elements within the tile are transposed properly, `threadIdx.x` and `threadIdx.y` are swapped when they read from `smemArray`. To ensure that the overall tile is placed in the correct place in `c`, the `tileRow` and `tileCol` indices are also swapped when they write to `c`. To ensure proper coalescing, `threadIdx.x` is used in the second argument to `INDX`, as shown by the statement below. In Python, this is again achieved by using `cuda.threadIdx.x` in the last index of the matrix index tuple.

C++

```
c[INDX( tileCol + threadIdx.y, tileRow + threadIdx.x, m )] =
    ↪ smemArray[threadIdx.y][threadIdx.x];
```

Python

```
c[(tile_col + cuda.threadIdx.y, tile_row + cuda.threadIdx.x)] =
    ↪ smemArray[(cuda.threadIdx.y, cuda.threadIdx.x)]
```

This kernel illustrates two common uses of shared memory.

- ▶ Shared memory is used to stage data from global memory to ensure that reads from and writes to global memory are both coalesced properly.
- ▶ Shared memory is used to allow threads in the same thread block to share data among themselves.

2.3.4.2.2 Shared Memory Bank Conflicts

In [Section 2.3.4.2](#), the bank structure of shared memory was described. In the previous matrix transpose example, the proper coalesced memory access to/from global memory was achieved, but no consideration was given to whether shared memory bank conflicts were present. Consider the following 2d shared memory declaration,

C++

```
__shared__ float smemArray[32][32];
```

Python

```
from numba import cuda
import numpy as np

smemArray = cuda.shared.array(shape=(32,32), dtype=np.float32)
```

Assume that the kernel is expected to be launched with a 2-d threadblock of 32 x 32 threads. Since a warp is 32 threads, each thread in a particular warp will have a fixed value for `threadIdx.y` and will have $0 \leq \text{threadIdx.x} < 32$.

The left panel of Figure 17 illustrates the situation when the threads in a warp access the data in a column of `smemArray`. Warp 0 is accessing memory locations `smemArray[0][0]` through `smemArray[31][0]` (`smemArray[(0,0)]` through `smemArray[(31,0)]` in Python). In both C++ and Python, multi-dimensional array ordering, the last index moves the fastest, so consecutive threads in warp 0 are accessing memory locations that are 32 elements apart. As illustrated in the figure, the colors denote the banks, and this access down the entire column by warp 0 results in a 32-way bank conflict.

The right panel of Figure 17 illustrates the situation when the threads in a warp access the data across a row of `smemArray`. Warp 0 is accessing memory locations `smemArray[0][0]` through `smemArray[0][31]` (`smemArray[(0,0)]` through `smemArray[(0,31)]` in Python). In this case, consecutive threads in warp 0 are accessing memory locations that are adjacent. As illustrated in the figure, the colors denote the banks, and this access across the entire row by warp 0 results in no bank conflicts. The ideal scenario is for each thread in a warp to access a shared memory location with a different color.

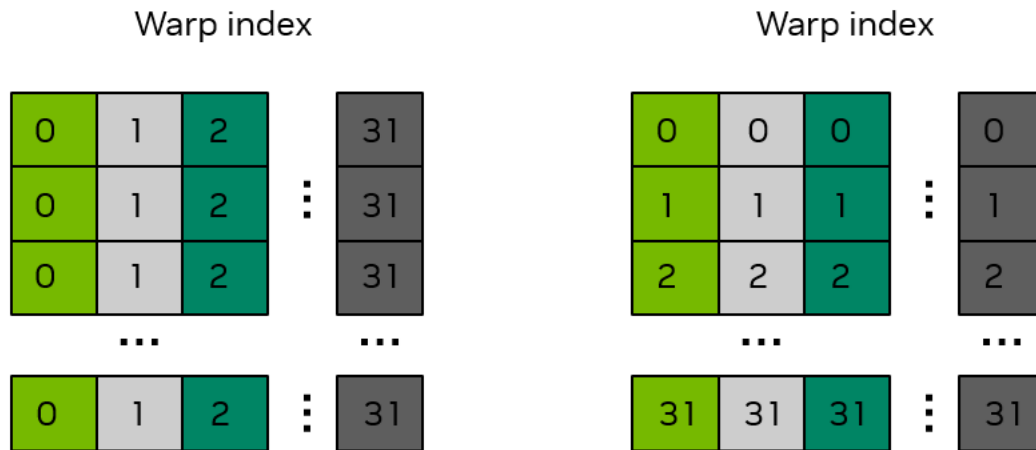


Figure 17: Bank structure in a 32 x 32 shared memory array.

The numbers in the boxes indicate the warp index. The colors indicate which bank is associated with that shared memory location.

Returning to the example from Section 2.3.4.2.1, one can examine the usage of shared memory to determine whether bank conflicts are present. The first usage of shared memory is when data from global memory is stored to shared memory:

C++

```
smemArray[threadIdx.x][threadIdx.y] = a[INDX( tileRow + threadIdx.y, tileCol
↪+ threadIdx.x, m )];
```

Python

```
smemArray[(cuda.threadIdx.x, cuda.threadIdx.y)] = a[(tile_row + cuda.
↪threadIdx.y, tile_col + cuda.threadIdx.x)]
```

Because arrays are stored in row-major order, consecutive threads in the same warp, as indicated by consecutive values of `threadIdx.x`, will access `smemArray` with a stride of 32 elements, because `threadIdx.x` is the first index into the array. This results in a 32-way bank conflict and is illustrated by the left panel of [Figure 17](#).

The second usage of shared memory is when data from shared memory is written back to global memory:

C++

```
c[INDX( tileCol + threadIdx.y, tileRow + threadIdx.x, m )] =
↪smemArray[threadIdx.y][threadIdx.x];
```

Python

```
c[(tile_col + cuda.threadIdx.y, tile_row + cuda.threadIdx.x)] =
↪smemArray[(cuda.threadIdx.y, cuda.threadIdx.x)]
```

In this case, because `threadIdx.x` is the second index into the `smemArray` array, consecutive threads in the same warp will access `smemArray` with a stride of 1 element. This results in no bank conflicts and is illustrated by the right panel of [Figure 17](#).

The matrix transpose kernel as illustrated in [Section 2.3.4.2.1](#) has one access of shared memory that has no bank conflicts and one access that has a 32-way bank conflict. A common fix to avoid bank conflicts is to pad the shared memory by adding one to the column dimension of the array as follows:

C++

```
__shared__ float smemArray[ THREADS_PER_BLOCK_X ][ THREADS_PER_BLOCK_Y+1 ];
```

Python

```
smemArray = cuda.shared.array(shape=(32, 32 + 1), dtype=np.float32)
```

This minor adjustment to the declaration of `smemArray` will eliminate the bank conflicts. To illustrate this, consider [Figure 18](#) where the shared memory array has been declared with a size of 32 x 33. One observes that whether the threads in the same warp access the shared memory array down an entire column or across an entire row, the bank conflicts have been eliminated, i.e., the threads in the same warp access locations with different colors.

2.3.5. Atomics

Performant CUDA kernels rely on expressing as much algorithmic parallelism as possible. The asynchronous nature of GPU kernel execution requires that threads operate as independently as possible. It's not always possible to have complete independence of threads and as we saw in [Shared Memory](#), there exists a mechanism for threads in the same thread block to exchange data and synchronize.

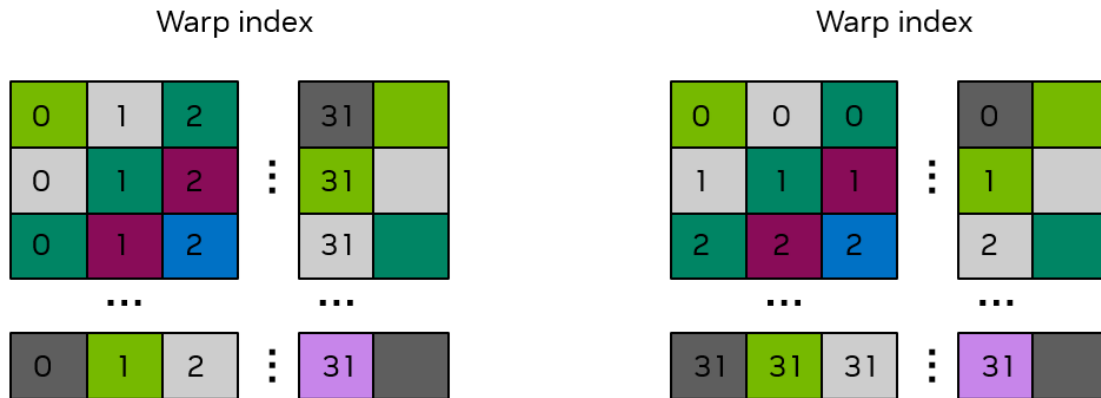


Figure 18: Bank structure in a 32 x 33 shared memory array.

The numbers in the boxes indicate the warp index. The colors indicate which bank is associated with that shared memory location.

On the level of an entire grid there is no such mechanism to synchronize all threads in a grid. There is however a mechanism to provide synchronous access to global memory locations via the use of atomic functions. Atomic functions allow a thread to obtain a lock on a global memory location and perform a read-modify-write operation on that location. No other thread can access the same location while the lock is held.

2.3.5.1 C++ `std::atomic`-like Atomics

In C++, CUDA provides similar syntax and behavior as the similarly-named C++ standard library atomics `cuda::std::atomic` and `cuda::std::atomic_ref`. CUDA also provides extended C++ atomics `cuda::atomic` and `cuda::atomic_ref` which allow the user to specify the *thread scope* of the atomic operation. The details of atomic functions are covered in *Atomic Functions*.

An example usage of `cuda::atomic_ref` to perform a device-wide atomic addition is as follows, where `array` is an array of floats, and `result` is a float pointer to a location in global memory which is the location where the sum of the array will be stored.

C++

```
__global__ void sumReduction(int n, float *array, float *result) {
    ...
    tid = threadIdx.x + blockIdx.x * blockDim.x;

    cuda::atomic_ref<float, cuda::thread_scope_device> result_ref(result);
    result_ref.fetch_add(array[tid]);
    ...
}
```

Atomic functions should be used sparingly as they enforce thread synchronization that can impact performance.

2.3.5.2 Memory Atomics in Python

In Python, atomic memory operations are provided by functions available to GPU code in the `numba.cuda.atomic` namespace. Typical operations such as `add`, `sub`, `max`, `min`, and `compare_and_swap` are available. The full list of supported atomic operations is available in [the Numba CUDA documentation](#).

The following code shows an example of a kernel using atomic memory access to compute the sum of all values in an array. Each thread block loads a slice of the array into shared memory. A single thread of each thread block computes a local sum and performs an atomic add to the result array, `s`. Because the data resides in shared memory, which is close to the compute resources of the SM, having a single thread perform the sum is usually still reasonably performant.

Python

```
import numpy as np
from numba import cuda
import cupy as cp

@cuda.jit
def sum_reduce(a, s):
    ## create a shared array to support a block size up to 512 threads
    ## even though we'll use fewer in this example
    shared_staging = cuda.shared.array(shape=512, dtype=np.float32)

    ## Load values into shared memory and then synchronize to make sure all
    → loads completed
    shared_staging[cuda.threadIdx.x] = a[cuda.blockIdx.x*cuda.blockDim.x +
    → cuda.threadIdx.x]
    cuda.syncthreads()

    ## only thread 0 of each block does the local additions, followed by a
    → single
    ## atomic operation per thread block
    local_sum = float(0.0)
    if cuda.threadIdx.x == 0:
        for i in range(cuda.blockDim.x):
            local_sum = local_sum + shared_staging[i]
            cuda.atomic.add(s, 0, local_sum)

array_length = 2**18

a = cp.ones(array_length)
s = cp.zeros(1, dtype=np.float32)

block_size = 256
grid_size = int(array_length/block_size)
sum_reduce[grid_size, block_size](a, s)

s_host = cp.asnumpy(s)
print(f"Sum is {int(s_host[0])}, expected {array_length}")
```

In this simple example, the input array is all 1s, and so the correct sum is the same as the ar-

ray_length.

If the line

```
cuda.atomic.add(s, 0, local_sum)
```

were instead the non-atomic add

```
s[0] = s[0] + local_sum
```

access to `s[0]` would not be atomic and the final value of `s[0]` will be less than `array_length`. Further, that value may change from run to run and when run on GPUs with different numbers of SMs. This illustrates how atomic memory access is required for correctness in this code.

Note

This example, while functional, is not intended to illustrate how to write peak-performance reduction operations on the GPU. The [CUDA Core Compute Libraries \(CCCL\)](#) from NVIDIA provide high-performance primitives for many operations, including reductions. For both productivity and performance, developers should prefer to use these highly-tuned implementations over reimplementing the same algorithms whenever possible. These primitives are available in Python through the [cuda.coop package](#).

Similar atomics are also available in C++, which are discussed in [Section 5.4.5.1](#), but use of the `std::atomic`-like atomics is recommended and considered best practice in CUDA C++.

2.3.6. Cooperative Groups

Cooperative groups is a software tool available in CUDA C++ that allows applications to define groups of threads which can synchronize with each other, even if that group of threads spans multiple thread blocks, multiple grids on a single GPU, or even across multiple GPUs. The CUDA programming model in general allows threads within a thread block or thread block cluster to synchronize efficiently, but does not provide a mechanism for specifying thread groups smaller than a thread block or cluster. Similarly, the CUDA programming model does not provide mechanisms or guarantees that enable synchronization across thread blocks.

Cooperative groups provide both of these capabilities through software. Cooperative groups allows the application to create thread groups that cross the boundary of thread blocks and clusters, though doing so comes with some semantic limitations and performance implications which are described in detail in the [feature section covering cooperative groups](#).

2.3.7. Kernel Launch and Occupancy

When a CUDA kernel is launched, CUDA threads are grouped into thread blocks and a grid based on the execution configuration specified at kernel launch. Once the kernel is launched, the scheduler assigns thread blocks to SMs. The details of which thread blocks are scheduled to execute on which SMs cannot be controlled or queried by the application and no ordering guarantees are made by the scheduler, so programs cannot rely on a specific scheduling order or scheme for correct execution.

The number of blocks that can be scheduled on an SM depends on the hardware resources a given thread block requires, and the hardware resources available on the SM. When a kernel is first launched, the scheduler begins assigning thread blocks to SMs. As long as SMs have sufficient hardware resources unoccupied by other thread blocks, the scheduler will continue assigning thread blocks to

SMs. If at some point no SM has the capacity to accept another thread block, the scheduler will wait until the SMs complete previously assigned thread blocks. Once this happens, SMs are free to accept more work, and the scheduler assigns thread blocks to them. This process continues until all thread blocks have been scheduled and executed.

The `cudaGetDeviceProperties` function allows an application to query the limits of each SM via [device properties](#). Note that there are limits per SM and per thread block.

- ▶ `maxBlocksPerMultiProcessor`: The maximum number of resident blocks per SM.
- ▶ `sharedMemPerMultiprocessor`: The amount of shared memory available per SM in bytes.
- ▶ `regsPerMultiprocessor`: The number of 32-bit registers available per SM.
- ▶ `maxThreadsPerMultiProcessor`: The maximum number of resident threads per SM.
- ▶ `sharedMemPerBlock`: The maximum amount of shared memory that can be allocated by a thread block in bytes.
- ▶ `regsPerBlock`: The maximum number of 32-bit registers that can be allocated by a thread block.
- ▶ `maxThreadsPerBlock`: The maximum number of threads per thread block.

The occupancy of a CUDA kernel is the ratio of the number of active warps to the maximum number of active warps supported by the SM. In general, it's a good practice to have occupancy as high as possible which hides latency and increases performance.

To calculate occupancy, one needs to know the resource limits of the SM, which were just described, and one needs to know what resources are required by the CUDA kernel in question. To determine resource usage on a per kernel basis, during program compilation one can use the `--resource-usage` [option](#) to `nvcc`, which will show the number of registers and shared memory required by the kernel.

To illustrate, consider a device such as compute capability 10.0 with the device properties enumerated in [Table 2](#).

Table 2: SM Resource Example

Resource	Value
<code>maxBlocksPerMultiProcessor</code>	32
<code>sharedMemPerMultiprocessor</code>	233472
<code>regsPerMultiprocessor</code>	65536
<code>maxThreadsPerMultiProcessor</code>	2048
<code>sharedMemPerBlock</code>	49152
<code>regsPerBlock</code>	65536
<code>maxThreadsPerBlock</code>	1024

If a kernel was launched as `testKernel<<<512, 768>>>()`, i.e., 768 threads per block, each SM would only be able to execute 2 thread blocks at a time. The scheduler cannot assign more than 2 thread blocks per SM because the `maxThreadsPerMultiProcessor` is 2048. So the occupancy would be $(768 * 2) / 2048$, or 75%.

If a kernel was launched as `testKernel<<<512, 32>>>()`, i.e., 32 threads per block, each SM would not run into a limit on `maxThreadsPerMultiProcessor`, but since the `maxBlocksPerMultiProcessor` is 32, the scheduler would only be able to assign 32 thread blocks to each SM. Since the number of threads in the block is 32, the total number of threads resident on the SM would be 32 blocks * 32

threads per block, or 1024 total threads. Since a compute capability 10.0 SM has a maximum value of 2048 resident threads per SM, the occupancy in this case is $1024 / 2048$, or 50%.

The same analysis can be done with shared memory. If a kernel uses 100KB of shared memory, for example, the scheduler would only be able to assign 2 thread blocks to each SM, because the third thread block on that SM would require another 100KB of shared memory for a total of 300KB, which is more than the 233472 bytes available per SM.

Threads per block and shared memory usage per block are explicitly controlled by the programmer and can be adjusted to achieve the desired occupancy. The programmer has limited control over register usage as the compiler and runtime will attempt to optimize register usage. However the programmer can specify a maximum number of registers per thread block via the `--maxrregcount` [option](#) to `nvcc`. If the kernel needs more registers than this specified amount, the kernel is likely to spill to local memory, which will change the performance characteristics of the kernel. In some cases even though spilling occurs, limiting registers allows more thread blocks to be scheduled which in turn increases occupancy and may result in a net increase in performance.

2.4. Writing Tile Kernels

CUDA Tile provides a different approach to writing GPU kernel code than the single instruction multiple thread (SIMT) model covered in the previous chapters. Tile programming allows programmers to express parallelism differently and leaves the lowest level of parallelism to the compiler and built-in operations. By doing so, tile provides a simpler way to access recent performance features of NVIDIA GPUs such as the [tensor memory accelerator \(TMA\)](#) unit and the tensor cores.

- ▶ CUDA Tile programming is available in Python through the cuTile Python package `cuda.tile`.
- ▶ CUDA Tile C++ is available in the CUDA Toolkit from version 13.3 onward.

The application code surrounding a tile kernel for tasks like allocating device memory, transferring data between host and device, and sequencing kernel launches is identical to what the preceding chapters described for SIMT kernels. A tile kernel operates on global memory allocated with the standard CUDA APIs, and its results are copied back to the host in the same way. The only thing that changes is what the programmer writes code inside the kernel itself.

In a SIMT kernel, the programmer thinks in terms of individual threads: computing a global thread index, loading the thread's elements, performing operations on them, and storing the results. In a tile kernel, the programmer thinks at the level of an entire block: loading a tile of many elements, performing an operation on the whole tile, and storing the result. The compiler takes responsibility for mapping tile operations to the hardware threads of each block, a concern that the SIMT programmer handles explicitly.

This chapter focuses exclusively on that difference: how to write the kernel entry point and the tile operations within it. Every pattern is demonstrated in both CuTile Python (`cuda.tile`) and CUDA Tile C++ (`cuda::tiles`), which share a common compiler backend (CUDA Tile IR) and therefore share the same execution semantics.

By convention the tile API is aliased to `ct` in both languages.

- ▶ `import cuda.tile as ct` in Python
- ▶ `namespace ct = cuda::tiles` in C++

In Python, the tile API lives in the module `cuda.tiles` which is imported as shown above.

In C++, the tile API lives in the `cuda::tiles` namespace, which is exposed by the `cuda_tile.h` header.


```
#include "cuda_tile.h"
namespace ct = cuda::tiles;
```

The `ct.` / `ct::` prefix in the snippets below refers to the tile API in whichever language you're reading.

2.4.1. Kernel and Function Declarations

A tile kernel is the GPU entry point, it executes once per block in the launch grid. A tile function is callable from a tile kernel or another tile function, but is not itself an entry point. As with SIMT kernels, tile kernels cannot be called directly from host code; they must be *launched*.

In CUDA Tile C++:

- ▶ `__tile_global__` is the tile analog of `__global__` and marks a tile kernel entry point
- ▶ `__tile__` is the tile analog of `__device__` and indicates a function that should be compiled for the GPU and be callable from other `__tile__` or `__tile_global__` functions.

Array and scalar parameters are passed the same way as in SIMT kernels. Tile code and SIMT code can coexist: a single `.cu` file can define both `__tile_global__` and `__global__` kernels, and a single host program can launch both.

Note

Currently, `__tile__` functions cannot be called from `__global__` or `__device__` functions. Similarly, `__device__` functions cannot be called from `__tile_global__` or `__tile__` functions. This limitation may be lifted in future versions of CUDA.

In cuTile Python:

- ▶ The `@ct.kernel` decorator marks a function as a tile kernel entry point
- ▶ The `@ct.function` decorator marks a function callable from a tile kernel or another tile function.

In practice, any function called from a kernel is automatically compiled as tile code, so the `@ct.function` decorator is optional. Array parameters accept any device-resident array that exposes DL-Pack or the CUDA Array Interface. For example, PyTorch tensors and CuPy arrays. Scalar parameters are passed directly.

C++

```
#include "cuda_tile.h"

// Tile kernel entry point. Cannot be called directly; must be launched.
__tile_global__ void my_kernel(float* a, float* b, float* c) {
    ...
}

// Tile function. Callable from tile kernels and tile functions.
__tile__ float helper(float x, float y) {
    return x + y;
}
```

Python

```
import cuda.tile as ct

# Tile kernel entry point. Cannot be called directly; must be launched.
@ct.kernel
def my_kernel(a, b, c):
    ...

# Tile function. Callable from tile kernels and tile functions.
# @ct.function is optional, any function called from tile code
# is automatically compiled as tile code.
@ct.function
def helper(x, y):
    return x + y
```

2.4.2. Launching Kernels

A tile kernel is launched on a grid of tile blocks, just as a SIMT kernel is launched on a grid of thread blocks. The programmer specifies the grid shape, up to three dimensions. From the programmer's perspective, each tile block is executed by a single logical thread. Parallelism within the block is managed by the compiler.

In C++, tile kernels reuse the familiar triple-chevron launch syntax from SIMT. The first chevron argument is the grid shape (number of tile blocks). The second is the per-block thread count from SIMT; for tile kernels the compiler determines the thread count internally, and the second argument **must be 1**. A tile kernel is also an ordinary CUDA kernel, so it can be launched through the runtime's existing APIs `cudaLaunchKernel` and `cudaLaunchKernelEx` with the same `grid, 1` configuration. This is useful when integrating tile kernels into a codebase that already drives launches through those APIs.

In Python, `ct.launch` takes four positional arguments: a CUDA stream, a grid tuple specifying the number of tile blocks in each dimension, the kernel object, and a tuple of kernel arguments.

C++

```
my_kernel<<<dim3(num_blocks_x, num_blocks_y), 1>>>(a, b, c); // second arg
↪ must be 1
```

Python

```
import torch

stream = torch.cuda.current_stream() # CUDA stream object
grid = (num_blocks_x, num_blocks_y, 1) # tile-block grid (x, y, z)
ct.launch(stream, grid, my_kernel, (a, b, c))
```

2.4.2.1 Grid-Sizing Pattern

A common pattern is to launch enough blocks to cover a full array, including a final block which potentially exceeds the size of the array in one or more dimensions.

C++

```
int num_blocks = (N + tile_size - 1) / tile_size;    // ceil division ->
↳ covers partial tail
kernel<<<num_blocks, 1>>>(in, out, N);
```

Python

```
import math

grid = (math.ceil(N / TILE),)    # ceil division -> covers partial tail
ct.launch(stream, grid, my_kernel, (arr_in, arr_out, TILE))
```

Handling the case where the size of an array is not perfectly divisible by the size of tiles is discussed in the subsections of [Section 2.4.6](#).

2.4.3. Querying Block Position

Each block needs to know where it sits in the grid so it can determine which portion of the data to process. In SIMT, the programmer combines `blockIdx` and `threadIdx` to compute a global thread index. In tile code, only the block index is needed. The compiler handles all thread-level indexing within the block.

In C++, `ct::bid()` returns a `uint3` containing the block index in all three dimensions. `ct::num_blocks()` returns a `dim3` with the total number of blocks in each dimension (as determined by the kernel launch parameters). Individual components are accessed via `.x`, `.y`, `.z`.

In Python, `ct.bid(axis)` returns the current block's index along the given axis (0, 1, or 2) as an `int32` scalar. `ct.num_blocks(axis)` returns the total number of blocks along that axis – useful for bounds checks and loop counts.

C++

```
#include "cuda_tile.h"

__tile_global__ void my_kernel(float* a, float* b, float* c) {
    namespace ct = cuda::tiles;
    int bid_x = ct::bid().x;           // block index along .x
    int bid_y = ct::bid().y;           // block index along .y
    int num_x = ct::num_blocks().x;    // total blocks along .x
}
```

Python

```
@ct.kernel
def my_kernel(a, b, c):
    bid_x = ct.bid(0)                 # block index along axis 0
    bid_y = ct.bid(1)                 # block index along axis 1
    num_x = ct.num_blocks(0)          # total blocks along axis 0
```

2.4.4. Creating Tiles

With the block's identity established, the next question is what tile kernels actually operate on. That's the tile: a fixed-size, multidimensional array of scalar elements whose shape and element type are known at compile time. Each dimension of a tile must be a power of two. Tiles have value semantics. This means copying a tile copies its elements and the two copies are fully independent. Despite this, copies are cheap because the compiler controls how tiles are represented internally in hardware. The programmer does not allocate or free memory for tiles.

In practice, tiles are created either by loading data from arrays (*Tile-Space Loads and Stores*) or by using factory functions that produce tiles filled with a specified pattern.

In C++, the tile type is explicit: `ct::tile<T, ct::shape<dims...>>`, where `T` is the element type and `ct::shape<dims...>` encodes the dimensions as template arguments (the integer values are the compile-time sizes along each axis). For example, `ct::tile<float, ct::shape<8>>` is a 1-D tile of 8 floats, and `ct::tile<float, ct::shape<4, 4>>` is a 4×4 float tile. Because the shape is part of the type, it is always known at compile time.

Factory functions take the full tile type (Tile below) as a template parameter:

- `ct::zeros<Tile>()` and `ct::ones<Tile>()` - tiles filled with zeros or ones.
- `ct::full<Tile>(val)` - tile where every element has value `val`.
- `ct::iota<Tile>()` - tile containing `(0, 1, ..., N-1)`, where `N` is the tile's size.

C++ examples throughout this chapter use a using alias (e.g., `using f32x4x4 = ct::tile<float, ct::shape<4, 4>>`) to keep tile types readable at the call site.

In Python, both the shape tuple and the dtype arguments to a tile factory are compile-time values. Python literals (like `(64, 64)` and `ct.float32`) satisfy this naturally. They can also be supplied using Constant-annotated kernel parameters as shown below in *Python Constant[T]*. The resulting tile exposes `.shape`, `.dtype`, and `.ndim` properties that reflect its compile-time attributes.

The factory functions are:

- `ct.zeros(shape, dtype)` and `ct.ones(shape, dtype)` - tiles filled with zeros or ones.
- `ct.full(shape, fill_value, dtype)` - tile with an arbitrary constant value.
- `ct.arange(size, dtype=...)` - 1-D tile containing `[0, 1, ..., size-1]`.

C++

```
#include "cuda_tile.h"

__tile__ void factories() {
    namespace ct = cuda::tiles;

    using i32x8    = ct::tile<int,    ct::shape<8>>;    // 1-D: 8 ints
    using f32x4x4  = ct::tile<float, ct::shape<4, 4>>;    // 2-D: 4x4 floats

    auto z        = ct::zeros<f32x4x4>();              // all zeros
    auto o        = ct::ones<f32x4x4>();                // all ones
    auto filled    = ct::full<f32x4x4>(3.14f);          // all 3.14
    auto seq       = ct::iota<i32x8>();                 // {0, 1, 2, 3, 4, 5, 6, 7}
}
```

Python

```
import cuda.tile as ct

@ct.function
def factories():
    zeros = ct.zeros((64, 64), dtype=ct.float32)           # 64x64 tile of
    ↪ 0.0
    ones = ct.ones((128, ), dtype=ct.float16)              # 128-element
    ↪ tile of 1.0
    filled = ct.full((32, 32), 3.14, dtype=ct.float32)      # 32x32 tile of
    ↪ 3.14
    seq = ct.arange(8, dtype=ct.int32)                      # [0, 1, 2, 3, 4,
    ↪ 5, 6, 7]
```

2.4.5. Compile-Time Constants

The tile compiler generates specialized machine code for each combination of tile shapes, data types, and other structural parameters. Values that influence the generated code must therefore be known at compile time. That is, the shape and data type of the tile must be known at compile time. [Creating Tiles](#) used literals to specify tile shapes and data types: `ct.zeros((64, 64), dtype=ct.float32)` and `ct::tile<int, ct::shape<8>>`.

The shape can also be passed through the kernel interface as compile-time-known values, as shown in the following sections.

2.4.5.1 Python Constant[T]

The `ct.Constant[T]` type hint on a kernel parameter marks it as *constant-embedded*. This means that every use of that parameter inside the kernel behaves as if the literal value were written in its place. The type argument is optional, `ct.Constant` without a type argument embeds a constant of any type. `ct.Constant` is most commonly used on integers, `ct.Constant[int]`, on parameters that drive tile shapes and loop bounds.

Python

```
import cuda.tile as ct

@ct.kernel
def my_kernel(TILE: ct.Constant[int]):
    # TILE is constant-embedded: wherever TILE appears, the compiler sees its
    # literal value (e.g., 128) and generates specialized code. Here TILE
    ↪ drives
    # the shape of a factory-built tile.
    zeros = ct.zeros((TILE, ), dtype=ct.float32)
```

2.4.5.2 C++ `integral_constant` and `_ic` Literals

In CUDA Tile C++, compile-time values are expressed through `ct::integral_constant`, a type whose numeric value is encoded in the type itself. The `_ic` literals from the `ct::literals` namespace provide a concise shorthand: `0_ic` produces a `ct::integral_constant<0>` value.

APIs that take compile-time values accept both the non-type template parameter (NTTP) form and the `_ic` literal form. For example, `ct::cat` concatenates two tiles along a given dimension, and that dimension must be known at compile time. The two lines below call `ct::cat` with the same compile-time axis; they differ only in where the compile-time value is written:

C++

```
#include "cuda_tile.h"

__tile__ void concat_demo() {
    namespace ct = cuda::tiles;
    using namespace ct::literals;

    using T = ct::tile<int, ct::shape<4, 8>>;
    T lhs = ct::full<T>(0);
    T rhs = ct::full<T>(1);

    auto a = ct::cat<0>(lhs, rhs);    // NTTP form
    auto b = ct::cat(lhs, rhs, 0_ic); // _ic form
}
```

There is one other place where `_ic` literals routinely appear. Each of `ct::extents` and `ct::shape` has an NTTP form (for example, `ct::extents<std::uint32_t, 4, 8>`) and a brace form. Unlike the NTTP form, the brace form accepts runtime values, so it is the form you reach for when one or more dimensions are only known at launch: `_ic` literals for the compile-time dims, plain variables for the runtime ones. Tile-space APIs like `ct::tensor_span` and `ct::partition_view` (covered in [Tile-Space Loads and Stores](#)) use this form to wrap such arrays:

C++

```
auto shape2d = ct::extents{8_ic, length}; // 8 is compile-time; length is
↳ runtime
```

`_ic` literals are the uniform shorthand for a compile-time value wherever a value-form API argument calls for one, such as a `ct::cat` dimension or an `extents` or `shape` component.

2.4.6. Loading and Storing Tiles

As [Section 1.2.2.3.1](#) first introduced, there are two key memory objects in the CUDA tile programming model: tiles and arrays. An array is a multidimensional container of elements in global memory visible to all blocks of a tile kernel. A tile is also a multidimensional container of elements, but is local to a single block of CUDA tile code. A tile is often a subset of elements of an array. This section discusses loading from arrays into tiles so they can be used in tile kernels, and storing tiles back to arrays.

Two methods of loading and storing tiles are covered in subsequent sections

- ▶ [Tile-Space Loads and Stores](#) covers loads and stores using tile-space indices that use view objects which prescribe predictable patterns for how the elements of the array map to tiles
- ▶ [Gather and Scatter](#) covers loads and stores which use a tile of indices or pointers to indicate the element of an array which is the source or target of a tile element when loaded or stored, respectively

Performance note: Tile-space loads can be lowered by the compiler to the Tensor Memory Accelerator (TMA) on supported hardware, which is significantly faster than per-element gather. (For the C++ side, see also [C++ Performance Tips](#).)

The programmer must decide what value out-of-bounds elements take on a load. Out-of-bounds writes are silently discarded in Python, and in C++ when the masked variants are used.

2.4.6.1 Tile-Space Loads and Stores

With tile-space loads, a view object is created which specifies how an array is partitioned into a grid of tile-sized regions. This mapping is called the *tile-space*, and a tile kernel can load or stores one region at a time using a tile-space index.

Central to the idea of tile-space loads is the *tiled view* of an array, which specifies how the elements of the array are mapped to tiles of a specified size. The tiled view shown in [Figure 19](#) is a *partition view*, which is a tile-space with non-overlapping tiles of the specified size with no gaps between tiles.

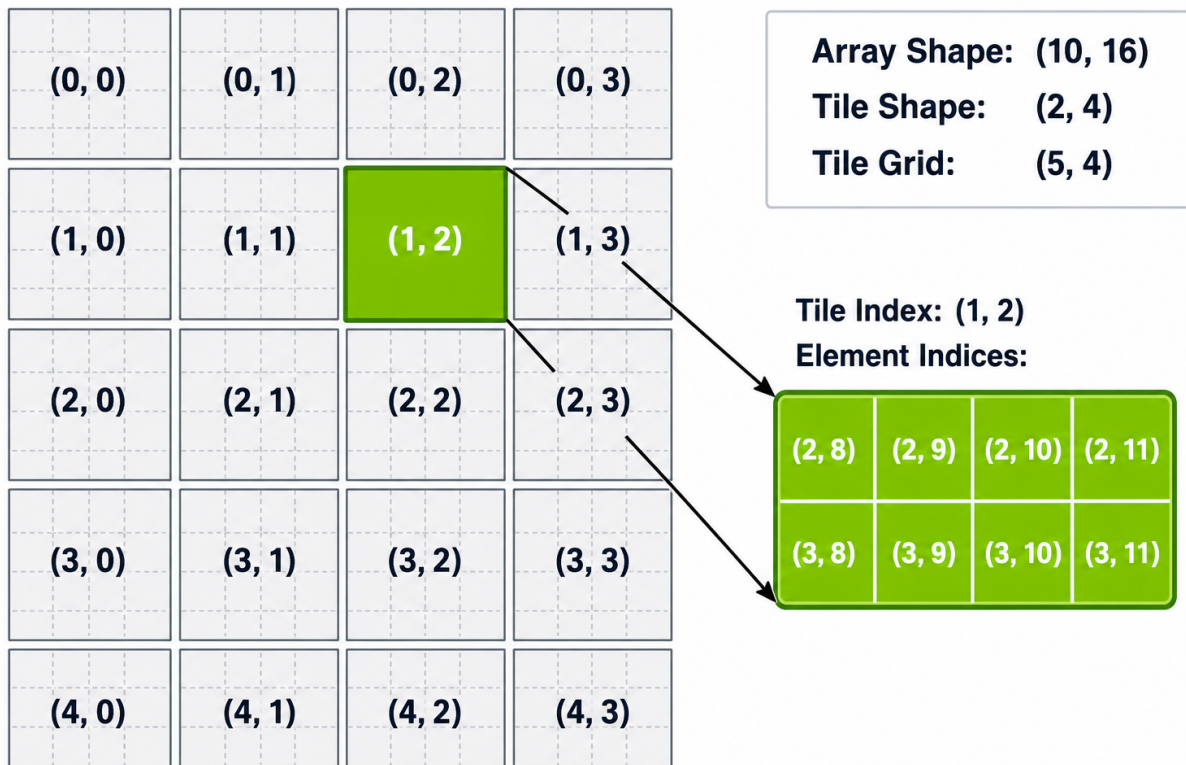


Figure 19: Tile-space indexing for a partition view. A two-dimensional array of shape (10, 16) is partitioned for tiles of shape (2, 4), producing a tile grid of shape (5, 4). Each cell shows its tile-space index (i, j). The highlighted region at tile-space index (1, 2) covers element indices (2, 8) through (3, 11).

When the array dimensions are not perfectly divisible into tiles, tiles which cross the array boundary in one or more dimension will be partially filled. The programmer can specify behavior when loading these tiles, which will be introduced in [Section 2.4.6.1.3](#).

Note

The examples and descriptions here use partition views to illustrate tile-space loads and stores because this was the first view type supported in CUDA Tile code. Other views types are expected to be added in subsequent versions of CUDA Tile.

2.4.6.1.1 Partition View Loads and Stores

The structured tile-space load is the preferred way to move data between global memory and tiles. The kernel must first build a view object that defines the tile-space, then load or store one tile at a time by its tile-space index.

In C++, a partition view is constructed in two steps:

- `ct::tensor_span` - pairs a raw pointer with a `ct::extents`, giving the pointer multidimensional structure.
- `ct::partition_view` - divides the span into a grid of fixed-size tiles and exposes `.load(idx...)` / `.store(tile, idx...)` methods that operate in tile-space coordinates.

In Python, `Array.tiled_view(tile_shape)` returns a `TiledView` that partitions the array into tiles of the given shape. The view exposes `.load(index)` / `.store(index, tile)` methods that take a tile-space index, mirroring C++ `partition_view` directly.

Note

The C++ example code in this chapter annotates pointer arguments with `__restrict__` and calls `ct::assume_aligned(ptr, 16_ic)` near the top of the kernel body. These are important performance annotations covered further in [Section 2.4.12](#). The `_ic` suffix on numeric literals (e.g., `128_ic`, `8_ic`) marks them as compile-time constants, as introduced in [Compile-Time Constants](#).

C++

```
__tile_global__ void vec_add(float* __restrict__ a, float* __restrict__ b,
    ↪ float* __restrict__ out) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;

    a = ct::assume_aligned(a, 16_ic);
    b = ct::assume_aligned(b, 16_ic);
    out = ct::assume_aligned(out, 16_ic);

    // Step 1: attach a shape to each raw pointer. 128_ic marks 128 as a
    ↪ compile-time constant.
    auto aSpan = ct::tensor_span{a, ct::extents{128_ic}};
    auto bSpan = ct::tensor_span{b, ct::extents{128_ic}};
    auto oSpan = ct::tensor_span{out, ct::extents{128_ic}};

    // Step 2: partition each span into a tile space of fixed 8-element tiles.
    auto aView = ct::partition_view{aSpan, ct::shape{8_ic}};
    auto bView = ct::partition_view{bSpan, ct::shape{8_ic}};
    auto oView = ct::partition_view{oSpan, ct::shape{8_ic}};
```

(continues on next page)

(continued from previous page)

```

    int bx      = ct::bid().x;           // this block's tile-space index
    ↪ along .x
    auto aTile = aView.load(bx);         // pick the bx-th tile of a
    auto bTile = bView.load(bx);
    oView.store(aTile + bTile, bx);      // write the tile back at the bx-th
    ↪ position of out
}

```

Python

```

@ct.kernel
def vec_add(a, b, c, TILE: ct.Constant[int]):
    a_view = a.tiled_view((TILE,))
    b_view = b.tiled_view((TILE,))
    c_view = c.tiled_view((TILE,))

    bid = ct.bid(0)
    a_tile = a_view.load((bid,))
    b_tile = b_view.load((bid,))
    c_view.store((bid,), a_tile + b_tile)

```

2.4.6.1.2 Python One-Call Load and Store

Python additionally offers a one-call form that takes the tile shape inline on each load and store, without an explicit view object. `ct.load(array, index, shape)` reads a tile of the given shape at the given tile-space index. `ct.store(array, index, tile)` is the corresponding write.

Both `ct.load/ct.store` and `Array.tiled_view` express the same tile-space access pattern. The difference is where the tile shape lives. With `Array.tiled_view`, the tile shape is bound once to a view object. With `ct.load/ct.store`, the tile shape is supplied inline on each call. Using `tiled_view` is preferred when the same partitioning is reused across multiple loads and stores. Use `ct.load/ct.store` when a single one-off load is more concise.

Python

```

@ct.kernel
def vec_add(a, b, c, TILE: ct.Constant[int]):
    bid = ct.bid(0)                                # this block's tile-
    ↪ space index along axis 0
    a_tile = ct.load(a, index=(bid,), shape=(TILE,)) # (index, shape) =
    ↪ pick the bid-th TILE-sized region of a
    b_tile = ct.load(b, index=(bid,), shape=(TILE,))
    ct.store(c, index=(bid,), tile=a_tile + b_tile) # write the tile back
    ↪ to the bid-th region of c

```

2.4.6.1.3 Tile-Space Boundary Handling

In C++, `partition_view` provides unmasked and masked variants:

- ▶ `.load(idx...)/.store(tile, idx...)` assumes the tile is fully in-bounds. Partially out-of-bounds access is undefined behavior.
- ▶ `.load_masked(idx...)/.store_masked(tile, idx...)` handles partial edge tiles safely.
 - ▶ `.load_masked()` fills out-of-bounds positions with zero by default; alternative padding modes (such as NaN for float tiles) can be selected.
 - ▶ `.store_masked()` silently discards out-of-bounds writes.

Using the unmasked load and store variant is preferred when the array is perfectly divisible by the tile size. When boundary conditions must be handled, the masked variant can be used even for tiles which are fully populated.

This is also the first C++ example in the guide in which an array dimension is a runtime value. `ct::extents{N}` accepts a runtime dimension and `ct::extents` supports any mix of compile-time (`_ic`) and runtime values, so the span and partition view can wrap arrays whose size is known only at kernel launch.

In Python, `ct.load` accepts a `padding_mode` parameter that controls what value out-of-bounds elements receive. Two commonly used modes are:

- ▶ `PaddingMode.ZERO` - out-of-bounds elements are filled with zero.
- ▶ `PaddingMode.UNDETERMINED` (the default) - out-of-bounds element values are left to the implementation. This is appropriate when the programmer knows the tile is fully in-bounds.

For stores, `ct.store` always silently discards writes to out-of-bounds positions, no `padding_mode` parameter is needed. The same rules apply to `tiled_view`, which fixes its `padding_mode` at view creation.

C++

```
__tile_global__ void edge_safe(float* __restrict__ in, float* __restrict__
→out, int N) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;

    in  = ct::assume_aligned(in, 16_ic);
    out = ct::assume_aligned(out, 16_ic);

    // ct::extents{N} uses a runtime dimension; 128_ic stays compile-time.
    auto inView = ct::partition_view{ct::tensor_span{in, ct::extents{N}},
→ct::shape{128_ic}};
    auto outView = ct::partition_view{ct::tensor_span{out, ct::extents{N}},
→ct::shape{128_ic}};

    int bx = ct::bid().x;
    auto tile = inView.load_masked(bx); // masked load: 00B lanes default
→to 0
    outView.store_masked(tile, bx); // masked store: 00B writes
→silently discarded
}
```

Python

```
@ct.kernel
def edge_safe(arr_in, arr_out, TILE: ct.Constant[int]):
    bid = ct.bid(0)
    tile = ct.load(arr_in, index=(bid,), shape=(TILE,),
                  padding_mode=ct.PaddingMode.ZERO) # OOB lanes of a
    ↪partial edge tile become 0
    ct.store(arr_out, index=(bid,), tile=tile)      # OOB writes are
    ↪silently discarded
```

Inside the C++ kernel, `.load_masked()` and `.store_masked()` handle the partial edge tile. Inside the Python kernel, `PaddingMode.ZERO` on the load ensures the partial edge tile is zero-padded, and `ct.store` silently discards writes beyond the array boundary. For the complete set of padding modes, masking options, and padding values, see the API reference for each language ([CUDA Tile C++ view padding](#), [cuTile Python padding modes](#)).

Loading from or storing to a tile that lies entirely outside the array is undefined. The boundary handling discussed here only applies to tiles that are partially out-of-bounds in one or more dimension.

2.4.6.2 Gather and Scatter

The tile-space loads in [Tile-Space Loads and Stores](#) used a partition view, which defines a regular, block-aligned partitioning of the array. When the access pattern is irregular or data-dependent such as a lookup table or a permutation, for example, gather and scatter operations allow a tile to be loaded from and stored to non-uniform and non-contiguous elements of an array. .. by arbitrary index or address.

Gather and scatter operations look slightly different in C++ and Python:

- Python uses integer index tiles passed to `ct.gather()` / `ct.scatter()`, with built-in bounds checking.
- C++ uses tiles of pointers passed to `ct::load()` / `ct::store()`, with masked variants `ct::load_masked()` and `ct::store_masked()` that accept a boolean mask tile to [handle tiles at array boundaries](#).

In C++, gather and scatter work by forming a tile of pointers, one pointer per element, and passing the pointer tile to `ct::load()` or `ct::store()`. Arithmetic between a scalar pointer and an integer tile is performed element-wise, producing a tile of pointers. This is the standard idiom for constructing gather/scatter index tiles in C++.

In Python, `ct.gather` loads the element at each index in the index tile. Bounds checking is on by default: out-of-bounds indices return a padding value (zero by default, configurable via `padding_value=`), and can be disabled with `check_bounds=False`. `ct.scatter` stores one value per index; out-of-bounds writes are silently discarded.

C++

```
__tile_global__ void vec_add_gather(int* __restrict__ a, int* __restrict__ b,
    ↪int* __restrict__ out) {
    namespace ct = cuda::tiles;
    using namespace ct::literals;
    using i32x8 = ct::tile<int, ct::shape<8>>;
```

(continues on next page)

(continued from previous page)

```

a  = ct::assume_aligned(a,  16_ic);
b  = ct::assume_aligned(b,  16_ic);
out = ct::assume_aligned(out, 16_ic);

int bx      = ct::bid().x;
auto offsets = 8 * bx + ct::iota<i32x8>(); // element-level offsets,
→one per lane

// scalar pointer + int tile = tile of pointers (one pointer per offset).
auto aPtrs = a + offsets;
auto bPtrs = b + offsets;

auto aTile = ct::load(aPtrs); // gather: one load per
→pointer
auto bTile = ct::load(bPtrs);
ct::store(out + offsets, aTile + bTile); // scatter: one store per
→pointer
}

```

Python

```

@ct.kernel
def vec_add_gather(a, b, c, TILE: ct.Constant[int]):
    bid = ct.bid(0)
    indices = bid * TILE + ct.arange(TILE, dtype=ct.int32) # one element
    →index per lane

    a_tile = ct.gather(a, indices) # load
    →a[indices[i]] per lane
    b_tile = ct.gather(b, indices)
    ct.scatter(c, indices, a_tile + b_tile) # store one
    →value per index into c

```

2.4.6.2.1 Gather and Scatter Boundary Handling

Boundary handling for the gather/scatter operations introduced in *Gather and Scatter* follows different rules.

In Python, `ct.gather` and `ct.scatter` are bounds-safe by default. Out-of-bounds reads return a padding value (zero by default), and out-of-bounds writes are silently dropped. Bounds checking can be disabled when you can prove every index is in range; doing so makes out-of-bounds access undefined behavior. See the API reference for the optional mask and padding-value knobs ([CUDA Tile C++ load operations](#), [cuTile Python load/store operations](#)).

In C++, bounds checking is not automatic. The programmer constructs a boolean mask (e.g., by comparing offsets against the array length) and passes it to `ct::load_masked` or `ct::store_masked`: